Into the Root







FACULTAD DE
INGENIERIA
UNIVERSIDAD NACIONAL DE JUJUY

Documentación:

Análisis, Diseño y Codificación

Docente:

Ing. Ariel Alejandro Vega - Programación de Videojuegos I

Guillermo Sebastian Buczek

Integrantes:

Gimenez Gutierrez, Awki Jarawi LU: TUV000719





ÍNDICE

pág. 5: Player Spring: movimiento horizontal y salto.

pág. 9: Player Spring: agacharse.

pág. 14: Player Spring: animaciones.

pág. 19: Player Spring: PlyaerStats.

pág. 25: HUD

pág: 29: Level Design

pág. 33: Savia

pág. 38: Game Maneger

pág 41: Mapa

Pág. 45: Player Kill

pág. 50: Level Exit

pág. 54: Menu

pág. 62: Mosquito con Strategy

pág. 69: NPC Merc-Ant

pág. 75: Ataque de raíz

pág. 80: Gusano Nivel 2

pág. 86: Gusano Nivel 3

pág. 95: Bomba Explosiva

pág. 99: Espina

pág. 102: Boss Contoller

pág. 116: Final Level FSM

pág. 130: Menu Pausa



pág. 136: Post Game Scene

pág. 139: Audio Manager

pág. 144: Escena de Introducción



Player Spring

Movimiento Spring

Fase de análisis:

Historia de usuario HU01:

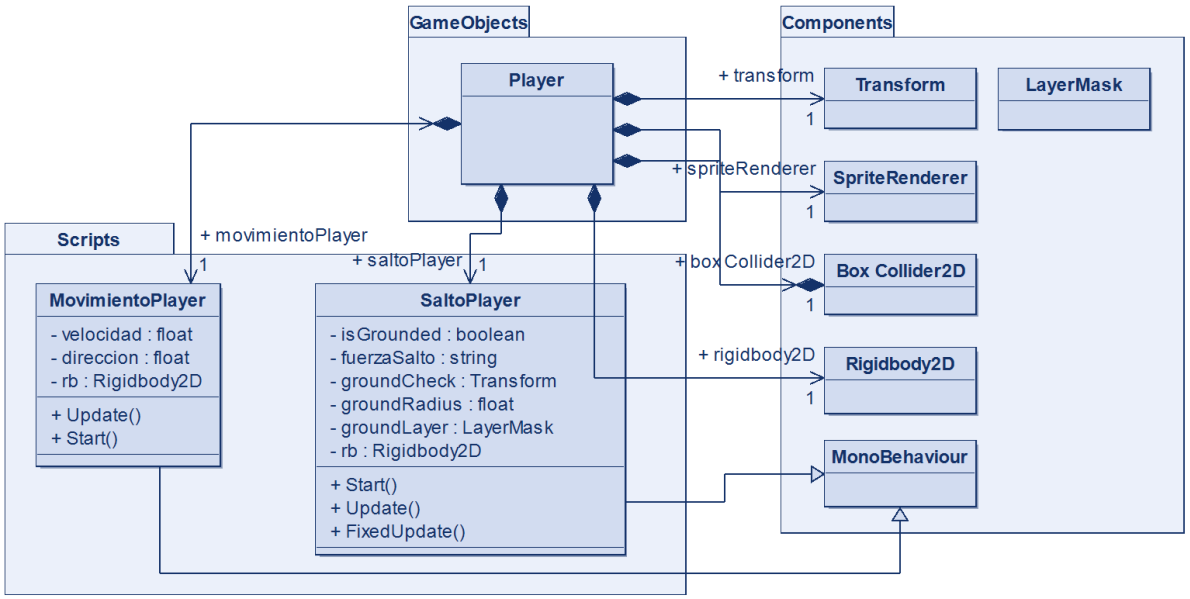
Quién: Como jugador.
Qué: Quiero moverme **horizontalmente** y **saltar** al presionar las teclas correspondientes.
Para qué: Para navegar a través de los niveles de plataformas y superar obstáculos.

Tabla EPS

Entrada	Proceso	Salida
Teclas de movimiento (`Flecha IZQUIERDA`, `Flecha DERECHA`, `ESPACIO`).	Se comprueba el estado del suelo (`isGrounded`). Si se detecta input, se aplica un **desplazamiento** o **fuerza/velocidad** al componente **Rigidbody2D** o **Transform** del GameObject para moverlo o saltar.	**Posición y Velocidad** de Spring (cambiadas).

Fase de diseño:

Diagrama de clases:



Objetivo central: Implementar el movimiento y salto del Player a través de las teclas de izquierda y derecha o 'A' y 'D', y el espacio para el salto.

Lista de Tareas HU01

- **Configuración del Proyecto:** Crear el proyecto 2D y establecer la estructura de carpetas (`Art`, `Audio`, `Scenes`, `Scripts`, etc.).
- **Configuración del GameObject Spring:** Crear el `GameObject` "Spring" con `SpriteRenderer`, `Rigidbody2D` y `Collider2D`.
- **Creación de Scripts:** Crear los scripts C# `MovimientoPlayer` y `SaltoPlayer` en `Assets/Scripts/Components`.
- **Implementación de Movimiento Horizontal:** Implementar la lectura del *input* horizontal y aplicar la velocidad al `Transform` en el script `MovimientoPlayer`.
- **Inicialización de Componentes:** Obtener la referencia al `Rigidbody2D` (`rb2d`) en la función `Start()` de SaltoPlayer.
- **Implementación de Detección de Suelo:** Implementar la lógica (`CheckGround()`) para verificar si Spring está tocando la capa de suelo (`groundLayer`) en el script `PlayerJump`.
- **Implementación de Lógica de Salto:** Implementar la lectura del *input* de salto (`ESPACIO`) en `Update()` y llamar a `Jump()` solo si `isGrounded` es verdadero en el script `PlayerJump`.
- **Asignación de Scripts:** Adjuntar los scripts `MovimientoPlayer` y SaltoPlayer al `GameObject` "Spring" y configurar sus valores públicos en el Inspector.

Criterios de Aceptación HU01



- Al presionar el *input* derecho (`Flecha DERECHA` o `D`), Spring se mueve consistentemente hacia la derecha (velocidad controlada por el parámetro `velocidad` en `MovimientoPlayer`).
- Al presionar el *input* izquierdo (`Flecha IZQUIERDA` o `A`), Spring se mueve consistentemente hacia la izquierda (velocidad controlada por el parámetro `velocidad` en `MovimientoPlayer`).
- Al no haber *input* horizontal, Spring se detiene rápidamente, sin deslizarse.
- Al presionar el *input* de salto (`ESPACIO`) **mientras está tocando el suelo**, se aplica una fuerza vertical inmediata (controlada por `fuerzaSalto` en `SaltoPlayer`).
- El componente `SaltoPlayer` determina con precisión si Spring está "en el suelo" (`isGrounded`) usando la capa de suelo (`groundLayer`) configurada.
- El salto está **bloqueado** si Spring no está tocando el suelo (evitando el doble salto).
- El `GameObject` "Spring" tiene los tres componentes clave (`Rigidbody2D`, `MovimientoPlayer`, y `SaltoPlayer`) correctamente adjuntos y configurados con sus valores iniciales en el Inspector.

Codificacion HU01

Script MovimientoPlayer:

```
using UnityEngine;

public class MovimientoPlayer : MonoBehaviour
{
    [SerializeField] private float velocidad;
    private float direccion;

    void Update()
    {
        // Capturar input horizontal (-1 a 1)
        direccion = Input.GetAxis("Horizontal");

        // Aplicar movimiento horizontal usando transform
        if (direccion != 0)
        {
            transform.Translate(Vector2.right * direccion * velocidad *
Time.deltaTime);
        }

        // Voltear el sprite según la dirección del movimiento
        if (direccion != 0)
        {
```



```
            transform.localScale = new Vector3(Mathf.Sign(direccion), 1, 1);
        }
    }
    // Método público para que otros scripts puedan consultar la dirección
    public float GetDireccion()
    {
        return direccion;
    }
}
```

Script SaltoPlayer

```
using UnityEngine;

public class SaltoPlayer : MonoBehaviour
{
    private Rigidbody2D rb;
    private bool isGrounded;
    [SerializeField] private float fuerzaSalto;
    [SerializeField] private Transform groundCheck;
    [SerializeField] private LayerMask groundLayer;
    [SerializeField] private float groundRadius;

    void Start()
    {
        // Obtener referencia al Rigidbody2D
        rb = GetComponent<Rigidbody2D>();
    }

    void Update()
    {
        // Ejecutar salto si está en el suelo y presiona el botón de salto
        if (isGrounded && Input.GetButtonDown("Jump"))
        {
            rb.linearVelocity = new Vector2(rb.linearVelocity.x,
fuerzaSalto);
        }
    }

    void FixedUpdate()
    {
        // Verificar si el jugador está tocando el suelo
        isGrounded = Physics2D.OverlapCircle(groundCheck.position,
groundRadius, groundLayer);
// Limitar la velocidad de caída máxima
```






```
        if (rb.linearVelocity.y < -30f)
        {
            rb.linearVelocity = new Vector2(rb.linearVelocity.x, -30f);
        }

    }

    // Método público para que otros scripts puedan consultar si está en el
suelo
    public bool IsGrounded()
    {
        return isGrounded;
    }
}
```

Agacharse Spring

Fase de análisis:

Historia de usuario HU02:

Quién: Como jugador.
Qué: Quiero agacharme cuando presione la tecla correspondiente.
Para qué: Para navegar a través de los niveles de plataformas y superar obstáculos.

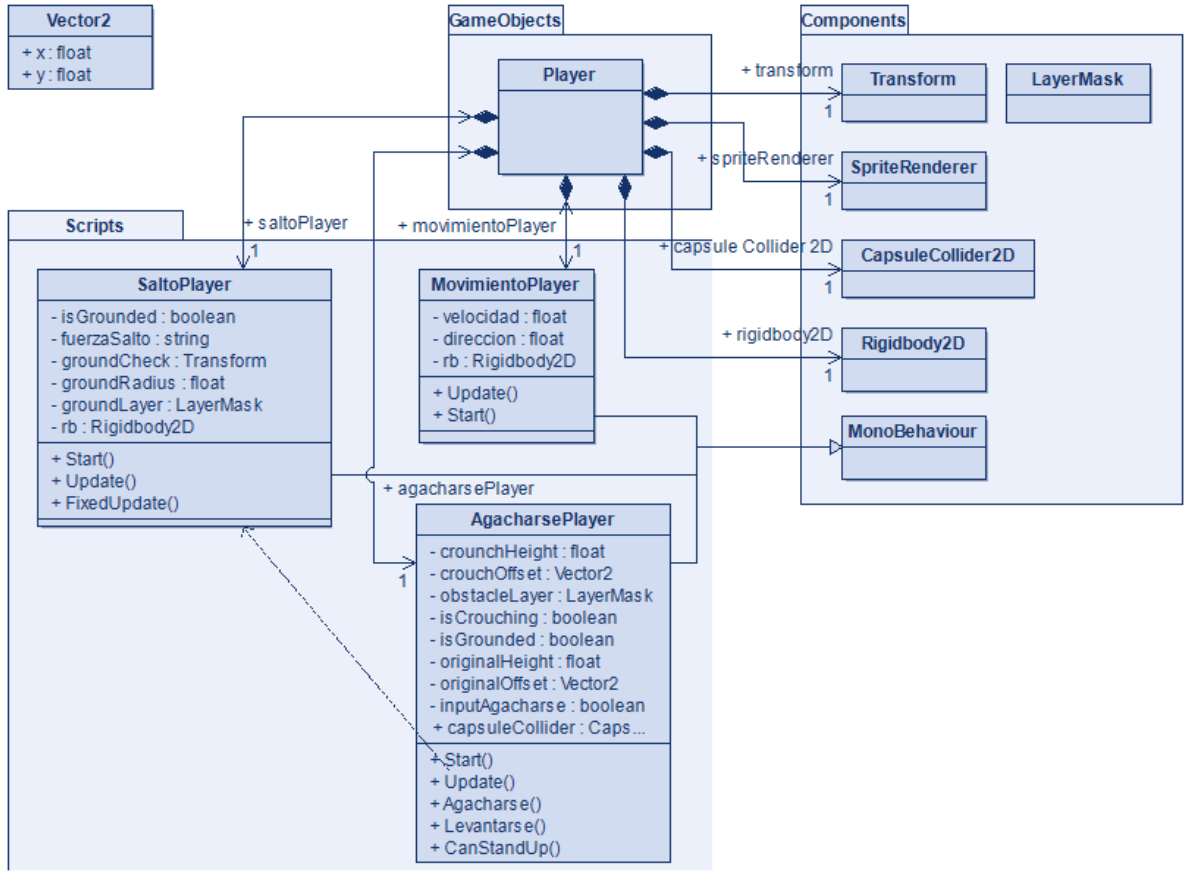
Tabla EPS

Entrada	Proceso	Salida
Tecla `Ctrl` (presionada y soltada).	1. Almacenar la altura y el offset Y originales del `CapsuleCollider2D`. 2. Detectar el *input* de agacharse. 3. Al presionar: Modificar `CapsuleCollider2D.height` y `CapsuleCollider2D.offset`	El componente `CapsuleCollider2D` cambia de tamaño. Un estado booleano (`isCrouching`) se actualiza

	a los valores de agachado (ej. mitad de altura). 4. Al soltar: Comprobar si hay espacio para levantarse. 5. Si hay espacio: Restaurar el collider a sus valores originales.	

Fase de diseño:

Diagrama de clases:



Objetivo Central: Implementar un sistema de agachado para poder navegar por diferentes niveles con versatilidad

Lista de Tareas para HU02



- **Modificar SaltoPlayer:** Crear un método accesor (getter) público que devuelva el valor de la variable privada isGrounded.
- **Crear el Script:** Crear el script C# AgacharsePlayer.
- **Adjuntar el Script:** Adjuntar el script AgacharsePlayer al GameObject "Spring".
- **Asegurar Collider:** Asegurarse de que el Collider de "Spring" sea un CapsuleCollider2D (ya que es el más fácil de redimensionar verticalmente).
- **Definir Variables (en AgacharsePlayer):**
    - Crear variables para la altura de agachado (crouchHeight) y el offset Y de agachado (crouchOffset).
    - Crear una variable LayerMask obstacleLayer (para saber qué es el "techo").
    - Crear una variable bool isCrouching (para que otros scripts sepan el estado).
- **Obtener Referencias (en Start() de AgacharsePlayer):**
    - Obtener la referencia al CapsuleCollider2D (ej. GetComponent<CapsuleCollider2D>()).
    - Obtener la referencia al script SaltoPlayer (ej. GetComponent<SaltoPlayer>()).
- **Guardar Estado Original (en Start() de AgacharsePlayer):**
    - Guardar la altura (originalHeight) y el offset Y (originalOffset) originales del CapsuleCollider2D.
- **Implementar Lógica de Agacharse (en Update() de AgacharsePlayer):**
    - Detectar cuándo se *presiona* la tecla Ctrl.
    - Al presionar, verificar si el método isGrounded devuelve true.
    - Si está en el suelo, cambiar el .height y .offset del CapsuleCollider2D a los valores de agachado (crouchHeight, crouchOffset) y setear isCrouching = true.
- **Implementar Lógica de Levantarse (en Update() de AgacharsePlayer):**
    - Detectar cuándo se *suelta* la tecla Ctrl (o si isCrouching es true y la tecla no está presionada).
- **Crear Chequeo de Obstáculos (en AgacharsePlayer):**
    - Crear una función privada CanStandUp() que use Physics2D.OverlapCapsule (o similar) para comprobar si hay un obstáculo (obstacleLayer) en el espacio donde el jugador estaría de pie.
- **Finalizar Lógica de Levantarse (en Update()):**
    - Si la tecla Ctrl se suelta (o no está presionada) y isCrouching es true, llamar a CanStandUp().
    - Si CanStandUp() devuelve true (no hay obstáculos), restaurar el CapsuleCollider2D a sus originalHeight y originalOffset, y setear isCrouching = false.

Criterios de Aceptación HU01

- Al presionar la tecla Ctrl **y** isGrounded devuelve true (el jugador está en el suelo), el CapsuleCollider2D de Spring reduce su altura y ajusta su offset a los valores de agachado.
- Al presionar la tecla Ctrl **mientras** isGrounded devuelve false (el jugador está en el aire), el collider **no** cambia y el jugador no se agacha.



- Al soltar la tecla Ctrl (mientras se está agachado) **y no hay obstáculos** encima (según la comprobación de CanStandUp()), el CapsuleCollider2D de Spring restaura su originalHeight y originalOffset.
- Al soltar la tecla Ctrl (mientras se está agachado) **mientras hay un obstáculo** encima, Spring permanece agachado (el collider no se restaura) hasta que el obstáculo desaparezca y la tecla Ctrl siga suelta.
- La variable pública isCrouching del script AgacharsePlayer es true solo cuando el collider está reducido, y false cuando está de pie.
- El script AgacharsePlayer obtiene correctamente la referencia al script SaltoPlayer en Start() y puede acceder a su estado isGrounded a través del método accesor (getter).
- El GameObject "Spring" tiene el CapsuleCollider2D y el script AgacharsePlayer adjuntos, con los valores (crouchHeight, crouchOffset, obstacleLayer) configurados en el Inspector.

Codificacion HU02

Script AgacharsePlayer

```
using UnityEngine;

public class AgacharsePlayer : MonoBehaviour
{
    // Variables para el collider cuando está agachado
    private float crouchHeight;
    private Vector2 crounchOffset;

    // Variables para el collider en posición normal
    private float originalHeight;
    private Vector2 originalOffset;

    // Control de estados y referencias
    private LayerMask obstacleLayer;
    private bool isCrouching = false;
    private bool isGrounded;
    private bool inputAgacharse;
    private CapsuleCollider2D capsuleCollider;

    void Start()
    {
        // Obtener referencia al collider
        capsuleCollider = GetComponent<CapsuleCollider2D>();

        // Configurar valores de posición normal
        originalHeight = 2.3f;
```



```
        originalOffset = new Vector2(capsuleCollider.offset.x, -0.18f);

        // Configurar valores para posición agachado
        crouchHeight = 1.5f;
        crounchOffset = new Vector2(capsuleCollider.offset.x, -0.58f);

        // Configurar layer de obstáculos
        obstacleLayer = LayerMask.GetMask("Obstacles");

    }

    void Update()
    {
        // Capturar input del jugador y verificar si está en el suelo
        inputAgacharse = Input.GetKey(KeyCode.LeftControl);
        isGrounded = GetComponent<SaltoPlayer>().IsGrounded();

        // Si presiona agacharse y está en el suelo, agacharse
        if (inputAgacharse && isGrounded)
        {
            Agacharse();
        }
        // Si deja de presionar y está agachado, intentar levantarse
        else if (!inputAgacharse && isCrouching)
        {
            Levantarse();
        }
    }

    // Cambia el collider para posición agachado
    private void Agacharse()
    {
        isCrouching = true;
        capsuleCollider.size = new Vector2(capsuleCollider.size.x,
crouchHeight);
        capsuleCollider.offset = crounchOffset;
    }

    // Intenta volver el collider a posición normal
    private void Levantarse()
    {
        if (CanStandUp())
        {
            isCrouching = false;
```






```
            capsuleCollider.size = new Vector2(capsuleCollider.size.x,
originalHeight);
            capsuleCollider.offset = originalOffset;
        }
    }

    // Verifica si hay espacio para levantarse
    private bool CanStandUp()
    {
        Vector2 standingCenter = (Vector2)transform.position +
originalOffset;
        Vector2 standingSize = new Vector2(capsuleCollider.size.x,
originalHeight);

        Collider2D obstacleFound = Physics2D.OverlapBox(
            standingCenter,
            standingSize,
            0f,
            obstacleLayer
            );
        return (obstacleFound == null);
    }
    // Método público para que otros scripts puedan consultar si está
agachado
    public bool IsCrouching()
    {
        return isCrouching;
    }
}
```

Animaciones Spring

Fase de análisis:

Historia de usuario HU03:

Quién: Como el **jugador** (observando a Spring)
Qué: Quiero **ver la animación** de Spring actualizada en tiempo real.
Para qué: Para que coincida con las acciones que estoy realizando (caminar, saltar, agacharme) y el juego se sienta fluido, receptivo y vivo.





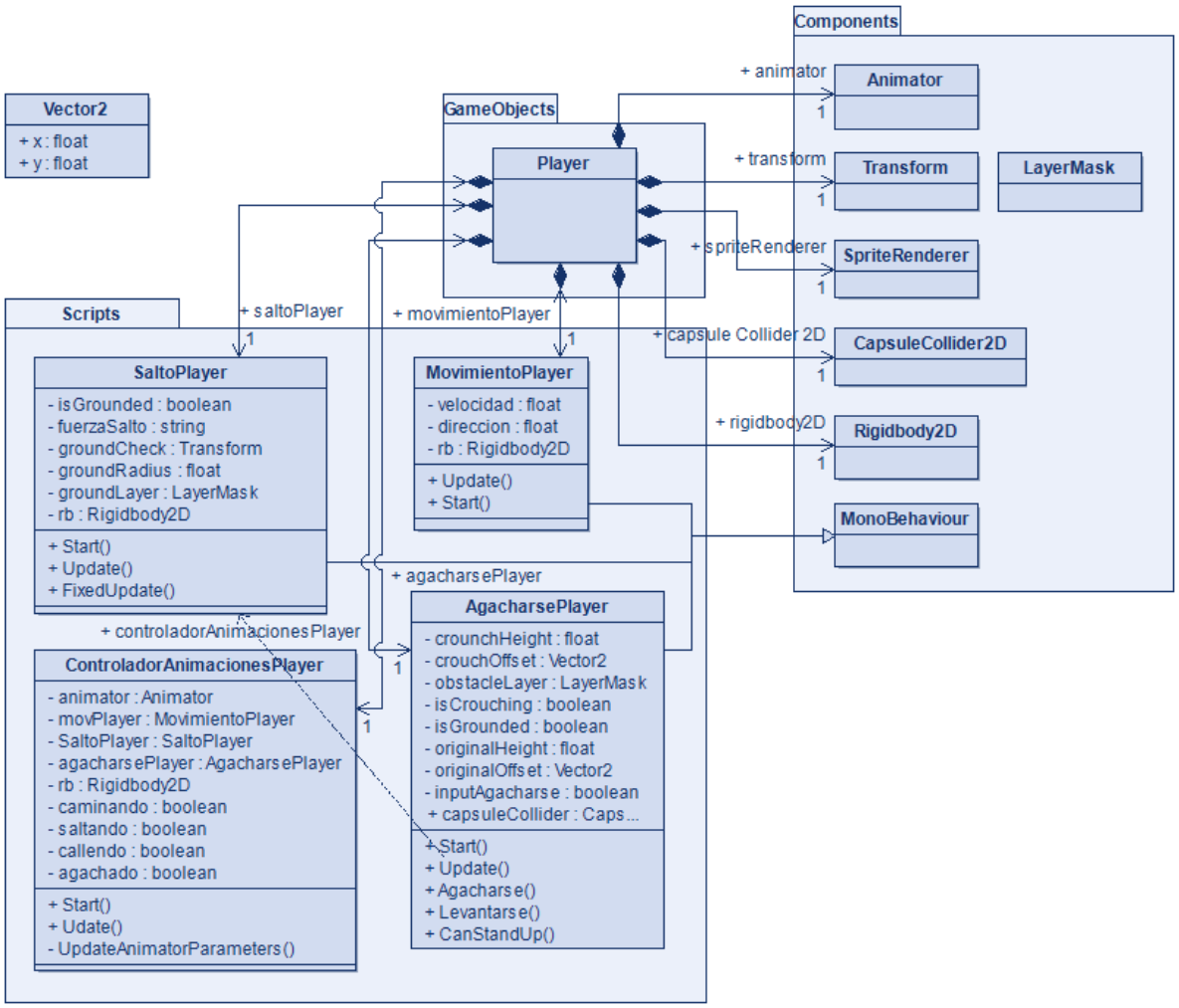
Tabla EPS

Entrada	Proceso	Salida
Los estados actuales de otros componentes: `MovimientoPlayer.InputHorizontal` (para saber si se mueve) `SaltoPlayer.EstaEnSuelo()` (para saber si está en el suelo) `AgacharsePlayer.isCrouching` (para saber si está agachado) `Rigidbody2D.velocity.y` (para saber si sube o baja)	**1. (Script `PlayerAnimationController`):** En `Update()`, lee todos los datos de entrada. **2. (Script `PlayerAnimationController`):** Procesa estos datos y los traduce a parámetros del `Animator` (ej. `animator.SetBool("isGrounded", true)`). **3. (Asset `AnimatorController`):** La máquina de estados detecta el cambio en los parámetros y gestiona la transición al estado de animación correcto.	El componente `Animator` reproduce el *clip de animación* correcto (Quieto, Caminando, Saltando, Agachado, etc.).

Fase de diseño:

Diagrama de clases:





Lista de Tareas HU03

A. Tareas en el Editor de Unity

- **Crear el Controlador:** En la carpeta del proyecto, crear un Animator Controller (ej. "AC_Spring").
- **Asignar el Controlador:** Asignar este *asset* AC_Spring al campo "Controller" del componente Animator en tu GameObject "Spring".
- **Definir Parámetros:** Abrir la ventana Animator (no Animation). En la pestaña "Parameters", crear los siguientes parámetros que usará el script:
    - caminando (Bool)
    - saltando (Bool)
    - callendo (bool)
    - agachado (bool)
- **Importar Animaciones:** Arrastrar las 8 clips de animación a la máquina de estados del Animator.
- **Crear Transiciones:** Conectar los estados con flechas (transiciones) y usar los



Parámetros como condiciones.
    - **De Quieto a Caminando:** Condición: caminando == true.
    - **De Caminando a Quieto:** Condición: caminando == false.
    - **De Quieto a Agachado:** Condición: agachado == true.
    - **De Agachado a Quieto:** Condición: agachado == false.
    - **etc…**

B. Tareas de Codificación (ControladorAnimacionesPlayer.cs)

- **Crear el Script:** Crear el script C# **ControladorAnimacionesPlayer**
- **Definir Variables:** Crear variables privadas para almacenar las referencias a los componentes: Animator, MovimientoPlayer, SaltoPlayer, AgacharsePlayer, y Rigidbody2D.
- **Obtener Referencias (en Start()):** Usar GetComponent<T>() para asignar todos los componentes a las variables privadas.
- **Implementar Lógica (en Update()):**
    - Obtener los estados de los otros scripts
    - Obtener la velocidad horizontal
    - Obtener la velocidad vertical
    - Obtener el estado de agachado
- **Actualizar Parámetros (en Update()):**
    - Llamar a animator.SetBool(...)..
- **Asignar el Script:** Adjuntar el script PlayerAnimationController al GameObject "Spring".

3. Criterios de Aceptación HU03

- Cuando Spring está en el suelo (isGrounded = T), no se mueve (moveSpeed < 0.1) y no está agachado (isCrouching = F), se reproduce la animación **"Quieto"** (estar quieto).
- Cuando Spring está en el suelo (isGrounded = T), se mueve (moveSpeed > 0.1) y no está agachado (isCrouching = F), se reproduce la animación **"Caminando"**.
- Cuando Spring está en el aire (isGrounded = F), su velocidad vertical es positiva (vVelocity > 0.1) y su velocidad horizontal es baja (moveSpeed < 0.1), se reproduce la animación **"Saltando"**.
- Cuando Spring está en el aire (isGrounded = F), su velocidad vertical es negativa (vVelocity < -0.1) y su velocidad horizontal es baja (moveSpeed < 0.1), se reproduce la animación **"Cayendo"**.
- Cuando Spring está en el aire (isGrounded = F), su velocidad vertical es positiva (vVelocity > 0.1) y se mueve horizontalmente (moveSpeed > 0.1), se reproduce la animación **"Saltando y caminando"**.
- Cuando Spring está en el aire (isGrounded = F), su velocidad vertical es negativa (vVelocity < -0.1) y se mueve horizontalmente (moveSpeed > 0.1), se reproduce la animación **"Cayendo y caminando"**.
- Cuando Spring está en el suelo (isGrounded = T) y está agachado (isCrouching = T) sin moverse (moveSpeed < 0.1), se reproduce la animación **"Agachado"**.



- Cuando Spring está en el suelo (isGrounded = T) y está agachado (isCrouching = T) y moviéndose (moveSpeed > 0.1), se reproduce la animación **"Agachado caminando"**.
- Las transiciones entre todas las animaciones se sienten fluidas y responden rápidamente a los cambios de estado del jugador.

Codificacion HU03

Script ControladorAnimacionesPlayer:

```
using UnityEngine;

public class ControladorAnimacionesPlayer : MonoBehaviour
{
    // Referencias a componentes
    private Animator animator;
    private MovimientoPlayer movPlayer;
    private SaltoPlayer saltoPlayer;
    private AgacharsePlayer agacharsePlayer;
    private Rigidbody2D rb;
    // Estados de animación
    private bool caminando;
    private bool saltando;
    private bool agachado;
    private bool cayendo;

    void Start()
    {
        // Obtener referencias a todos los componentes necesarios
        animator = GetComponent<Animator>();
        movPlayer = GetComponent<MovimientoPlayer>();
        saltoPlayer = GetComponent<SaltoPlayer>();
        agacharsePlayer = GetComponent<AgacharsePlayer>();
        rb = GetComponent<Rigidbody2D>();
    }
    void Update()
    {
        // Actualizar estados basados en los componentes de movimiento
        caminando = movPlayer.GetDireccion() != 0;
        saltando = !saltoPlayer.IsGrounded() && rb.linearVelocity.y > 0;
        cayendo = !saltoPlayer.IsGrounded() && rb.linearVelocity.y < 0;
        agachado = agacharsePlayer.IsCrouching();
        // Enviar los estados al Animator
        UpdateAnimatorParameters();
```



```
    }
    // Actualiza todos los parámetros del Animator
    private void UpdateAnimatorParameters()
    {
        animator.SetBool("caminando", caminando);
        animator.SetBool("saltando", saltando);
        animator.SetBool("cayendo", cayendo);
        animator.SetBool("agachado", agachado);
    }
}
```

Contenedor de datos Spring

Fase de análisis:

Historia de usuario HU04:

Quién: Como **Sistema de Juego** (UI, Managers, Componentes del Player)
Qué: Necesito un repositorio de datos **centralizado y seguro** para los atributos del jugador (vidas, savia, ítems).
Para qué: Para que el estado del jugador pueda ser consultado o modificado de forma **controlada**, protegiendo la integridad de los datos (encapsulación).

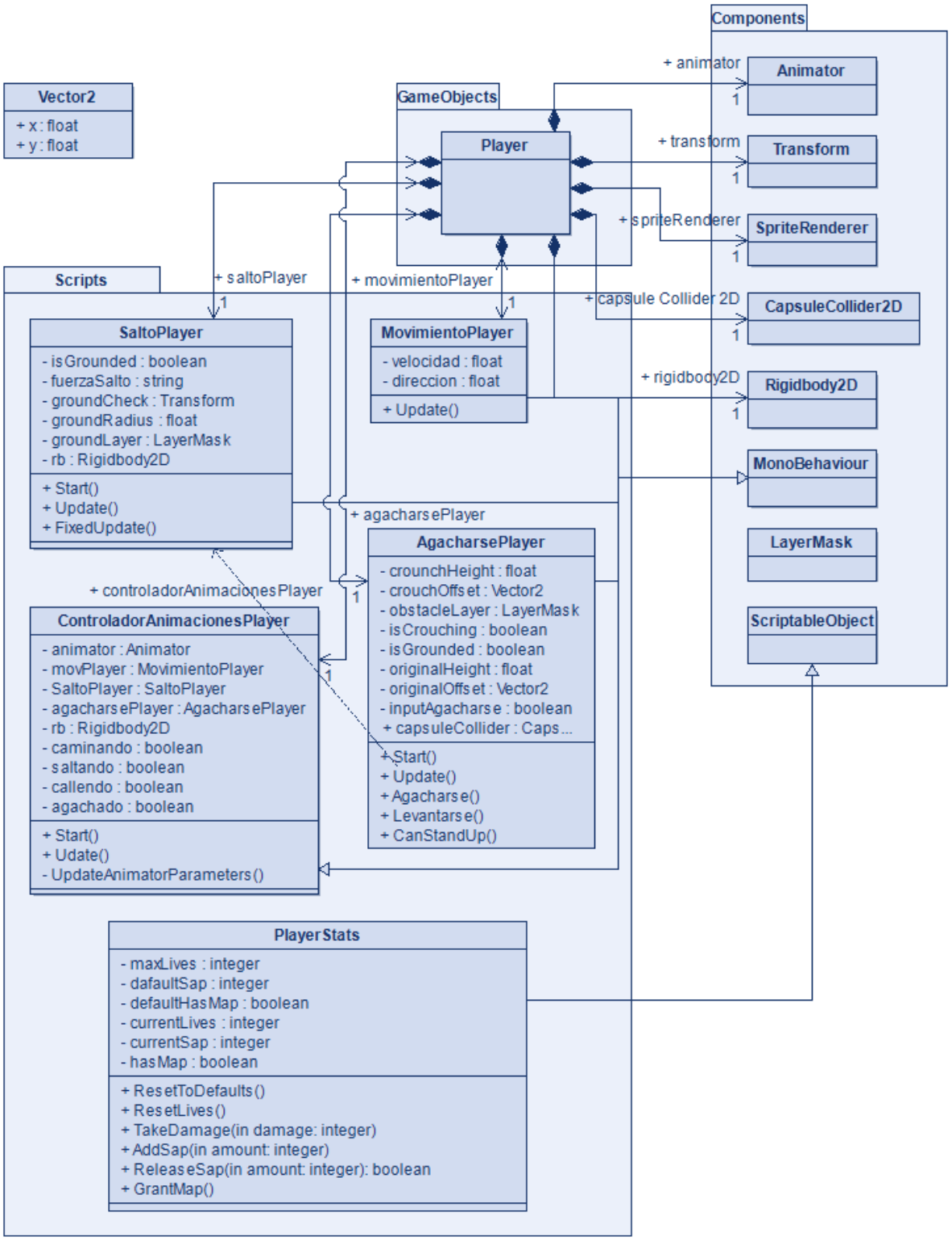
Tabla EPS

Entrada	Proceso	Salida
Llamadas a Setters: (Ej. `SetCurrentLives(int amount)`). **Valores de configuración:** (Ej. `_maxLives` establecido en el Inspector).	1. **Almacenamiento:** El SO guarda los datos en *campos privados*. 2. **Validación:** Los métodos Setters (ej. `AddSap`) validan la entrada (ej. `_currentSap + amount` no debe exceder `_maxSap`). 3. **Reset:** Un método `ResetToDefaults()` restaura los valores privados a su estado inicial.	**Llamadas a Getters:** (Ej. `GetCurrentLives()` devuelve el valor de `_currentLives`).



Fase de diseño:

Diagrama de clases:



Lista de Tareas HU04



- **Crear el Script (Molde):** Crear el archivo C# PlayerStats.cs en la carpeta Prefabs/Characters/Player/Scripts/.
- **Definir el Molde:**
    - Hacer que la clase PlayerStats herede de ScriptableObject (en lugar de MonoBehaviour).
    - Añadir el atributo [CreateAssetMenu(fileName = "NewPlayerStats", menuName = "Into The Root/Player Stats")] encima de la clase.
    - Definir los **campos de datos privados** con [SerializeField] (ej. [SerializeField] private int _maxLives;, [SerializeField] private int _currentLives;).
    - Implementar los **métodos Getters públicos** (ej. public int GetCurrentLives()).
    - Implementar los **métodos Setters/Modificadores públicos** (ej. public void TakeDamage(int damage)).
    - Implementar el método público ResetToDefaults() para restaurar los valores iniciales.
- **Crear el Asset (Instancia):**
    - Ir a la carpeta Prefabs/Characters/Player/.
    - Crear una nueva subcarpeta llamada Data.
    - Hacer clic derecho *dentro* de la carpeta Data y seleccionar Create > Into The Root > Player Stats.
    - Renombrar el nuevo archivo creado a **PlayerStats.asset**.
- **Configurar el Asset:** Seleccionar el archivo PlayerStats.asset en el Inspector y asignar los valores por defecto (ej. Max Lives = 3).

Criterios de Aceptación HU04

- El script PlayerStats.cs existe en la carpeta Scripts/ y hereda de ScriptableObject.
- El atributo [CreateAssetMenu] está presente en PlayerStats.cs.
- El archivo PlayerStats.asset existe físicamente en la carpeta Prefabs/Characters/Player/Data/.
- Al seleccionar PlayerStats.asset, se pueden ver y editar sus campos (ej. Max Lives) en el Inspector.
- Un script externo (ej. PlayerHealth) **no puede** acceder a _currentLives (debe dar error de compilación por ser privado).
- Un script externo (ej. PlayerHealth) **sí puede** llamar a los métodos públicos como GetCurrentLives() y TakeDamage().

Codificacion HU04



Script PlayerStats

```
using UnityEngine;

[CreateAssetMenu(fileName = "NewPlayerStats", menuName = "Into The
Root/Player Stats", order = 1)]
public class PlayerStats : ScriptableObject
{
    // --- CAMPOS PRIVADOS (Encapsulación) ---
    [Header("Configuración y Reset")]
    [Tooltip("Máximo de vidas que el jugador puede tener.")]
    [SerializeField] private int maxLives = 3;
    [SerializeField] private int defaultSap = 0;
    [SerializeField] private bool defaultHasMap = false;

    [Header("Estado Actual del Jugador")]
    [SerializeField] private int currentLives;
    [SerializeField] private int currentSap;
    [SerializeField] private bool hasMap;
    // ------------------------------------------
    // --- MÉTODOS DE INICIALIZACIÓN Y RESETEO ---
    // ------------------------------------------
    public void ResetToDefaults()
    {
        currentLives = maxLives;
        currentSap = defaultSap;
        hasMap = defaultHasMap;

        Debug.Log("PlayerStats reseteado: Valores de inicio de partida.");
    }

    public void ResetLives()
    {
        currentLives = maxLives;
        Debug.Log("Vidas reseteadas a MAX. Savia y ítems preservados.");
    }
    // ------------------------------------------
    // --- SETTERS/MODIFICADORES ---
    // ------------------------------------------
    public void TakeDamage(int damage)
    {
        if (damage > 0)
        {
            // Asegura que no baje de 0
            currentLives = Mathf.Max(0, currentLives - damage);
        }
```






```
    }

    public void AddSap(int amount)
    {
        if (amount > 0)
        {
            currentSap += amount;
        }
    }
    /// <summary>
    /// Intenta liberar una cantidad de savia del inventario del jugador.
    /// Se usa específicamente para la mecánica de curación del Gran Árbol.
    /// </summary>
    /// <returns>True si la savia fue liberada con éxito, False si no hay
suficiente.</returns>
    public bool ReleaseSap(int amount)
    {
        // 1. Validar que la cantidad es positiva y tenemos suficiente savia
        if (amount > 0 && currentSap >= amount)
        {
            // 2. Ejecutar la acción
            currentSap -= amount;
            Debug.Log($"[Mecánica Final] Se liberaron {amount} de savia.
Restante: {currentSap}");
            // Aquí se dispararía el evento para curar al árbol o activar la
UI
            return true;
        }
        // 3. Fallo
        Debug.Log($"Fallo al liberar savia. Necesario: {amount}, Actual:
{currentSap}");
        return false;
    }

    public void GrantMap()
    {
        hasMap = true;
    }
    // ---------------------------------------------
    // --- GETTERS ---
    // ---------------------------------------------
    public int GetCurrentLives()
    {
        return currentLives;
    }
```






```
    public int GetCurrentSap()
    {
        return currentSap;
    }

    public bool HasMap()
    {
        return hasMap;
    }
}
```

HUD

Fase de análisis:

Historia de usuario HU05:

Quién: Como **jugador**
Qué: Necesito ver el estado vital del Player (vidas, savia, mapa) actualizado en la pantalla.
Para qué: Para tomar decisiones estratégicas durante el juego y sentirme inmerso en la interfaz.

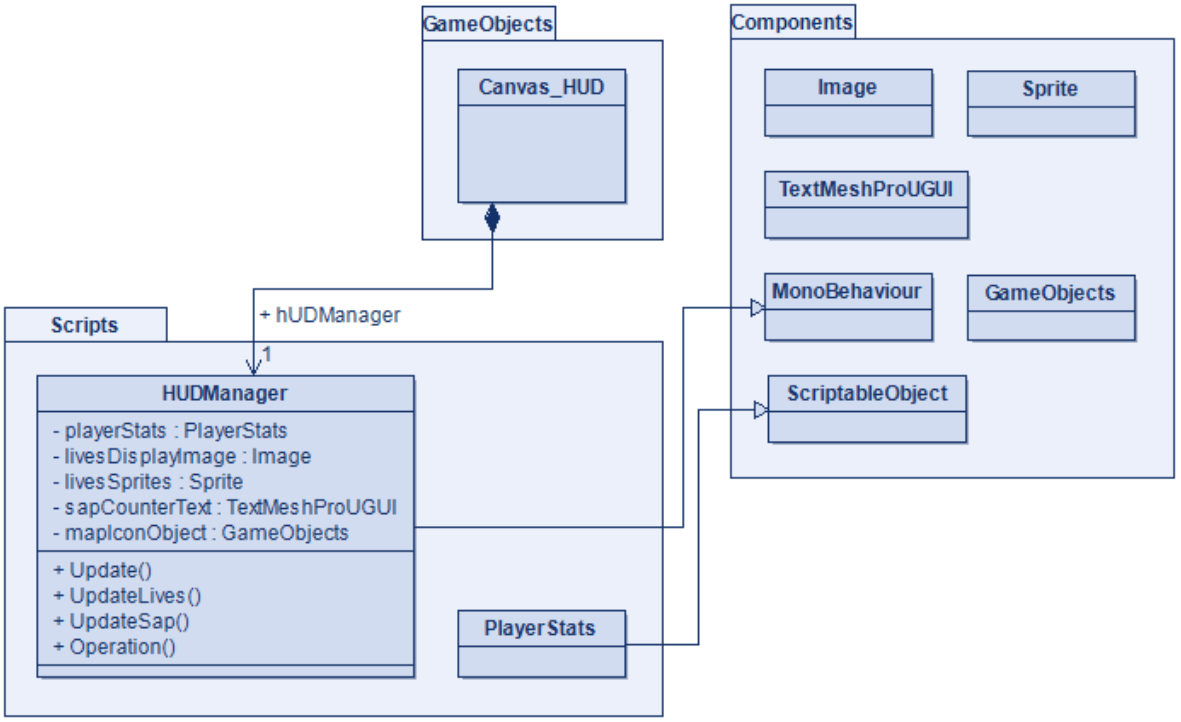
Tabla EPS

Entrada	Proceso	Salida
`PlayerStats.asset` - `GetCurrentLives()` - `GetCurrentSap()` - `HasMap()`	**1. Conexión:** El script obtiene la referencia al `PlayerStats.asset`. **2. Tick:** El script llama a una función (ej. `UpdateHUD()`) repetidamente (en `Update()` o por eventos). **3. Evaluación:** Dentro de `UpdateHUD()`, evalúa los valores (ej. `if (vidas == 3)`). **4. Asignación:** Asigna los sprites o valores de texto a los	Los componentes de UI cambian: - Imagen de vidas (cambia el *Sprite*). - Texto de savia (cambia la propiedad `.text`). - Imagen del mapa (se activa/desactiva el `GameObject`).

	componentes visuales (imágenes, textos).	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU05

- **Configurar Jerarquía UI:** Crear un Canvas_HUD en la escena con Render Mode en Screen Space - Overlay.
- **Crear Elementos UI:** Crear los siguientes objetos como hijos del Canvas o de un panel:
  - Un GameObject **Image** para la representación de las vidas (LivesDisplay).
  - Un GameObject **Text** (TextMeshPro) para el contador de savia (SapCounter).
  - Un GameObject **Image** para el ícono del mapa (MapIcon).
- **Crear el Script:** Crear el script C# HUDManager en la carpeta Scripts/UI/.
- **Declarar Conexión de Datos:** En HUDManager.cs, declarar la variable [SerializeField] PlayerStats playerStatsAsset;.
- **Declarar Referencias Visuales:** En HUDManager.cs, declarar las variables [SerializeField] para el componente Image de vidas, el componente TextMeshProUGUI para la savia, el GameObject para el ícono del mapa, y un **array de Sprite** para los diferentes estados de vida.



- **Asignar Referencia de Datos:** En el Editor de Unity, arrastrar el archivo **PlayerStats.asset** desde la carpeta Prefabs/Characters/Player/Data/ al campo PlayerStats Asset del HUDManager.
- **Asignar Referencias Visuales:** En el Editor de Unity, arrastrar los componentes de la escena (LivesDisplay, SapCounter, MapIcon) y los sprites de vida a los campos correspondientes del HUDManager.
- **Implementar Lógica:** En HUDManager.cs, implementar el método UpdateHUD() (llamado desde Update()):
    - Implementar la lógica para cambiar LivesDisplay.sprite usando playerStatsAsset.GetCurrentLives() como índice para el array de sprites.
    - Implementar la lógica para actualizar SapCounter.text usando playerStatsAsset.GetCurrentSap().
    - Implementar la lógica para activar/desactivar el MapIcon usando playerStatsAsset.HasMap().

Criterios de Aceptación HU05

- La imagen de vidas (LivesDisplay) cambia su sprite para reflejar con precisión el valor devuelto por playerStats.GetCurrentLives().
- El texto del contador de savia (SapCounter) muestra el valor exacto de playerStats.GetCurrentSap().
- El ícono del mapa (MapIcon) está activo (true) si playerStats.HasMap() es verdadero y desactivado (false) si es falso.
- Un solo script (HUDManager.cs) controla la actualización de todos los elementos visuales del HUD.
- El HUDManager no tiene una referencia directa al GameObject "Spring", solo al PlayerStats.asset.

Codificación HU05

Script HUDManager

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class HUDManager : MonoBehaviour
{
    // --- CONEXIÓN A DATOS ---

    [Header("Conexión de Datos")]
    [SerializeField] private PlayerStats playerStats;

    // --- REFERENCIAS VISUALES ---
```



```
    [Header("Visuales de Vida")]
    [SerializeField] private Image livesDisplayImage;
    [SerializeField] private Sprite[] livesSprites;

    [Header("Visuales de Savia y Mapa")]
    [SerializeField] private TextMeshProUGUI sapCounterText;
    [SerializeField] private GameObject mapIconObject;

    // ---  LÓGICA DE ACTUALIZACIÓN ---
    private void Update()
    {
        UpdateLives();
        UpdateSap();
        UpdateMap();
    }

    private void UpdateLives()
    {
        // 1. Obtener el valor actual de forma encapsulada (Getter)
        int currentLives = playerStats.GetCurrentLives();

        // 2. Determinar el índice correcto del array de sprites
        // Si tienes 4 sprites (3, 2, 1, 0 vidas), el índice va de 0 a
3.
        // Índice 0 = 3 vidas (livesSprites[0])
        // Índice 3 = 0 vidas (livesSprites[3])
        int maxLives = playerStats.GetMaxLives();
        int spriteIndex = maxLives - currentLives;

        // Aseguramos que el índice no esté fuera del límite del array
        spriteIndex = Mathf.Clamp(spriteIndex, 0, livesSprites.Length -
1);

        // 3. Asignar el sprite
        if (livesDisplayImage != null && livesSprites.Length > 0)
        {
            livesDisplayImage.sprite = livesSprites[spriteIndex];
        }
    }

    private void UpdateSap()
    {
        // 1. Obtener el valor actual de forma encapsulada (Getter)
        int currentSap = playerStats.GetCurrentSap();
```




```
        // 2. Asignar al componente de texto
        if (sapCounterText != null)
        {
            sapCounterText.text = currentSap.ToString();
        }
    }

    private void UpdateMap()
    {
        // 1. Obtener el estado del mapa de forma encapsulada (Getter)
        bool hasMap = playerStats.HasMap();

        // 2. Activar o desactivar el GameObject del ícono
        if (mapIconObject != null)
        {
            // Solo actualizamos si el estado es diferente para evitar
llamadas innecesarias
            if (mapIconObject.activeSelf != hasMap)
            {
                mapIconObject.SetActive(hasMap);
            }
        }
    }
}
```

Level Design

Fase de análisis:

Historia de usuario HU06:

Quién: Como **Jugador**
Qué: Necesito que el **mundo visual** (la imagen del nivel) tenga **superficies físicas sólidas** (colliders)...
Para qué: ...para poder **caminar, saltar e interactuar** con las plataformas y los límites del nivel sin atravesarlos

Tabla EPS

Entrada	Proceso	Salida



1. Las imágenes completas de los niveles (ej. `Nivel_1_Arbol.png`). 2. El Prefab del Jugador (con `SaltoPlayer` y `MovimientoPlayer`).	1. **Visual:** Colocar la imagen del nivel en la escena usando un `SpriteRenderer`. 2. **Físico:** Crear `GameObjects` vacíos para agrupar la colisión (ej. `_Environment`). 3. **Trazado:** Usar `BoxCollider2D` y `EdgeCollider2D` en `GameObjects` hijos para "dibujar" las superficies físicas sobre la imagen de fondo. 4. **Asignación:** Asignar la capa (Layer) `Ground` a todos los colliders de plataforma.	Una escena de Unity (`.unity`) con un entorno físico jugable y organizado jerárquicamente.

Fase de diseño:

Lista de Tareas HU06

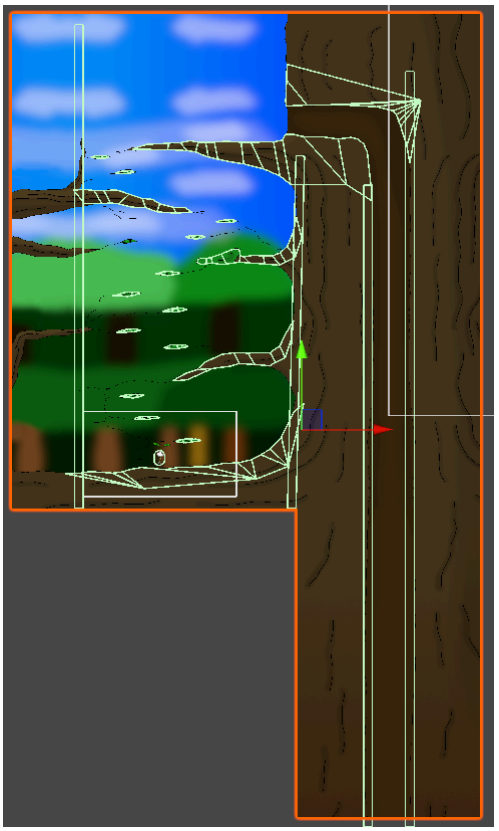
- Crear una nueva escena (ej. Nivel_1_Arbol).
- Crear un GameObject (ej. Fondo_Visual) y añadir un SpriteRenderer para mostrar la imagen de fondo completa del nivel.
- Crear un GameObject vacío padre (ej. **_Environment**) para agrupar todas las colisiones.
- Crear GameObjects vacíos hijos (ej. **Platforms**, **Walls**, **Hazards**) dentro de _Environment para organizar la jerarquía.
- Añadir componentes BoxCollider2D y EdgeCollider2D a GameObjects vacíos hijos dentro de los grupos Platforms y Walls.
- Ajustar manualmente (mover, escalar, editar puntos) cada Collider2D para que coincida perfectamente con la geometría de la imagen de fondo.
- Colocar el Prefab del Player en la posición inicial de la escena
- Repetir con los 3 niveles..

Criterios de Aceptación HU06

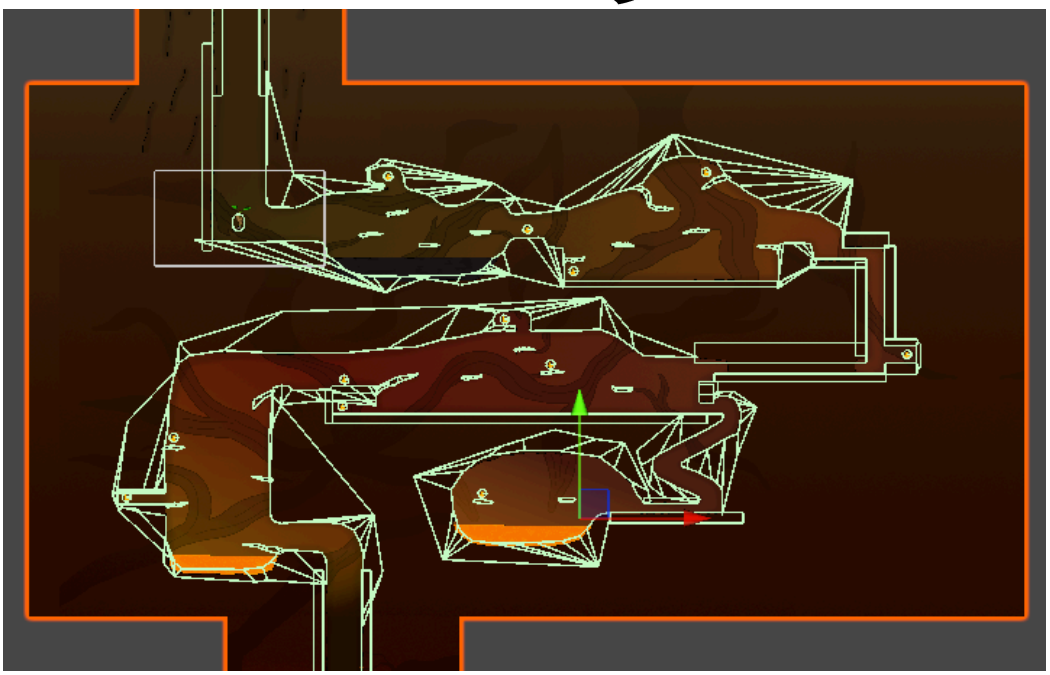
- La imagen completa del nivel se renderiza correctamente como fondo.
- El jugador (Spring) colisiona con el suelo y las plataformas (colliders de capa Ground) y no las atraviesa.
- El script SaltoPlayer detecta correctamente el suelo (isGrounded es true) cuando el jugador está sobre una plataforma.
- El jugador es bloqueado por los muros (colliders de capa Ground) y no puede salirse de los límites del nivel.
- Los GameObjects de colisión son invisibles (no tienen SpriteRenderer).
- La jerarquía de la escena está limpia, con todos los colliders de nivel agrupados bajo el GameObject padre _Environment.

Implementacion:

Nivel_1_Arbol



Nivel_2_Raices



Nivel_3_Mosquitos



Nivel 4 Purificacion



Savia

Fase de análisis:

Historia de usuario HU07:

Quién: Como **Jugador**
Qué: Necesito **colisionar** con el ítem de "Savia Sagrada".
Para qué: Para que se **incremente mi contador** de savia (en `PlayerStats`) y el ítem **desaparezca** del mundo.

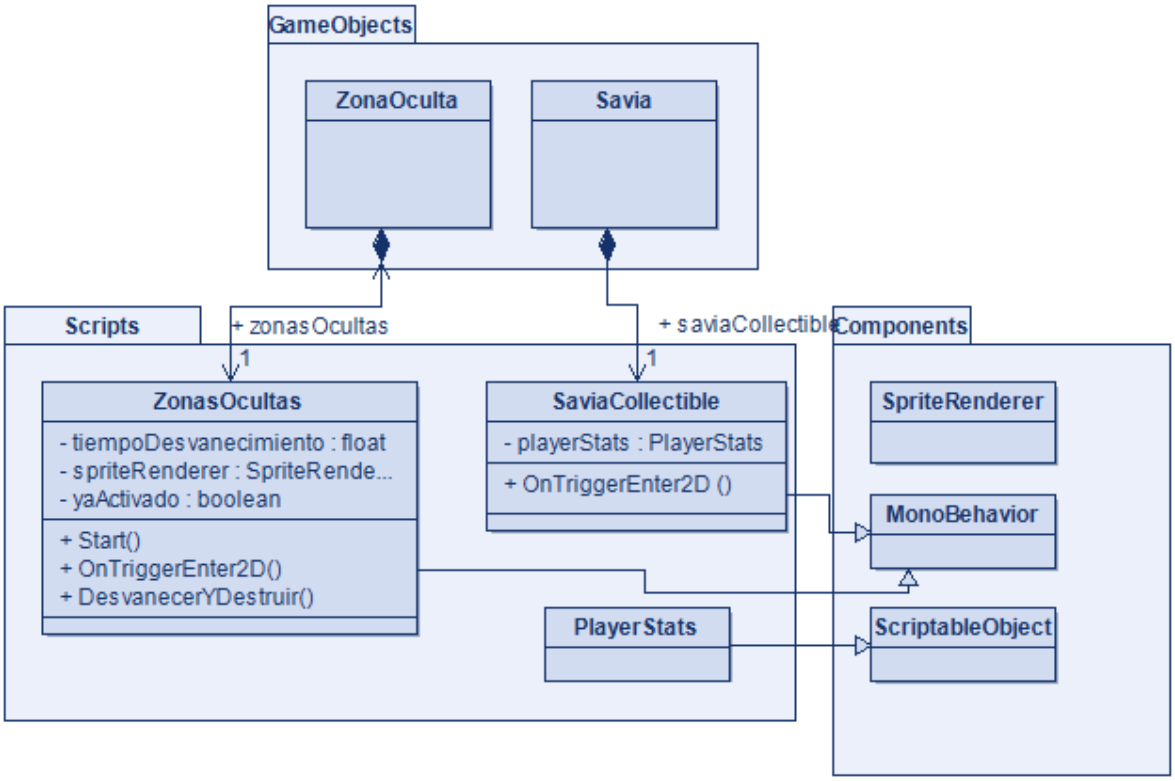
Tabla EPS

Entrada	Proceso	Salida
El motor de física de Unity detecta una colisión (`OnTriggerEnter2D`) entre el `Collider2D` del Player y el `Collider2D` (Trigger) de la Savia.	**1. (Script `SaviaCollectible.cs`):** El método `OnTriggerEnter2D` se activa. **2. (Validación):** El script comprueba si el objeto que	1. El valor `_currentSap` en el `PlayerStats.asset` aumenta en 1.

	colisionó tiene la etiqueta (Tag) "Player". **3. (Acción):** Si es "Player", el script accede a su referencia del `PlayerStats.asset` y llama al método `AddSap(1)`. **4. (Limpieza):** El script llama a `Destroy(gameObject)` para destruirse a sí mismo	2. El `GameObject` de la Savia Sagrada se elimina de la escena.

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU07

- Asegurarse de que tu Prefab del Player tenga la etiqueta (Tag) **"Player"**.
- Crear la carpeta Prefabs/Items/Savia/ (con Scripts/ y Sprites/).



- Crear el GameObject base de la Savia con su SpriteRenderer y un Collider2D marcado como **Is Trigger**.
- Crear el script SaviaCollectible.cs en la carpeta de scripts de la Savia.
- En el script, declarar la referencia al PlayerStats.asset ([SerializeField]).
- Implementar la lógica OnTriggerEnter2D para:
    - Comprobar la etiqueta "Player".
    - Llamar a playerStatsAsset.AddSap(1).
    - Llamar a Destroy(gameObject).
- Adjuntar el script al GameObject de la Savia y conectar el PlayerStats.asset en el Inspector.
- Crear el Prefab de la Savia arrastrándolo a su carpeta Prefabs/Items/Savia/.

Criterios de Aceptación HU07

- Existe un Prefab Savia_Collectible con un Collider2D configurado como Is Trigger.
- El Prefab tiene el script SaviaCollectible.cs adjunto, y este tiene la referencia al PlayerStats.asset conectada.
- Al ser tocado por el Player (Tag "Player"), el contador de savia en PlayerStats aumenta en 1.
- El ítem de Savia se destruye inmediatamente después de ser recolectado por el Player.
- El ítem ignora colisiones con objetos que no tengan la etiqueta "Player".

Lista de Tareas 2 HU07

- Preparar el *sprite* (la imagen) de la "pared falsa" que tapará la zona oculta.
- Crear el GameObject de la pared falsa en la escena.
- Añadir un SpriteRenderer al GameObject y asignarle el *sprite* de la pared falsa.
- Añadir un Collider2D (ej. BoxCollider2D) al GameObject y marcar la casilla **Is Trigger**.
- Crear el script C# (ej. ZonasOcultas.cs).
- En el script, declarar una variable pública para la velocidad de desvanecimiento (tiempoDesvanecimiento).
- Implementar la función OnTriggerEnter2D(Collider2D other) para comprobar si other.CompareTag("Player").
- Si es el "Player", iniciar una **Corrutina** (ej. DesvanecerYDestruir()).
- Implementar la Corrutina FadeOut():
    - Obtener el componente SpriteRenderer.
    - Crear un bucle while que se ejecute mientras la opacidad (el color.a) sea mayor que 0.
    - Dentro del bucle, reducir el color.a gradualmente usando Time.deltaTime * fadeSpeed.
    - Asegurarse de esperar al siguiente frame (yield return null;) dentro del bucle.
- Al finalizar el bucle (cuando la opacidad sea 0), llamar a Destroy(gameObject).



- Colocar el GameObject de la pared falsa en el nivel, tapando la zona secreta (donde están las Savia).

Criterios de Aceptación 2 HU07

- La pared falsa es 100% opaca (visible) al inicio.
- El GameObject de la pared falsa tiene un Collider2D configurado como Is Trigger.
- Al ser tocada por el Player (Tag "Player"), la pared falsa comienza a desvanecerse (su opacidad baja **gradualmente**).
- Una vez que la opacidad llega a 0 (es invisible), el GameObject de la pared falsa se destruye automáticamente.
- La zona secreta (con las Savias) que estaba detrás se vuelve visible y accesible para el Player.
- El desvanecimiento **no** se activa si un objeto sin la etiqueta "Player" (como un enemigo) toca el *trigger*.

Codificacion HU07

Script SaviaColectible

```
using UnityEngine;

public class SaviaCollectible : MonoBehaviour
{
    [SerializeField] private PlayerStats playerStats;

    // Detectar colisión con el jugador
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            // Aumentar la cantidad de savia en el jugador
            playerStats.AddSap(1);
            // Destruir el objeto recolectable
            Destroy(gameObject);
        }
    }
}
```

Script ZonasOcultas

```
using UnityEngine;
using System.Collections;
```



```
public class ZonasOcultas : MonoBehaviour
{
    [SerializeField] private float tiempoDesvanecimiento = 1f; // Tiempo
para desvanecerse
    private SpriteRenderer spriteRenderer;
    private bool yaActivado = false; // Para evitar múltiples activaciones

    void Start()
    {
        // Obtener referencia al SpriteRenderer
        spriteRenderer = GetComponent<SpriteRenderer>();

        // Verificar que tenga SpriteRenderer
        if (spriteRenderer == null)
        {
            Debug.LogError("ZonasOcultas: No se encontró SpriteRenderer en "
+ gameObject.name);
        }
    }
    // Detecta cuando el jugador entra en contacto con el collider
    void OnTriggerEnter2D(Collider2D other)
    {
        // Verificar si es el jugador y no se ha activado antes
        if (other.CompareTag("Player") && !yaActivado)
        {
            yaActivado = true;
            StartCoroutine(DesvanecerYDestruir());
        }
    }
    // Corrutina que desvanece progresivamente la opacidad
    private IEnumerator DesvanecerYDestruir()
    {
        float tiempoTranscurrido = 0f;
        Color colorInicial = spriteRenderer.color;
        // Reducir opacidad progresivamente durante el tiempo especificado
        while (tiempoTranscurrido < tiempoDesvanecimiento)
        {
            tiempoTranscurrido += Time.deltaTime;
            float alpha = Mathf.Lerp(1f, 0f, tiempoTranscurrido /
tiempoDesvanecimiento);
            spriteRenderer.color = new Color(colorInicial.r, colorInicial.g,
colorInicial.b, alpha);
            yield return null;
        }
        // Asegurar que la opacidad sea 0 al final
        spriteRenderer.color = new Color(colorInicial.r, colorInicial.g,
```



```
colorInicial.b, 0f);
        // Destruir el GameObject
        Destroy(gameObject);
    }
}
```

Game Manager

Fase de análisis:

Historia de usuario HU08:

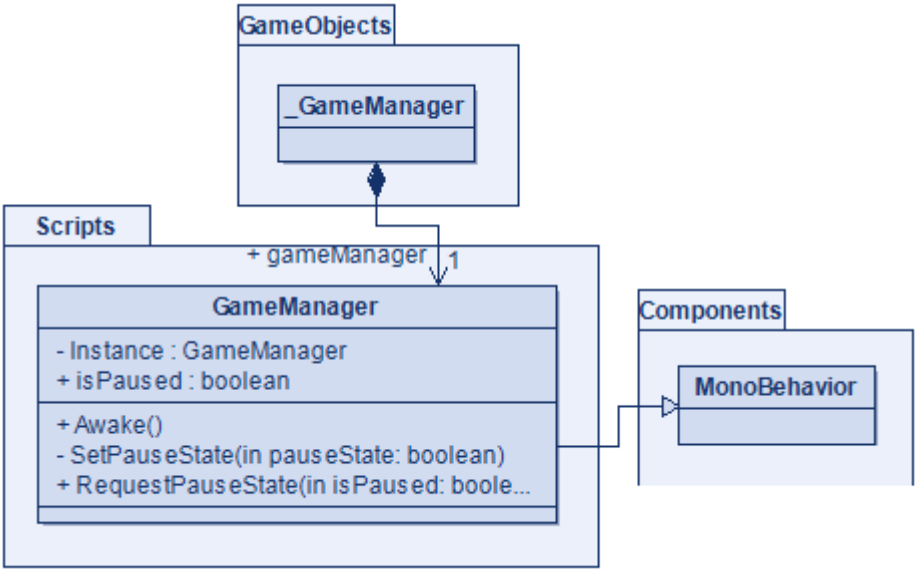
Quién: Como **Sistema de Juego**
Qué: Necesito gestionar un **estado de pausa global** (`IsPaused`) y controlar el `Time.timeScale`...
Para qué: ...para que los menús (Mapa, Pausa) puedan **solicitar** una pausa y los scripts del jugador (Movimiento) puedan **obedecerla** bloqueando el *input*.

Tabla EPS

Entrada	Proceso	Salida
Una llamada al método `GameManager.Instance.SetPauseState(true)` desde otro script (ej. `MapController`).	1. El `GameManager` actualiza su variable interna `IsPaused`. 2. El `GameManager` cambia el `Time.timeScale` global a `0f` (pausado) o `1f` (normal).	1. El tiempo del juego (`Time.timeScale`) se congela o reanuda. 2. La variable `IsPaused` es pública para que otros scripts (como `MovimientoPlayer`) puedan leerla.

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU08

- Crear el GameObject vacío _GameManager en la escena.
- Crear el script GameManager.cs.
- Adjuntar GameManager.cs al GameObject _GameManager.
- Implementar el **Singleton privado** (private static GameManager _instance) en Awake().
- Crear la variable privada IsPaused (con un *getter* público o [SerializeField]).
- Implementar el **método de lógica privado** private void SetPauseState(bool isPaused) para cambiar Time.timeScale y la variable IsPaused.
- Implementar la **fachada estática pública** con los métodos public static void RequestPauseState(bool isPaused) y public static bool GetIsPaused().
- Modificar los scripts de *input* del jugador (ej. MovimientoPlayer) para que incluyan el chequeo if (GameManager.GetIsPaused()) return; al inicio de Update().

Criterios de Aceptación HU08

- El juego **no** produce errores si otro script llama a GameManager.RequestPauseState() antes de que _instance esté lista (gracias al chequeo if (_instance != null)).
- Al llamar a GameManager.RequestPauseState(true), el Time.timeScale cambia a 0 y GameManager.GetIsPaused() devuelve true.
- Al llamar a GameManager.RequestPauseState(false), el Time.timeScale cambia a 1 y GameManager.GetIsPaused() devuelve false.
- Scripts externos **no pueden** acceder a la instancia del GameManager (el _instance es privado).
- Scripts externos **no pueden** llamar directamente al método de lógica SetPauseState() (es privado).



- Cuando GameManager.GetIsPaused() devuelve true, el jugador no puede moverse ni saltar (el *input* está bloqueado).

Codificacion

Script GameManager

```
// 📁 GameManager.cs
using UnityEngine;

public class GameManager : MonoBehaviour
{
    // 1. LA INSTANCIA
    private static GameManager instance;

    // 2. EL ESTADO
    public bool IsPaused { get; private set; }

    private void Awake()
    {
        // Configuración del Singleton
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }
    // --- 3. LA LÓGICA ---
    // Este método solo puede ser llamado por esta misma clase.
    private void SetPauseState(bool isPaused)
    {
        IsPaused = isPaused;
        Time.timeScale = isPaused ? 0f : 1f;
    }
    // --- 4. LA FACHADA ESTÁTICA ---
    // Otros scripts solo pueden llamar a estos métodos estáticos.

    /// <summary>
    /// Solicita al GameManager que cambie el estado de pausa.
    /// </summary>
    public static void RequestPauseState(bool isPaused)
    {
```






```
        // El método estático llama al método privado de la instancia
        if (instance != null)
        {
            instance.SetPauseState(isPaused);
        }
    }
    /// <summary>
    /// Devuelve el estado actual de la pausa.
    /// </summary>
    public static bool GetIsPaused()
    {
        if (instance != null)
        {
            return instance.IsPaused;
        }
        return false; // Estado seguro por defecto si no hay instancia
    }
}
```

Mapa

Fase de análisis:

Historia de usuario HU09:

Quién: Como **Jugador**
Qué: Necesito presionar la tecla 'M' para **alternar la vista del mapa**...
Para qué: ...para que el juego se **pause**, el **HUD se oculte** y yo pueda revisar el mapa, *solo si ya he conseguido el ítem del mapa*.

Tabla EPS

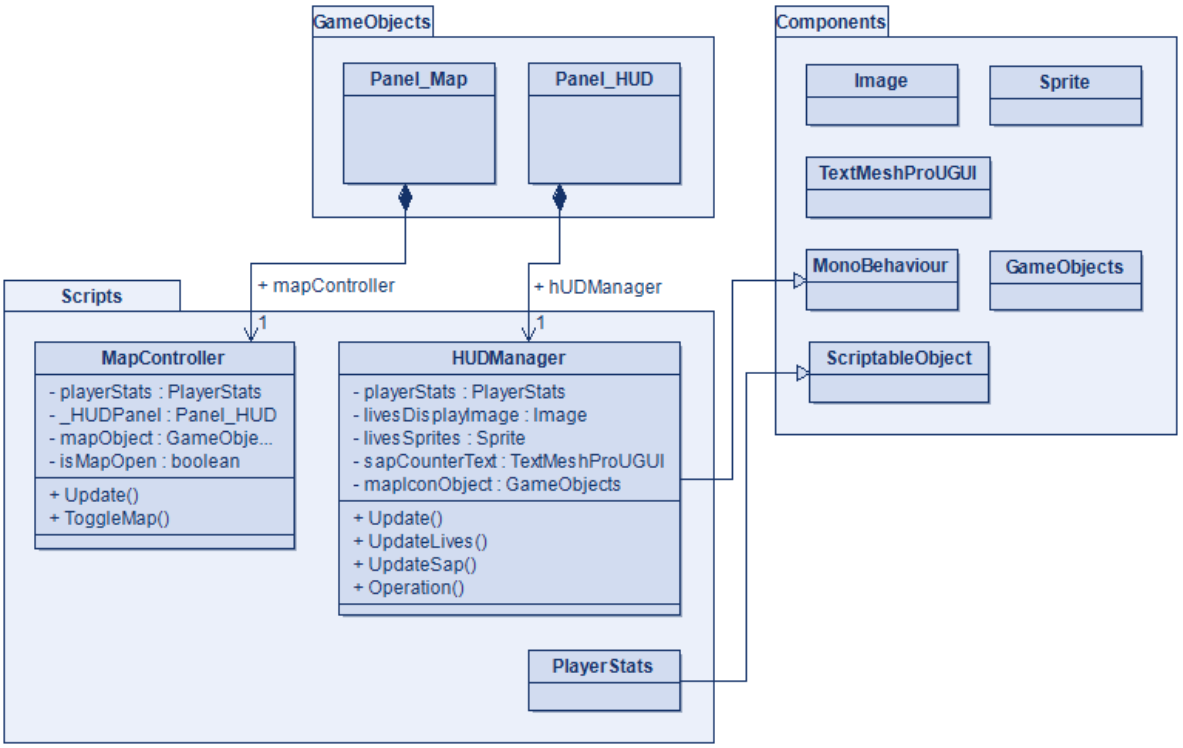
Entrada	Proceso	Salida
1. El jugador presiona la tecla 'M'. 2. El estado `HasMap()` de `PlayerStats`.	1. El script (`MapController.cs`) detecta la tecla 'M' en `Update()`. 2. Comprueba si `playerStatsAsset.HasMap(`	1. El juego está pausado (`Time.timeScale = 0`) y el *input* del jugador bloqueado. 2. El `Panel_Mapa` está visible.



	`)` es `true`. Si es `false`, no hace nada. 3. Si es `true`, invierte el estado (de abierto a cerrado, o viceversa). 4. Llama a `GameManager.RequestPauseState()` (para pausar/despausar). 5. Activa/desactiva el `GameObject` del panel del Mapa. 6. Activa/desactiva el `GameObject` del HUD regular.	3. El `Panel_HUD` (vidas/savia) está oculto.

Fase de diseño:

Diagrama de clases:





Lista de Tareas HU09

- Crear el GameObject del panel del mapa (ej. Panel_Map) dentro de tu Canvas. Asegúrate de que esté **desactivado** por defecto.
- Crear un GameObject padre para tu HUD regular (ej. Panel_HUD).
- Crear el script MapController.cs
- Adjuntar MapController.cs al GameObject Panel_Map.
- En MapController.cs, declarar las variables [SerializeField] para:
    - El PlayerStats.asset.
    - El GameObject del HUD regular (ej. Panel_HUD).
- Implementar la lógica en Update() para detectar la tecla 'M'.
- Implementar la función ToggleMap() que:
    - Comprueba si playerStats.HasMap() es false. Si lo es, return.
    - Invierte un bool local (ej. isMapOpen).
    - Llama a GameManager.RequestPauseState(isMapOpen).
    - Llama a gameObject.SetActive(isMapOpen) (para mostrar/ocultar el panel del mapa).
    - Llama a Panel_HUD.SetActive(!isMapOpen) (para ocultar/mostrar el HUD regular).
- Conectar las referencias (arrastrar el PlayerStats.asset y el Panel_HUD) en el Inspector del Panel_Map.

Criterios de Aceptación HU09

- Al presionar 'M', no ocurre nada si PlayerStats.HasMap() es false.
- Al presionar 'M' (y si HasMap() es true), el panel del mapa aparece.
- Cuando el mapa está abierto, el juego se pausa (Time.timeScale = 0).
- Cuando el mapa está abierto, el HUD regular (vidas/savia) desaparece.
- Cuando el mapa está abierto, el jugador no puede moverse (porque GameManager.GetIsPaused() es true).
- Al presionar 'M' de nuevo, el panel del mapa desaparece, el juego se despausa y el HUD regular reaparece.

Codificacion

Script MapController

```
using UnityEngine;

public class MapController : MonoBehaviour
{
    [SerializeField] private PlayerStats playerStats;
    [SerializeField] private GameObject mapObject;
    [SerializeField] private GameObject _HUDPanel;
    private bool isMapOpen = false;
    private void Start()
```



```
    {
        // Asegurarse que el mapa esté cerrado al iniciar
        if (mapObject != null)
        {
            mapObject.SetActive(false);
        }
        isMapOpen = false;
    }

    void Update()
    {
        // Escucha la tecla 'M'
        if (Input.GetKeyDown(KeyCode.M))
        {
            // Llama a la función de alternar
            ToggleMap();
        }
    }

    public void ToggleMap()
    {
        // 1. Condición: ¿El jugador tiene el mapa?
        // Si no tiene el asset o no tiene el mapa, no hacer nada.
        if (playerStats == null || !playerStats.HasMap())
        {
            return;
        }
        // 2. Invertir el estado (de abierto a cerrado, o viceversa)
        isMapOpen = !isMapOpen;

        // 3. Solicitar Pausa/Despausa al GameManager
        GameManager.RequestPauseState(isMapOpen);
        // 4. Activar/Desactivar Paneles
        if (mapObject != null)
        {
            mapObject.SetActive(isMapOpen);
        }
        if (_HUDPanel != null)
        {
            // Oculta el HUD si el mapa está abierto (y viceversa)
            _HUDPanel.SetActive(!isMapOpen);
        }
    }
}
```






Player Kill

Fase de análisis:

Historia de usuario HU10:

Quién: Como **Jugador**
Qué: Necesito que, al recibir **un solo golpe** (daño), se active mi **invencibilidad** (cooldown)...
Para qué: ...para **ejecutar mi secuencia de muerte** (animación y fade a negro) y **evitar golpes múltiples** seguidos.

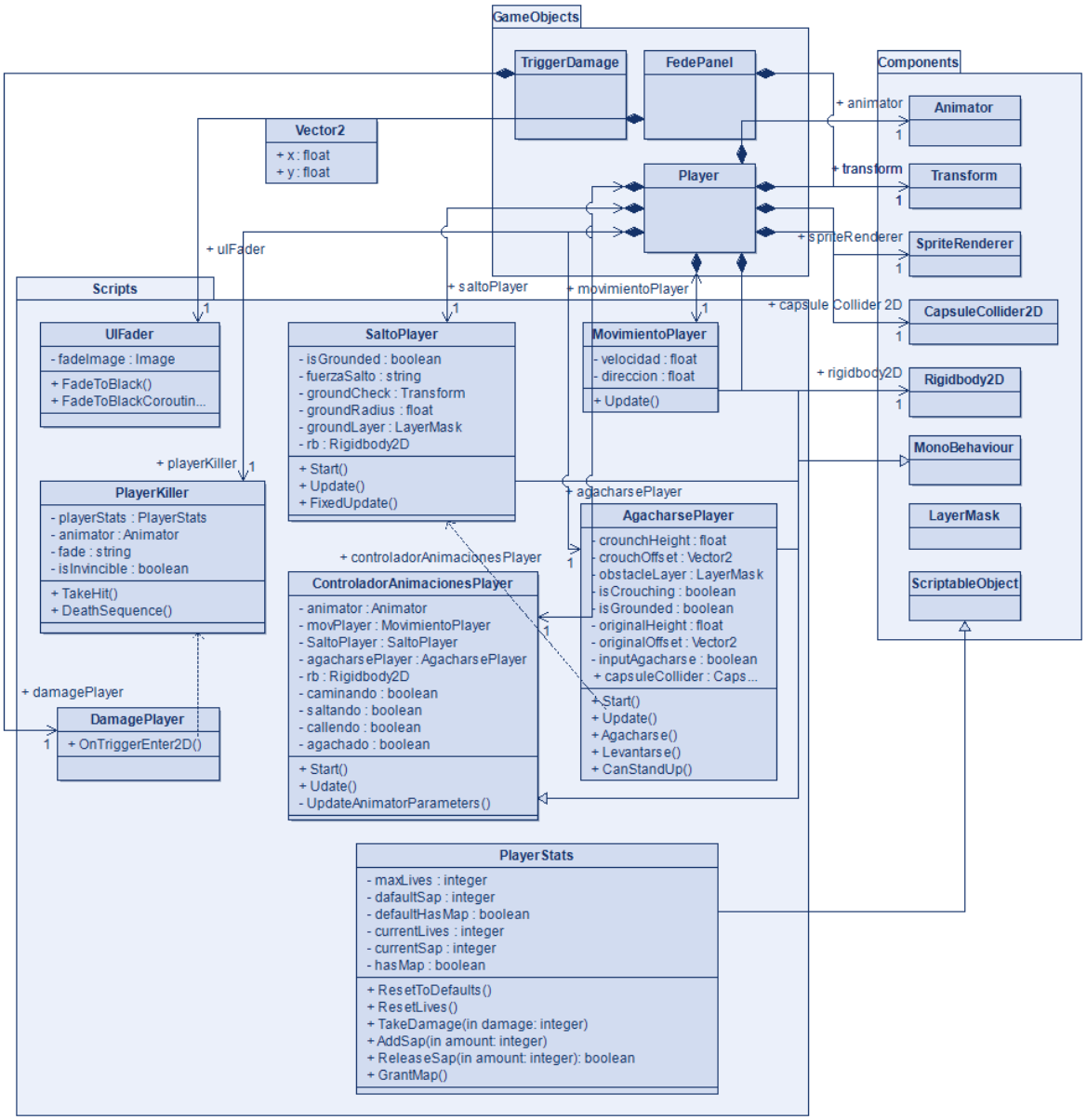
Tabla EPS

Entrada	Proceso	Salida
`OnTriggerEnter2D` entre `TargetDamage` (Trigger) y `Player` (Tag "Player").	1. **(DamagePlayer):** Detecta el "Player", usa `GetComponent` para encontrar `PlayerKiller`. 2. **(DamagePlayer):** Llama a `player.TakeHit()`. 3. **(PlayerKiller):** `TakeHit()` comprueba si `isInvincible` es `false`. 4. **(PlayerKiller):** Si no es invencible: a. Setea `isInvincible = true`. b. Llama a `GameManager.RequestInputLock(true)`. c. Llama a `playerStats.TakeDamage(1)`.	1. El input del jugador está bloqueado. 2. La animación "Muerte" se reproduce (en `Unscaled Time`). 3. La pantalla se vuelve negra (`UIFader`). 4. `PlayerStats` pierde 1 "intento". 5. El Player se vuelve invencible por X segundos.

	d. Inicia la corrutina `DeathSequence()`. e. Inicia la corrutina `InvincibilityCooldown()`	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU10

- **PlayerKiller.cs (Script en el Player):**



    - Crear el script PlayerKiller.cs.
    - Definir las variables [SerializeField] (PlayerStats, Animator, UIFader) .
    - Implementar public void TakeHit() (resta vida, bloquea input, inicia corrutinas).
    - Implementar la corrutina DeathSequence() (dispara SetTrigger("Muerte"), espera tiempo real, llama a uiFader.FadeToBlack(), recarga la escena o manda a la escena de GameOver dependiendo de la vida).
- **DamagePlayer.cs (Script en el Prefab de Daño):**
    - Crear el script DamagePlayer.cs.
    - Implementar OnTriggerEnter2D(Collider2D other).
    - En OnTriggerEnter2D, comprobar el CompareTag("Player"), usar other.GetComponent<PlayerKiller>() y llamar a TakeHit().
- **TargetDamage (Prefab):**
    - Crear el GameObject base (ej. "Pinchos").
    - Añadirle un Collider2D y marcarlo como **Is Trigger**.
    - Adjuntar el script DamagePlayer.cs.
    - Convertirlo en un Prefab (ej. TargetDamage.prefab).
- **UIFader.cs (Script en el Panel UI):**
    - Crear un Panel de UI (Canvas) con una Image negra que cubra la pantalla.
    - Crear el script UIFader.cs.
    - Implementar el método público public void FadeToBlack(float duration) (usará una corrutina para cambiar el alpha del color de la imagen).
- **Configuración Final (Player):**
    - Adjuntar PlayerKiller.cs al Prefab del Player.
    - Conectar todas sus referencias (PlayerStats, Animator, UIFader) en el Inspector.
    - **Importante:** Cambiar el Update Mode del Animator del Player a **Unscaled Time**.

Criterios de Aceptación HU10

- El Prefab TargetDamage (con Is Trigger y DamagePlayer.cs) existe.
- Al colisionar TargetDamage con el Player (Tag "Player"), se llama exitosamente a TakeHit() en el PlayerKiller.
- Si isInvincible es true, el golpe es ignorado por completo.
- Si isInvincible es false, el input del jugador se bloquea (GameManager.IsInputLocked() devuelve true).
- La animación "Muerte" se reproduce (incluso si el input está bloqueado, gracias a Unscaled Time).
- La pantalla se vuelve negra (UIFader) después de que la animación termina.
- PlayerStats pierde 1 vida (intento).
- El Player se vuelve invencible por X segundos después del golpe Y al reaparecer (para evitar bucles de muerte).
-

Codificacion HU10



Script PlayerKiller

```
using UnityEngine;
using System.Collections;
using UnityEngine.SceneManagement;

public class PlayerKiller : MonoBehaviour
{
    // Referencias asignables desde el Inspector
    [SerializeField] private PlayerStats playerStats;
    [SerializeField] private Animator animator;
    [SerializeField] private UIFader uiFader;
    // Llamar cuando el jugador recibe un golpe
    public void TakeHit()
    {
        // aplicar daño
        playerStats.TakeDamage(1);
        GameManager.RequestInputLock(true);
        // Iniciar corrutinas de muerte
        StartCoroutine(DeathSequence());
    }
    // Corrutina que reproduce la animación de muerte y luego dispara el
fade UI
    public IEnumerator DeathSequence()
    {
        animator.SetTrigger("muriendo");
        yield return new WaitForSecondsRealtime(1.2f);
        uiFader.FadeToBlack();
        yield return new WaitForSecondsRealtime(0.5f);
        if (playerStats.GetCurrentLives() > 0)
        {
            SceneManager.LoadScene(SceneManager.GetActiveScene().name);
            GameManager.RequestInputLock(false);
        }
        else
        {
            SceneManager.LoadScene("GameOverScene");
        }
    }
}
```

Script DamagePlayer

```
using UnityEngine;
```






```
public class DamagePlayer : MonoBehaviour
{
    void OnTriggerEnter2D(Collider2D other)
    {
        // 1. Verificar si el objeto que entró es el jugador
        if (other.CompareTag("Player"))
        {
            // 2. Buscar el script "PlayerKiller" EN EL OBJETO QUE COLISIONÓ
(other)
            PlayerKiller player = other.GetComponent<PlayerKiller>();

            // 3. Verificar si se encontró y usarlo
            if (player != null)
            {
                // 4. Llamar al método en el script que encontramos
                player.TakeHit();
            }
        }
    }
}
```

Script UIFader

```
using UnityEngine;
using UnityEngine.UI;
using System.Collections;

public class UIFader : MonoBehaviour
{
    // Image que se usará para el fade (puede asignarse desde el Inspector)
    [SerializeField] private Image fadeImage;
    // Inicia el fade a negro en 0.5 segundos
    public void FadeToBlack()
    {
        // Iniciar corrutina que hace el desvanecimiento
        StartCoroutine(FadeToBlackCoroutine(0.5f));
    }
    // Corrutina que hace el fade progresivo usando tiempo no-escalado
    private IEnumerator FadeToBlackCoroutine(float duration)
    {
        // Preparar colores inicial y objetivo
        Color startColor = fadeImage.color;
        Color targetColor = Color.black;
        float startAlpha = startColor.a;
        float elapsed = 0f;
        // Usar unscaledDeltaTime para que el fade funcione aunque
```



```
Time.timeScale == 0
        while (elapsed < duration)
        {
            elapsed += Time.unscaledDeltaTime;
            float t = Mathf.Clamp01(elapsed / duration);
            float alpha = Mathf.Lerp(startAlpha, 1f, t);
            // Aplicar color negro con alpha interpolado
            fadeImage.color = new Color(targetColor.r, targetColor.g,
targetColor.b, alpha);
            yield return null;
        }
        // Asegurar opacidad final completa
        fadeImage.color = new Color(targetColor.r, targetColor.g,
targetColor.b, 1f);
    }
}
```

Level Exit

Fase de análisis:

Historia de usuario HU11:

Quién: Como **Jugador**
Qué: Necesito que, al tocar el *trigger* al final del túnel, el juego **guarde mi progreso** (desbloquee el nivel)...
Para qué: ...y **cargue la siguiente escena** (Nivel 2), asegurándose de que yo aparezca **cayendo a la velocidad correcta** (-30f en Y) para mantener la continuidad.

Tabla EPS

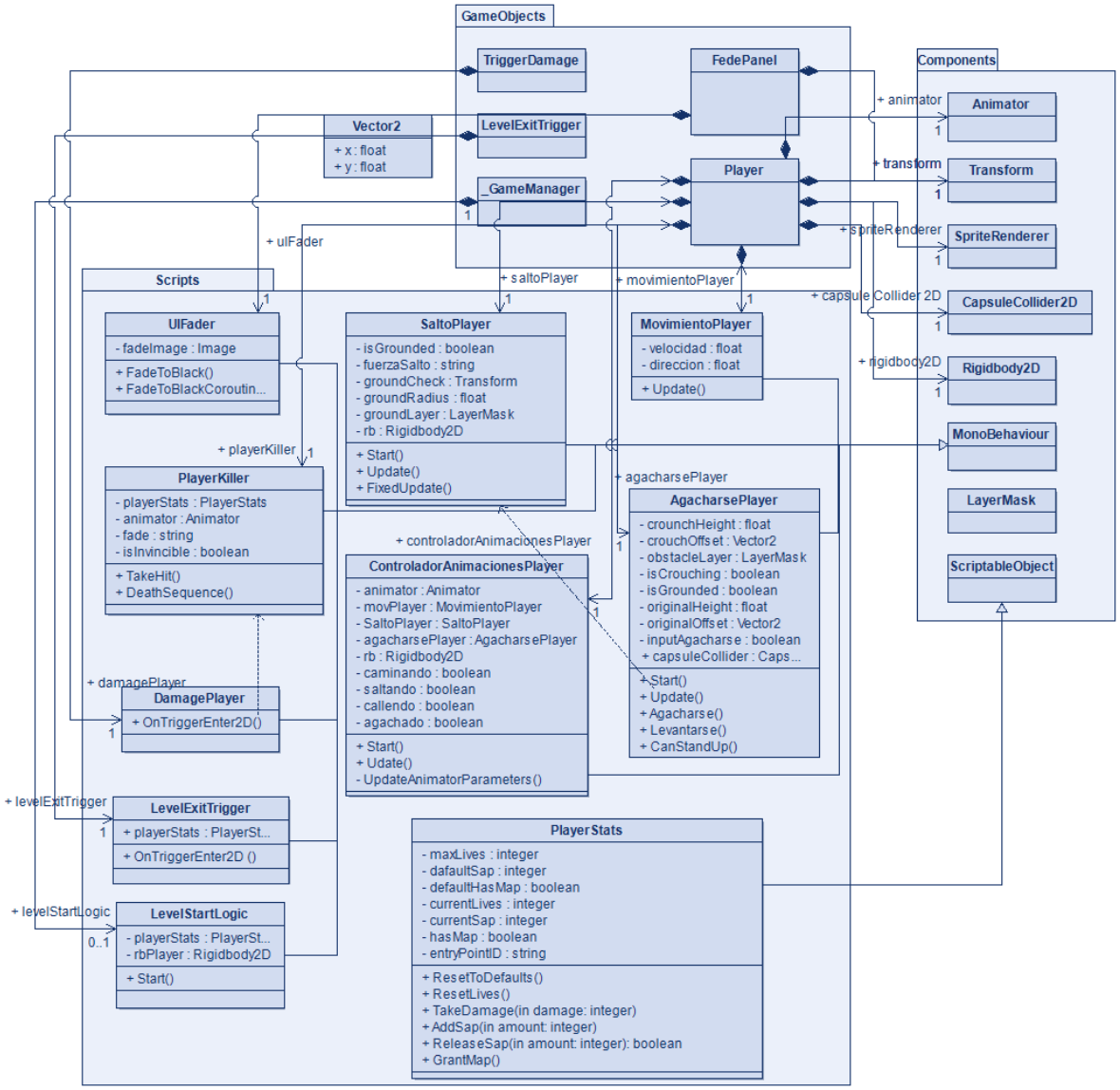
Entrada	Proceso	Salida
`OnTriggerEnter2D` (el Player toca el `LevelExitTrigger` en el Nivel 1).	1. **(Nivel 1 - `LevelExitTrigger.cs`):** a. Detecta al Player (Tag "Player"). b. Escribe el "mensaje" en el ScriptableObject (ej.	1. El Nivel 2 está cargado y activo. 2. El Player aparece en el Nivel 2, cayendo a -30f/s en Y. 3. El Nivel 2 está permanentemente



	`playerStats.entryPointID = "CaidaNivel1"`). c. Escribe el desbloqueo permanente en el disco (`PlayerPrefs.SetInt("HighestLevelReached", 2)`). d. Carga la nueva escena (`SceneManager.LoadScene("Nivel_2")`). 2. **(Nivel 2 - `LevelStartLogic.cs`):** a. Lee el "mensaje" (`playerStats.entryPointID`). b. Si el ID coincide ("CaidaNivel1"), busca el `Rigidbody2D` del Player. c. Aplica la velocidad inicial (`playerRigidbody.velocity = new Vector2(0f, -30f)`). d. Limpia el "mensaje" (`playerStats.entryPointID = ""`).	desbloqueado en el menú principal.

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU11

- **PlayerStats.cs:**
  - Añadir el campo (oculto) [HideInInspector] string entryPointID; para usarlo como "mensajero".
- **LevelExitTrigger.cs (Script de Salida):**
  - Crear el script LevelExitTrigger.cs y adjuntarlo a un GameObject (Prefab) de *trigger* en el túnel de salida (Nivel 1).
  - Implementar OnTriggerEnter2D para que compruebe el Tag "Player".
  - Dentro del *trigger*, implementar la lógica para setear el entryPointID en PlayerStats.
  - Implementar la lógica para guardar el progreso en PlayerPrefs (ej. PlayerPrefs.SetInt("HighestLevelReached", 2)).



    - Implementar la llamada para cargar la siguiente escena (SceneManager.LoadScene(...)).
- **LevelStartLogic.cs (Script de Entrada):**
    - Crear el script LevelStartLogic.cs y adjuntarlo a un objeto _GameManager en la escena del **Nivel 2**.
    - Definir las variables [SerializeField] para el PlayerStats.asset y el Rigidbody2D del Player (o buscarlo por Tag).
    - Implementar la lógica en Start():
        - Comprobar si playerStats.entryPointID coincide con el ID esperado (ej. "CaidaNivel1").
        - Si coincide, setear la playerRigidbody.velocity al valor de caída (0, -30f).
        - Limpiar el entryPointID en PlayerStats (setearlo a "") para que no se repita.
- **Configuración de Escena (Nivel 2):**
    - Colocar al Player (Prefab) en la posición exacta donde comienza el túnel de entrada del Nivel 2.

Criterios de Aceptación HU11

- Al tocar el *trigger* de salida en el Nivel 1, el Nivel 2 se carga correctamente.
- Al cargar el Nivel 2 (después de tocar el *trigger*), el Player aparece cayendo con una velocidad Y de -30f (o la configurada).
- El valor entryPointID en PlayerStats queda vacío ("") después de que el Nivel 2 ha cargado (para evitar que se aplique la velocidad al morir).
- PlayerPrefs guarda correctamente el nivel más alto alcanzado (ej. "HighestLevelReached" = 2).
- Si se carga el Nivel 2 directamente desde el menú (sin pasar por el *trigger*), el Player aparece quieto (velocity es 0,0) porque el entryPointID está vacío.

Codificación HU11

Script LevelExitTrigger

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class LevelExitTrigger : MonoBehaviour
{
    [SerializeField] private PlayerStats playerStats;
    private string sceneToLoad;
    private string entryPointID;

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
```



```
        {
            entryPointID = SceneManager.GetActiveScene().name.ToString();
            if(entryPointID == "Nivel_1_Arbol")
            {
                playerStats.SetEntryPointID(entryPointID);
                sceneToLoad = "Nivel_2_Raices";
                SceneManager.LoadScene(sceneToLoad);
                PlayerPrefs.SetInt("HighestLevelReached", 2);
            }else if(entryPointID == "Nivel_2_Raices")
            {
                playerStats.SetEntryPointID(entryPointID);
                sceneToLoad = "Nivel_3_Purificacion";
                SceneManager.LoadScene(sceneToLoad);
                PlayerPrefs.SetInt("HighestLevelReached", 3);
            }
        }
    }
}
```

Script LevelStartLogic

```
using UnityEngine;

public class LevelStartLogic : MonoBehaviour
{
    [SerializeField] private PlayerStats playerStats;
    [SerializeField] private Rigidbody2D playerRigidbody;
    void Start()
    {
        // 1. LEER EL MENSAJE
        if (playerStats.GetEntryPointID() == "Nivel_1_Arbol" ||
            playerStats.GetEntryPointID() == "Nivel_2_Raices")
        {
            // 2. APLICAR LA VELOCIDAD
            playerRigidbody.linearVelocity = new Vector2(0f, -30f);
        }
    }
}
```

Menu

Fase de análisis:

Historia de usuario HU12:



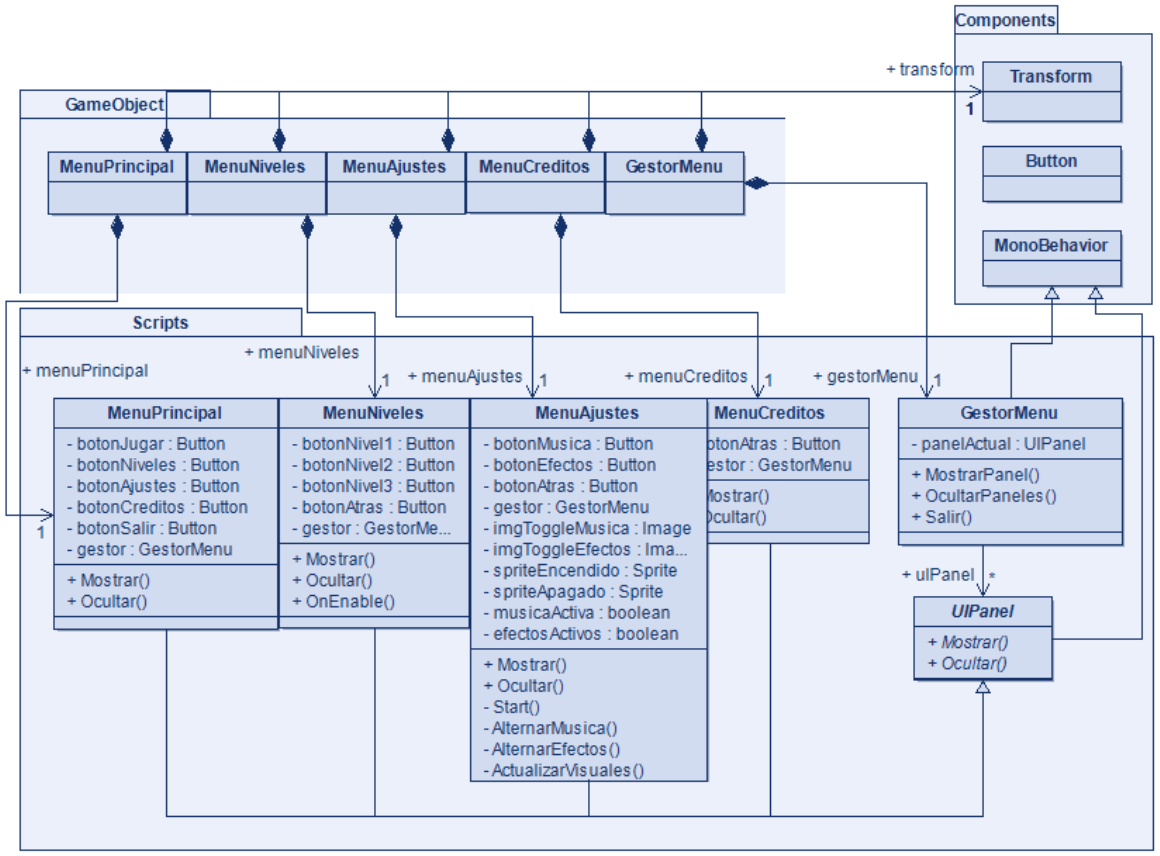
Quién: Sistema de Navegación
Qué: Necesito gestionar una colección de paneles y asegurarme de que solo uno esté visible a la vez
Para qué: para que el jugador pueda navegar por los menús sin ver pantallas superpuestas

Tabla EPS

Entrada	Proceso	Salida
El jugador hace clic en un botón (ej. `botonAjustes`). El `OnClick` de ese botón llama a `gestor.MostrarPanel(2)`.	1. **(GestorMenu):** Recibe el índice del panel a mostrar (ej. `index = 2`). 2. **(GestorMenu):** Llama al método `OcultarPaneles()` (recorre el *array* de `UIPanel` y llama a `panel.Ocultar()` en **todos**). 3. **(GestorMenu):** Llama al método `panel[index].Mostrar()` (activa solo el panel solicitado). 1. **(MenuNiveles):** `OnEnable()` se dispara. 2. **(MenuNiveles):** Llama a `PlayerPrefs.GetInt("HighestLevelReached", 1)`. 3. **(MenuNiveles):** Aplica la lógica `botonNivel2.interactable = (progreso >= 2)`.	El `MenuPrincipal` se oculta (`SetActive(false)`) y el `MenuAjustes` se muestra (`SetActive(true)`). Los botones de `MenuNiveles` están correctamente habilitados/deshabilitados.

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU12

- **(Base)** Crear la clase abstracta UIPanel con los métodos abstractos Mostrar() y Ocultar().
- **(Gestor)** Crear el script GestorMenu.cs.
- **(Gestor)** En GestorMenu, implementar un [SerializeField] private UIPanel[] paneles;.
- **(Gestor)** Implementar el método public void MostrarPanel(int index) (que oculta todos y muestra solo el index).
- **(Paneles)** Crear los scripts MenuPrincipal.cs, MenuNiveles.cs, MenuAjustes.cs y MenuCreditos.cs.
- **(Paneles)** Hacer que todos los scripts de panel hereden de UIPanel e implementen Mostrar() y Ocultar() (usando gameObject.SetActive(true/false)).
- **(Paneles)** En MenuPrincipal (y otros), añadir la referencia [SerializeField] private GestorMenu gestor;.
- **(Niveles)** En MenuNiveles.cs, implementar OnEnable() para leer PlayerPrefs y actualizar el estado interactable de los botones.
- **(Editor)** Asegurarse de que EventSystem esté en la escena.
- **(Editor)** Configurar todos los GameObjects del Canvas (Paneles y GestorMenu).
- **(Editor)** Asignar las referencias en el Inspector (arrastrar el GestorMenu a los paneles, arrastrar los UIPanel al *array* del GestorMenu).
- **(Editor)** Conectar los eventos OnClick de los botones (ej. botonNiveles.OnClick -> llama a gestor.MostrarPanel(1)).



Criterios de Aceptación HU12

- Al iniciar la escena, solo el MenuPrincipal está visible.
- Al hacer clic en botonNiveles, el MenuPrincipal se oculta y MenuNiveles se muestra.
- Al hacer clic en botonAtras (en MenuNiveles), MenuNiveles se oculta y MenuPrincipal se muestra.
- Si PlayerPrefs "HighestLevelReached" es 1, los botones de Nivel 2 y 3 no son interactuables.
- Si PlayerPrefs "HighestLevelReached" es 3, todos los botones de nivel son interactuables.
- Hacer clic en botonJugar carga la escena de introducción/Nivel 1.
- Hacer clic en botonSalir cierra la aplicación.

Codificacion HU12

Script GestorMenu

```
using UnityEngine;

public class GestorMenu : MonoBehaviour
{
    [SerializeField] private UIPanel[] paneles;
    private UIPanel panelActual;
    void Start()
    {
        OcultarPaneles();
        MostrarPanel(0);
    }
    public void MostrarPanel(int index)
    {
        if (index < 0 || index >= paneles.Length)
        {
            Debug.LogWarning("Índice de panel fuera de rango: " + index);
            return;
        }
        // Ocultar el panel actual si existe
        if (panelActual != null)
        {
            panelActual.Ocultar();
        }
        // Mostrar el nuevo panel
        paneles[index].Mostrar();
        panelActual = paneles[index];
    }
    public void OcultarPaneles()
    {
        for (int i = 0; i < paneles.Length; i++)
        {
```



```
            paneles[i].Ocultar();
        }
    }
    public void Salir()
    {
        #if UNITY_EDITOR
            UnityEditor.EditorApplication.isPlaying = false;
        #else
            Application.Quit();
        #endif
    }
}
```

Script UIPanel

```
using UnityEngine;

public abstract class UIPanel : MonoBehaviour
{
    // Métodos abstractos para mostrar y ocultar el panel
    public abstract void Mostrar();
    public abstract void Ocultar();
}
```

Script MenuPrincipal

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;
public class MenuPrincipal : UIPanel
{

    [SerializeField] private Button botonJugar;
    [SerializeField] private Button botonNiveles;
    [SerializeField] private Button botonAjustes;
    [SerializeField] private Button botonCreditos;
    [SerializeField] private Button botonSalir;
    [SerializeField] private GestorMenu gestor;
    public override void Mostrar()
    {
        gameObject.SetActive(true);
        if (gestor == null)
        {
            Debug.LogError("GestorMenu no asignado en MenuPrincipal");
            return;
        }
```






```
        botonJugar.onClick.RemoveAllListeners();
        botonJugar.onClick.AddListener(() =>
SceneManager.LoadScene("Nivel_1_Arbol"));

        botonNiveles.onClick.RemoveAllListeners();
        botonNiveles.onClick.AddListener(() => gestor.MostrarPanel(1));

        botonAjustes.onClick.RemoveAllListeners();
        botonAjustes.onClick.AddListener(() => gestor.MostrarPanel(2));

        botonCreditos.onClick.RemoveAllListeners();
        botonCreditos.onClick.AddListener(() => gestor.MostrarPanel(3));

        botonSalir.onClick.RemoveAllListeners();
        botonSalir.onClick.AddListener(() => gestor.Salir());

    }

    public override void Ocultar()
    {
        gameObject.SetActive(false);
    }
}
```

Script MenuNiveles

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;

public class MenuNiveles : UIPanel
{
    [SerializeField] private Button botonNivel1;
    [SerializeField] private Button botonNivel2;
    [SerializeField] private Button botonNivel3;
    [SerializeField] private Button botonAtras;
    [SerializeField] private GestorMenu gestor;
    public override void Mostrar()
    {
        gameObject.SetActive(true);
        if (gestor == null)
        {
            Debug.LogError("GestorMenu no asignado en MenuNiveles");
            return;
        }
```






```
        botonNivel1.onClick.RemoveAllListeners();
        botonNivel1.onClick.AddListener(() =>
SceneManager.LoadScene("Nivel_1_Arbol"));

        botonNivel2.onClick.RemoveAllListeners();
        botonNivel2.onClick.AddListener(() =>
SceneManager.LoadScene("Nivel_2_Raices"));

        botonNivel3.onClick.RemoveAllListeners();
        botonNivel3.onClick.AddListener(() =>
SceneManager.LoadScene("Nivel_3_Purificacion"));

        botonAtras.onClick.RemoveAllListeners();
        botonAtras.onClick.AddListener(() => gestor.MostrarPanel(0));
    }

    public override void Ocultar()
    {
        gameObject.SetActive(false);
    }
    private void OnEnable()
    {
        // 3. Traer el dato guardado
        int highestLevel = PlayerPrefs.GetInt("HighestLevelReached", 1);

        // 4. Aplicar la lógica condicional

        // (Asumimos que el Nivel 1 siempre está disponible)
        botonNivel1.interactable = true;

        // El Nivel 2 solo es interactuable si el progreso es 2 o más
        botonNivel2.interactable = (highestLevel >= 2);

        // El Nivel 3 solo si el progreso es 3 o más
        botonNivel3.interactable = (highestLevel >= 3);
    }
}
```

Script MenuAjustes

```
using UnityEngine;
using UnityEngine.UI;

public class MenuAjustes : UIPanel
{
```



```
    [SerializeField] private Button botonMusica;
    [SerializeField] private Button botonEfectos;
    [SerializeField] private Button botonAtras;
    [SerializeField] private GestorMenu gestor;

    public override void Mostrar()
    {
        gameObject.SetActive(true);
        if (gestor == null)
        {
            Debug.LogError("GestorMenu no asignado en MenuAjustes");
            return;
        }
        botonAtras.onClick.RemoveAllListeners();
        botonAtras.onClick.AddListener(() => gestor.MostrarPanel(0));
    }

    public override void Ocultar()
    {
        gameObject.SetActive(false);
    }
}
```

Script MenuCreditos

```
using UnityEngine;
using UnityEngine.UI;


public class MenuCreditos : UIPanel
{
    [SerializeField] private Button botonAtras;
    [SerializeField] private GestorMenu gestor;

    public override void Mostrar()
    {
        gameObject.SetActive(true);
        if (gestor == null)
        {
            Debug.LogError("GestorMenu no asignado en MenuCreditos");
            return;
        }
        botonAtras.onClick.RemoveAllListeners();
        botonAtras.onClick.AddListener(() => gestor.MostrarPanel(0));
    }
```






```
    public override void Ocultar()
    {
        gameObject.SetActive(false);
    }
}
```

Mosquito con Strategy

Fase de análisis:

Historia de usuario HU13:

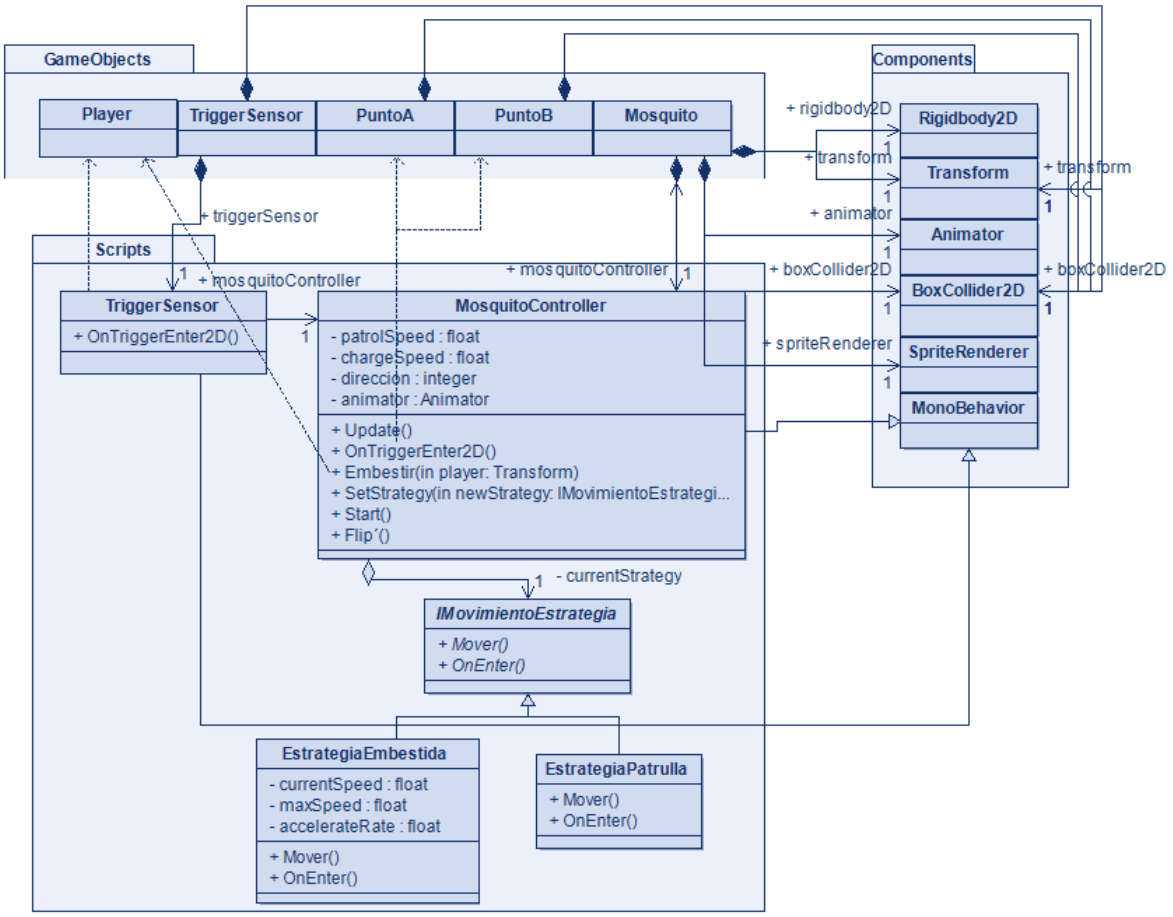
Quién: Enemigo Mosquito
Qué: Necesito patrullar kinemáticamente entre dos Puntos Límite (`Transform`) y cambiar mi estrategia a una "Embestida Acelerada" cuando mi sensor detecte al jugador
Para qué: para que pueda atacar de forma dinámica, volviendo a patrullar solo al chocar con un Punto Límite
Tabla EPS

Entrada	Proceso	Salida
1. `OnTriggerEnter2D` (del `TriggerSensor` detectando al Player). 2. `Mosquito.transform.position` (posición actual). 3. `puntoLimite1.position` y `puntoLimite2.position` (los *waypoints*).	1. `MosquitoController` (Context) mantiene la `currentStrategy` y la `direccion`. 2. `FixedUpdate` llama a `CheckBoundsAndDirection()` (lógica de decisión) para ver si debe invertir la `direccion` o cambiar a `Patrulla`. 3. `FixedUpdate` delega el movimiento a `currentStrategy.Move(this)`. 4. `TriggerSensor` detecta al Player y llama a	El `transform.position` del Mosquito cambia (a `patrolSpeed` constante o acelerando de `patrolSpeed` a `chargeSpeed`).

	`MosquitoController.Embestir()`. 5. `Embestir()` calcula la nueva `direccion` y llama a `SetStrategy(EstrategiaEmbestida)`, que a su vez llama a `OnEnter()` para iniciar la aceleración.	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU13

- **Interfaz:** Crear IMovimientoEstrategia con los métodos OnEnter(MosquitoController c) y Move(MosquitoController c).



- **Estrategias (Scripts):**
    - Crear EstrategiaPatrulla (implementa Move usando transform.position y patrolSpeed).
    - Crear EstrategiaEmbestida (implementa OnEnter para resetear currentSpeed a patrolSpeed; implementa Move para acelerar currentSpeed hacia chargeSpeed y mover el transform).
- **Controlador (Script):**
    - Crear MosquitoController.cs.
    - Implementar la lógica de FixedUpdate (que llama a CheckBoundsAndDirection y currentStrategy.Move).
    - Implementar CheckBoundsAndDirection (compara transform.position con puntoLimite1 y puntoLimite2 para invertir la direccion).
    - Implementar SetStrategy(IMovimientoEstrategia newStrategy) (que llama a newStrategy.OnEnter).
    - Implementar public void Embestir(Transform player) (calcula la dirección y llama a SetStrategy).
- **Sensor (Script):**
    - Crear TriggerSensor.cs (detecta al Player, llama a GetComponentInParent<MosquitoController>().Embestir()).
- **Prefab (Ensamblaje):**
    - Ensamblar el Prefab del Mosquito (sin Rigidbody) y adjuntar MosquitoController.cs.
    - Crear los GameObjects vacíos PuntoLimite1 y PuntoLimite2 en la escena y asignarlos al Inspector del Mosquito.
    - Crear el hijo "Sensor" (con Collider2D Trigger) y adjuntarle TriggerSensor.cs.
    - Crear el hijo "Aguijón" y adjuntarle el Prefab TargetDamage.

Criterios de Aceptación HU13

- El mosquito patrulla (moviendo transform) entre puntoLimite1 y puntoLimite2 a patrolSpeed.
- Al tocar el "Sensor" (Trigger), el mosquito *comienza* a moverse a patrolSpeed y *acelera* hasta chargeSpeed en dirección al jugador.
- Si el mosquito (en cualquier estado) alcanza puntoLimite1 o puntoLimite2, invierte su dirección.
- Si el mosquito estaba en "Embestida" y toca un límite, vuelve al estado de "Patrulla".
- El script TriggerSensor llama exitosamente a Embestir() en el MosquitoController al detectar al Player.

Codificacion HU13

Interface IMovimientoEstrategia

```
using UnityEngine;
```






```
// Su única tarea es mover el Transform del mosquito.
public abstract class IMovimientoEstrategia
{
    /// <summary>
    /// Ejecuta la lógica de movimiento usando el Transform.
    /// </summary>
    /// <param name="controller">El mosquito que se está moviendo (para leer
sus datos)</param>
    public abstract void Move(MosquitoController controller);
    public abstract void OnEnter(MosquitoController controller);
}
```

Script EstrategiaPatrulla

```
using UnityEngine;

// Implementa la interfaz con la lógica de patrulla lenta.
// Mueve el objeto modificando su 'transform.position'.
public class EstrategiaPatrulla : IMovimientoEstrategia
{
    public override void Move(MosquitoController controller)
    {
        // 1. Obtener los datos del controlador
        float speed = controller.patrolSpeed;
        int direccion = controller.direccion;

        // 2. Calcular el vector de movimiento
        Vector2 movimiento = new Vector2(direccion * speed *
Time.fixedDeltaTime, 0f);

        // 3. Aplicar el movimiento al transform
        // (Usamos fixedDeltaTime porque el movimiento se llama desde
FixedUpdate)
        controller.transform.position += (Vector3)movimiento;
    }

    public override void OnEnter(MosquitoController controller)
    {
        // No se necesita lógica especial al entrar en patrulla
    }
}
```

Script EstrategiaEmbestida

```
using UnityEngine;
```






```
public class EstrategiaEmbestida : IMovimientoEstrategia
{
    // Estado interno de la estrategia
    private float currentSpeed;
    private float maxSpeed;

    // Qué tan rápido acelera
    private float accelerationRate = 20f;

    /// <summary>
    /// Se llama cuando el mosquito decide embestir.
    /// Reinicia la velocidad para que empiece desde la velocidad de
patrulla.
    /// </summary>
    public override void OnEnter(MosquitoController controller)
    {
        currentSpeed = controller.patrolSpeed; // ¡Empieza lento!
        maxSpeed = controller.chargeSpeed;     // Define el tope
    }

    /// <summary>
    /// Se llama cada frame (FixedUpdate).
    /// Acelera y luego se mueve.
    /// </summary>
    public override void Move(MosquitoController controller)
    {
        // 1. ACELERAR
        // Aumenta 'currentSpeed' hacia 'maxSpeed',
        // sin pasarse, usando 'accelerationRate'.
        currentSpeed = Mathf.MoveTowards(
            currentSpeed,
            maxSpeed,
            accelerationRate * Time.fixedDeltaTime
        );

        // 2. OBTENER DATOS
        int direccion = controller.direccion;

        // 3. CALCULAR MOVIMIENTO
        // Usa la 'currentSpeed' (que está cambiando)
        Vector2 movimiento = new Vector2(direccion * currentSpeed *
Time.fixedDeltaTime, 0f);

        // 4. APLICAR MOVIMIENTO
        controller.transform.position += (Vector3)movimiento;
```



```
    }
}
```

Script MosquitoController

```
using UnityEngine;

public class MosquitoController : MonoBehaviour
{
    [Header("Configuración")]
    [SerializeField] private float _patrolSpeed = 2f;
    [SerializeField] private float _chargeSpeed = 50f;
    [Tooltip("Tag de los objetos que marcan el fin de la patrulla (Puntos
A/B)")]
    [SerializeField] private Animator animator;

    // --- Propiedades Públicas ---
    public float patrolSpeed => _patrolSpeed;
    public float chargeSpeed => _chargeSpeed;
    public int direccion { get; private set; } = 1;

    // --- Estado Interno ---
    private IMovimientoEstrategia currentStrategy;
    private readonly EstrategiaPatrulla stratPatrulla = new
EstrategiaPatrulla();
    private readonly EstrategiaEmbestida stratEmbestida = new
EstrategiaEmbestida();

    private void Start()
    {
        SetStrategy(stratPatrulla);
    }

    private void Update()
    {
        // Usamos Update() porque ahora movemos el Transform, no físicas.
        currentStrategy?.Move(this);
    }

    // --- Detección de Puntos de Patrulla ---
    private void OnTriggerEnter2D(Collider2D other)
    {
        Debug.Log("Mosquito tocó: " + other.name);
        // Si tocamos un "PatrolPoint"...
        if (other.CompareTag("PatrolPoint"))
        {
```



```
            // ...invertimos la dirección.
            direccion *= -1;
            Flip();

            // Si estaba embistiendo, vuelve a patrullar.
            if (currentStrategy == stratEmbestida)
            {
                SetStrategy(stratPatrulla);
            }
        }
    }

    // --- Método Público para el Sensor ---
    public void Embestir(Transform player)
    {
        if (currentStrategy == stratEmbestida) return;

        // Calcular dirección hacia el jugador
        float dirToPlayer = player.position.x - transform.position.x;
        int nuevaDireccion = (dirToPlayer > 0) ? 1 : -1;

        if (nuevaDireccion != direccion)
        {
            direccion = nuevaDireccion;
            Flip();
        }

        SetStrategy(stratEmbestida);
        currentStrategy.OnEnter(this);
        animator.SetBool("embistiendo", true);
    }

    private void SetStrategy(IMovimientoEstrategia newStrategy)
    {
        currentStrategy = newStrategy;
    }

    private void Flip()
    {
        // Invertir el sprite visualmente
        Vector3 scale = transform.localScale;
        scale.x = Mathf.Abs(scale.x) * -direccion;
        transform.localScale = scale;
        animator.SetBool("embistiendo", false);
    }
}
```






Script TriggerSensor

```
using UnityEngine;

[RequireComponent(typeof(Collider2D))]
public class TriggerSensor : MonoBehaviour
{
    [SerializeField] private MosquitoController mosquitoController;

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            // Llama al Controlador para que maneje la lógica
            mosquitoController.Embestir(other.transform);
        }
    }
}
```

NPC Merc-Ant

Fase de análisis:

Historia de usuario HU14:

Quién: Como el NPC Merc-Ant
Qué: Necesito reproducir una secuencia de diálogo (sprites) al detectar al jugador por primera vez y luego mostrar un aviso ('E')
Para qué: para que el jugador sepa que puede presionar 'E' para intercambiar 1 Savia por el Mapa (solo si aún no lo tiene)
Tabla EPS

Entrada	Proceso	Salida
1. `OnTriggerEnter2D` (Player entra al collider). 2. `OnTriggerExit2D` (Player sale).	1. **(Trigger Enter):** `OnTriggerEnter2D` se activa. Comprueba si `playerStats.HasMap()` es `true` (si es así, ignora todo). Si `haSaludado` es `false`, inicia la corrutina `SaludoInicial()`.	1. El `panelDialogo` (con `imagenDialogo`) se muestra y oculta. 2. El `interactPrompt` ('E') se muestra y oculta.

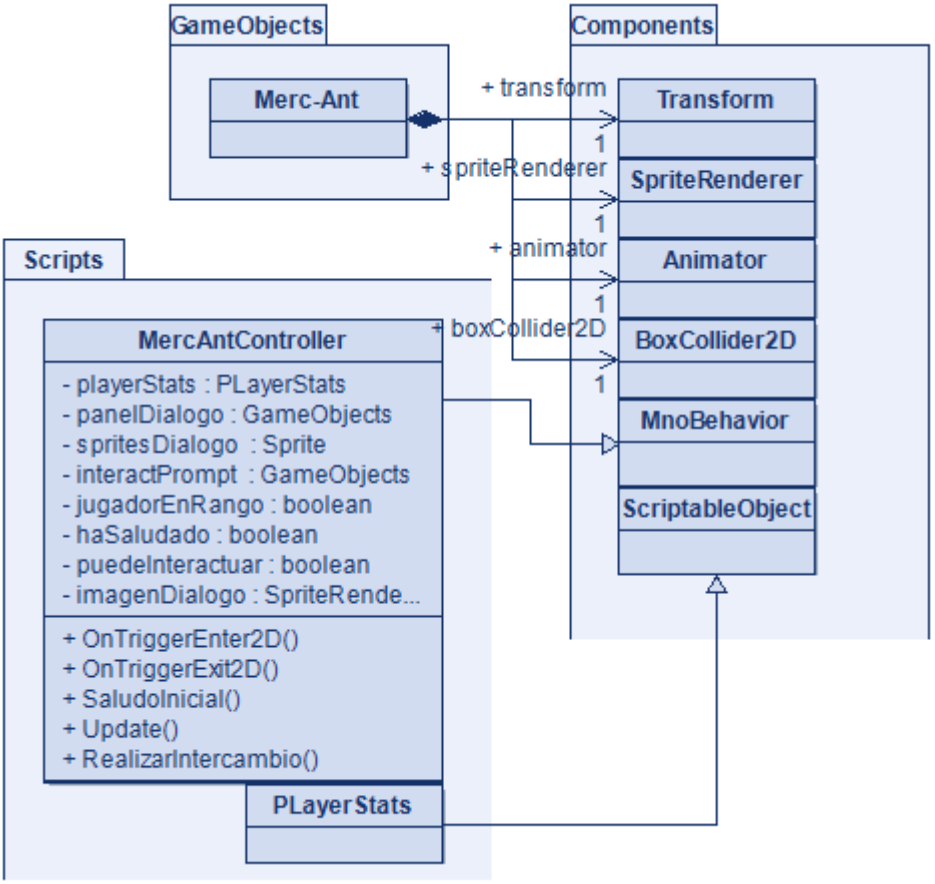




3. `GameInput.GetKeyDown(KeyCode.E)`.	Setea `jugadorEnRango = true`. 2. **(Corrutina):** `SaludoInicial()` muestra el `panelDialogo`, recorre el array `spritesDialogo` (mostrando cada sprite y esperando), oculta el panel, y setea `puedeInteractuar = true`. 3. **(Update):** Comprueba constantemente: a. `interactPrompt.SetActive(jugadorEnRango && puedeInteractuar)`. (Maneja el aviso 'E'). b. Si (Input 'E' Y `jugadorEnRango` Y `puedeInteractuar`), llama a `RealizarIntercambio()`. 4. **(Intercambio):** `RealizarIntercambio()` llama a `playerStats.ConsumeSap(1)`. Si tiene éxito, llama a `playerStats.GrantMap()` y setea `puedeInteractuar = false` (para ocultar el aviso 'E' y finalizar).	3. `PlayerStats` pierde 1 Savia y gana el Mapa.

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU14

- Crear el script MercAntController.cs.
- Añadir variables [SerializeField] para: playerStats, panelDialogo (Image), spritesDialogo (array Sprite[]), y interactPrompt (GameObject del aviso 'E').
- Añadir variables de estado privadas (bool jugadorEnRango, bool haSaludado, bool puedeInteractuar).
- Implementar OnTriggerEnter2D:
    - Comprobar si playerStats.HasMap() (ignorar si es true).
    - Setear jugadorEnRango = true.
    - Iniciar SaludoInicial() (solo si haSaludado es false).
- Implementar OnTriggerExit2D:
    - Setear jugadorEnRango = false.
- Implementar la Corrutina IEnumerator SaludoInicial():
    - Setear haSaludado = true.
    - Activar panelDialogo, iterar sobre spritesDialogo (cambiando imagenDialogo.sprite y usando WaitForSeconds), ocultar panelDialogo, y setear puedeInteractuar = true.
- Implementar Update():



    - Gestionar la visibilidad del aviso: interactPrompt.SetActive(jugadorEnRango && puedeInteractuar).
    - Detectar el input 'E' (solo si jugadorEnRango y puedeInteractuar).
- Implementar private void RealizarIntercambio():
    - Llamar a playerStats.ConsumeSap(1).
    - Si es exitoso, llamar a playerStats.GrantMap() y setear puedeInteractuar = false.
- Configurar el Prefab de Merc-Ant (Collider Is Trigger, MercAntController.cs, y arrastrar todas las referencias del Canvas y PlayerStats).

Criterios de Aceptación HU14

- Merc-Ant ignora al jugador si este ya tiene el mapa.
- La primera vez que el jugador se acerca, se reproduce la secuencia de diálogo (todos los *sprites* del *array*).
- El aviso de interacción ('E') **no** se muestra durante la secuencia de diálogo.
- El aviso de interacción ('E') **sí** se muestra cuando el jugador está en rango *y* la secuencia de diálogo ha terminado.
- Presionar 'E' (mientras el aviso está visible) consume 1 Savia y otorga el Mapa (PlayerStats se actualiza).
- Después del intercambio, el aviso ('E') desaparece permanentemente (porque puedeInteractuar es false).
- Si el jugador se aleja, el aviso ('E') desaparece. Si vuelve a acercarse (y aún no ha comprado), el aviso reaparece (sin repetir el diálogo).

Codificacion HU14

Script MercAntController

```
using UnityEngine;
using System.Collections;
public class MercAntController : MonoBehaviour
{
    [SerializeField] private PlayerStats playerStats;
    [SerializeField] private SpriteRenderer imagenDialogo;
    [SerializeField] private Sprite[] spritesDialogo;
    [SerializeField] private GameObject interactPrompt;
    [SerializeField] private GameObject panelDialogo;
    private bool jugadorEnRango = false;
    private bool haSaludado = false;
    private bool puedeInteractuar = false;
    private Coroutine dialogoCoroutine = null;

    void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
```



```
        {
            jugadorEnRango = true;

            // 1. ¿Ya tiene el mapa? Si es así, no hacer nada.
            if (playerStats.HasMap()) return;

            // 2. Si es la primera vez que entra (y no ha saludado), iniciar
saludo.
            if (!haSaludado)
            {
                // Iniciar la corrutina de saludo
                dialogoCoroutine = StartCoroutine(SaludoInicial());
            }
            else
            {
                // 3. Si ya saludó (y no ha comprado), marcar como listo
para interactuar
                puedeInteractuar = true;
            }
        }
    }

    void OnTriggerExit2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            jugadorEnRango = false;
            puedeInteractuar = false; // No se puede interactuar si está
fuera de rango

            // Ocultar todos los prompts
            interactPrompt.SetActive(false);
            panelDialogo.SetActive(false);

            // Si el jugador se va a mitad del saludo, detener la corrutina
            if (dialogoCoroutine != null)
            {
                StopCoroutine(dialogoCoroutine);
                dialogoCoroutine = null;
                haSaludado = false; // Permitir que el saludo se repita si
se interrumpió
            }
        }
    }

    public IEnumerator SaludoInicial()
```






```
    {
        haSaludado = true;
        panelDialogo.SetActive(true); // Mostrar la caja de diálogo

        // Iterar sobre todos los sprites en el array
        foreach (Sprite spriteMsg in spritesDialogo)
        {
            // Actualizar la imagen del diálogo
            imagenDialogo.sprite = spriteMsg;
            yield return new WaitForSeconds(4f);
        }

        // Saludo terminado
        puedeInteractuar = true; // Ahora sí puede presionar 'E'
        dialogoCoroutine = null;

    }

    void Update()
    {
        if (GameManager.IsInputLocked()) return;
        // 1. Gestionar el aviso de Interacción ('E')
        // (Usamos un if separado para que el aviso se muestre/oculte en
tiempo real)
        bool mostrarAviso = jugadorEnRango && puedeInteractuar &&
!playerStats.HasMap();
        if (interactPrompt.activeSelf != mostrarAviso)
        {
            interactPrompt.SetActive(mostrarAviso);
        }

        // 2. Comprobar si el jugador presiona 'E'
        if (mostrarAviso && Input.GetKeyDown(KeyCode.E))
        {
            RealizarIntercambio();
        }
    }

    void RealizarIntercambio()
    {
        // Intentar consumir 1 Savia (el método devuelve true si tuvo éxito)
        if (playerStats.ReleaseSap(1))
        {
            // ÉXITO: Dar el mapa
            playerStats.GrantMap();
            puedeInteractuar = false; // Desactivar futuras interacciones
```



```
            panelDialogo.SetActive(false);

        }
        else
        {
            // FALLO: No tiene savia
            Debug.Log("Merc-Ant: ¡No tienes suficiente savia!");
        }
    }
}
```

Ataque de Raíz

Fase de análisis:

Historia de usuario HU15:

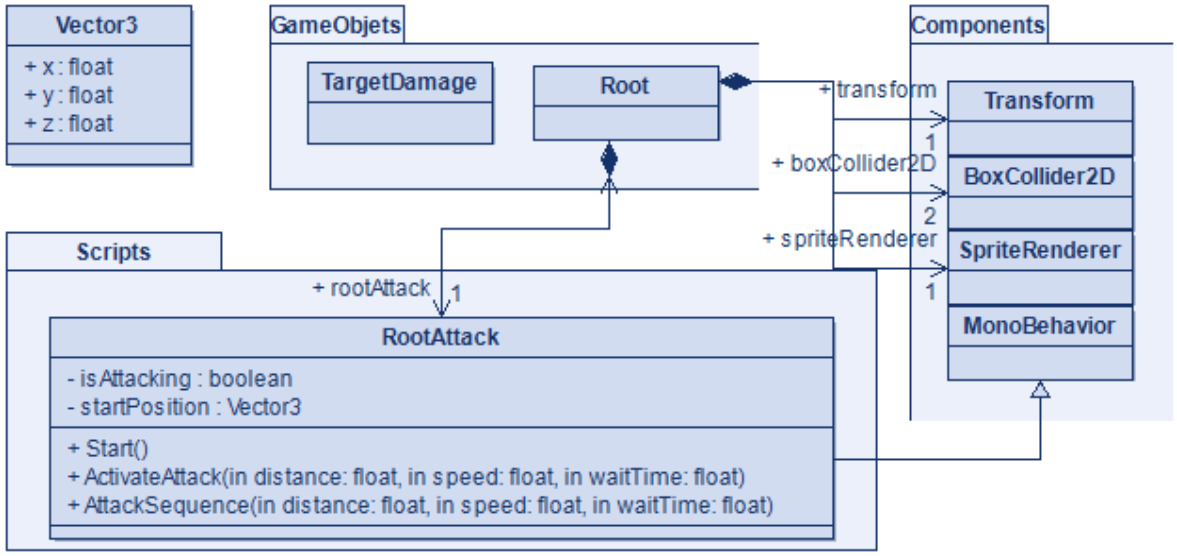
Quién: Como un Prefab de Raíz (`RootAttack`)
Qué: Necesito un método público `ActivateAttack()` que reciba parámetros (distancia, velocidad, espera)
Para qué: para que pueda ejecutar una corrutina de ataque (extenderse, esperar, y retraerse) cuando el "Controlador del Jefe" me lo ordene
Tabla EPS

Entrada	Proceso	Salida
Un script externo (el `BossController`) llama a `public void ActivateAttack(float distance, float speed, float waitTime)`.	1. `ActivateAttack()` comprueba si la raíz ya está atacando (usando un `bool isAttacking`). Si lo está, ignora la llamada. 2. Si no está atacando, setea `isAttacking = true` y guarda la `startPosition`. 3. Inicia la corrutina `AttackSequence(distance, speed, waitTime)`.	El `GameObject` de la Raíz (y su `TargetDamage` hijo) se mueve físicamente hacia adelante y luego hacia atrás.

	4. **(Corrutina):** Mueve el `transform` hacia la `targetPosition` (startPos + distance) a la `speed` dada. 5. **(Corrutina):** Llama a `yield return new WaitForSeconds(waitTime)`. 6. **(Corrutina):** Mueve el `transform` de vuelta a la `startPosition`. 7. **(Corrutina):** Setea `isAttacking = false` (lista para el próximo ataque).	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU15



- Crear el script C# RootAttack.cs.
- Definir una variable privada bool isAttacking = false (para prevenir ataques múltiples/superpuestos).
- Definir una variable privada Vector3 startPosition (para saber dónde está el "hogar").
- Implementar Start() para que guarde la startPosition inicial.
- Implementar el método público **public void ActivateAttack(float distance, float speed, float waitTime)**.
- Implementar la corrutina privada **IEnumerator AttackSequence(float distance, float speed, float waitTime)**.
- En ActivateAttack(), implementar la lógica de seguridad (chequear if (isAttacking) return;) antes de iniciar la corrutina.
- En AttackSequence(), implementar la lógica de movimiento (usando Vector3.MoveTowards en bucles while), la espera (WaitForSeconds), y el reseteo del *flag* isAttacking.
- Crear el Prefab RootAttack:
    - Un GameObject padre con el script RootAttack.cs.
    - Un GameObject hijo ("Punta") que contenga el Prefab TargetDamage (para que haga daño al tocar al jugador).

Criterios de Aceptación HU15

- La raíz permanece inmóvil (isAttacking = false) por defecto.
- Cuando el BossController llama a ActivateAttack(5, 10, 2), la raíz se mueve 5 unidades, a 10 de velocidad, y espera 2 segundos.
- Después de esperar, la raíz regresa a su startPosition original.
- Si ActivateAttack() es llamada mientras la raíz ya está en movimiento o esperando, la nueva llamada es ignorada.
- El GameObject hijo "Punta" (con TargetDamage) se mueve junto con la raíz y puede dañar al jugador.

Codificacion HU15

Script RootAttack

```
using UnityEngine;
using System.Collections;

public class RootAttack : MonoBehaviour
{
    private bool isAttacking = false;
    private Vector3 startPosition;

    // Almacena la corrutina actual para poder detenerla si es necesario
    private Coroutine activeAttackCoroutine;
```



```
    /// <summary>
    /// Guarda la posición "de hogar" al iniciar.
    /// </summary>
    private void Start()
    {
        startPosition = transform.position;
    }

    // --- 1. El Método de Activación (Llamado por el Controller) ---

    /// <summary>
    /// Inicia la secuencia de ataque.
    /// El Controller llama a esto.
    /// </summary>
    /// <param name="distance">Distancia (en unidades) que se extenderá la
raíz.</param>
    /// <param name="speed">Velocidad de movimiento (unidades por
segundo).</param>
    /// <param name="waitTime">Tiempo (en segundos) que esperará
extendida.</param>
    public void ActivateAttack(float distance, float speed, float waitTime)
    {
        // Si ya está atacando, ignora esta nueva orden.
        if (isAttacking)
        {
            return;
        }

        // Iniciar la corrutina y guardar la referencia
        activeAttackCoroutine = StartCoroutine(AttackSequence(distance,
speed, waitTime));
    }

    // --- 2. La Secuencia de Ataque (La Corrutina) ---

    /// <summary>
    /// Corrutina que maneja la secuencia completa: Extender, Esperar,
Retraer.
    /// </summary>
    private IEnumerator AttackSequence(float distance, float speed, float
waitTime)
    {
        // 1. Marcar como "ocupada"
        isAttacking = true;
```






```
        float tiempoInicio = Time.time;

        while (Time.time < tiempoInicio + 1f)
        {
            // Calcular un offset aleatorio en X e Y
            // (Usamos Vector3 para que sea compatible con startPosition)
            Vector3 randomOffset = (Vector3)Random.insideUnitCircle * 0.1f;

            // Aplicar el offset a la posición inicial (para que no se
"vaya" del sitio)
            transform.position = startPosition + randomOffset;

            yield return null; // Esperar al próximo frame
        }

        // Asegurarse de que termine en la posición inicial exacta antes de
moverse
        transform.position = startPosition;

        // 2. Calcular posiciones
        // (Usamos 'transform.right' asumiendo que la raíz apunta hacia la
derecha.
        // Cambia esto a transform.up, .forward, etc., según la rotación de
tu prefab)
        Vector3 targetPosition = startPosition + (transform.right *
distance);

        // --- 3. FASE DE EXTENSIÓN ---
        // Moverse desde 'startPosition' hasta 'targetPosition'
        while (Vector3.Distance(transform.position, targetPosition) > 0.01f)
        {
            transform.position = Vector3.MoveTowards(
                transform.position,
                targetPosition,
                speed * Time.deltaTime // Usar Time.deltaTime para un
movimiento suave
            );
            yield return null; // Esperar al próximo frame
        }
        // Asegurarse de que quede exacto al final
        transform.position = targetPosition;

        // --- 4. FASE DE ESPERA ---
        yield return new WaitForSeconds(waitTime);

        // --- 5. FASE DE RETRACCIÓN ---
```




```
        // Moverse desde 'targetPosition' de vuelta a 'startPosition'
        while (Vector3.Distance(transform.position, startPosition) > 0.01f)
        {
            transform.position = Vector3.MoveTowards(
                transform.position,
                startPosition,
                speed * Time.deltaTime // Usar la misma velocidad para
volver
            );
            yield return null; // Esperar al próximo frame
        }
        // Asegurarse de que quede exacto al final
        transform.position = startPosition;

        // 6. Marcar como "disponible"
        isAttacking = false;
        activeAttackCoroutine = null;
    }
}
```

Gusano Nivel 2

Fase de análisis:

Historia de usuario HU16:

Quién: Como el Gusano Terrario (Nivel 2)
Qué: Necesito reproducir un diálogo, esperar la tecla 'E', y luego ejecutar una corrutina de excavación que me mueva fuera de la pantalla
Para qué: para que al finalizar, pueda destruir el `GameObject` 'TapaTúnel' y desbloquear el camino para el jugador
Tabla EPS

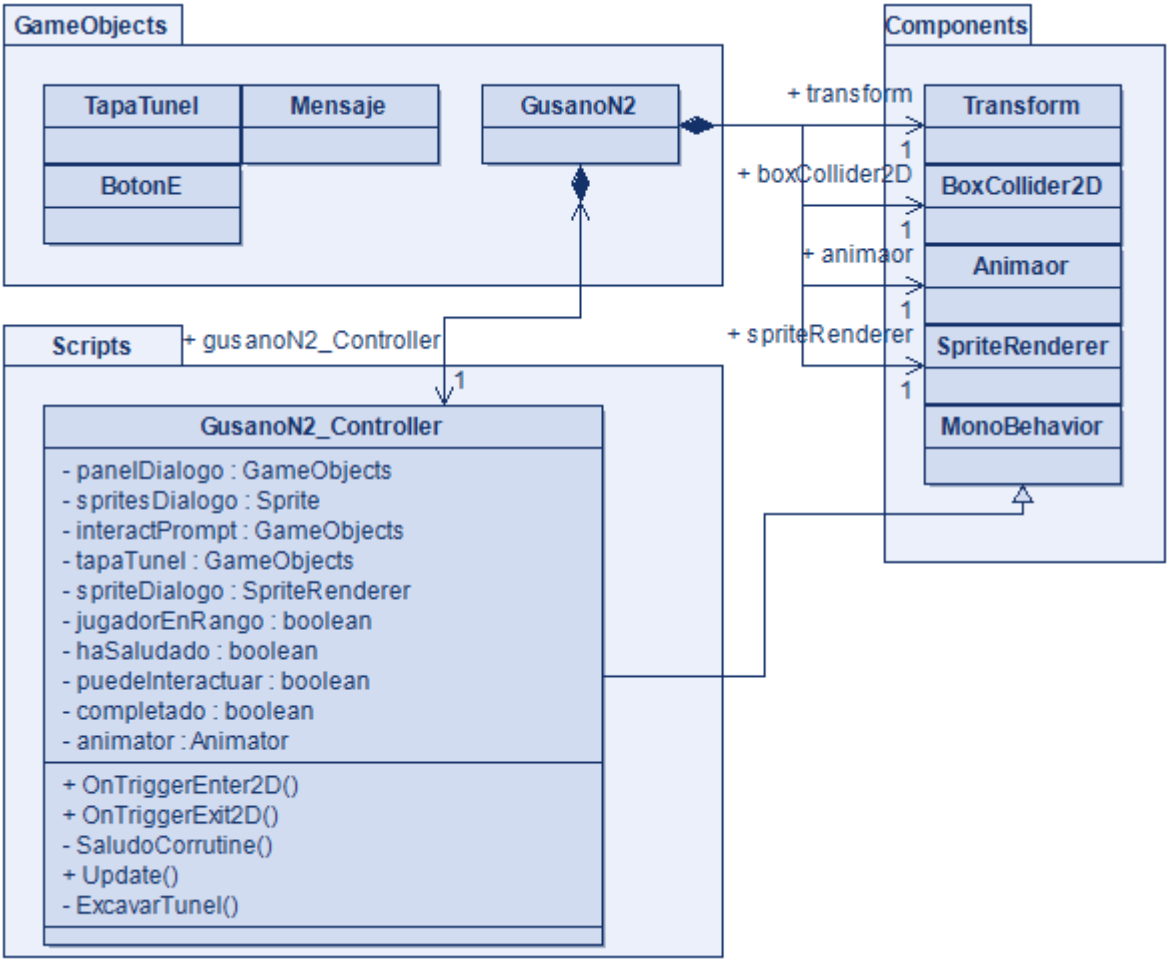
Entrada	Proceso	Salida
1. `OnTriggerEnter2D` (Player entra al collider). 2. `OnTriggerExit2D` (Player sale).	1. **(Trigger Enter):** Detecta al Player. Si es la primera vez (`!haSaludado` y `!completado`), inicia `SaludoCoroutine()`. Setea `jugadorEnRango = true`.	1. El diálogo y el aviso 'E' se muestran y ocultan. 2. El `Gusano` se mueve y se autodestruye



3. `GameInput.GetKeyDown(KeyCode.E)`.	2. **(Corrutina 1 - Saludo):** Muestra el `panelDialogo` (sprites en array), espera, oculta el panel, y setea `puedeInteractuar = true`. 3. **(Update):** Gestiona el aviso 'E' (`interactPrompt.SetActive(...)`). Si el jugador presiona 'E' (y puede), llama a `StartCoroutine(ExcavarTunel())`. 4. **(Corrutina 2 - Excavación):** Setea `completado = true`. Mueve el `transform` del gusano hacia un destino (fuera de pantalla).	(`Destroy(gameObject)`). 3. El `tapaTunel` (referencia externa) se destruye (`Destroy(tapaTunel)`), despejando el camino.

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU16

- Crear el GameObject Gusano_Nivel_2 en la escena.
- Crear el script GusanoNivel2_Controller.cs y adjuntarlo.
- Crear el GameObject TapaTunel_Izquierda (con un Collider2D) y colocarlo bloqueando el camino.
- En GusanoNivel2_Controller.cs, definir [SerializeField] para: panelDialogo (Image), spritesDialogo (array Sprite[]), interactPrompt (GO 'E'), y tapaTunel (GameObject).
- Añadir variables de estado privadas (bool jugadorEnRango, bool haSaludado, bool puedeInteractuar, bool completado).
- Implementar OnTriggerEnter2D y OnTriggerExit2D para gestionar jugadorEnRango.
- Implementar la Corrutina IEnumerator SaludoCoroutine() (mostrar sprites en secuencia, setear haSaludado = true y puedeInteractuar = true).
- Implementar Update() para gestionar el aviso 'E' y detectar el *input* 'E' (que llama a ExcavarTunel).
- Implementar la Corrutina IEnumerator ExcavarTunel():
    - Setear completado = true y puedeInteractuar = false.
    - Mover el transform del gusano (con MoveTowards) hasta un destino fuera de pantalla.



    - Al llegar, llamar a Destroy(tapaTunel).
    - Llamar a Destroy(gameObject) (para autodestruirse).
- Conectar todas las referencias (paneles, sprites, tapaTunel) en el Inspector del Gusano_Nivel_2.

Criterios de Aceptación HU16

- El gusano ignora al jugador si la excavación ya está completada.
- Al acercarse por primera vez, se reproduce la secuencia de diálogo (sprites).
- El aviso 'E' solo aparece *después* del diálogo y *mientras* el jugador está en rango.
- Presionar 'E' (mientras el aviso está visible) inicia la corrutina de excavación.
- Durante la excavación, el gusano se mueve horizontalmente hasta salir de la pantalla.
- Una vez que el gusano desaparece, el GameObject tapaTunel se destruye, permitiendo el paso.
- El GameObject del gusano se destruye después de completar su tarea.

Codificacion HU16

Script GusanoN2Controller

```
using UnityEngine;
using System.Collections;

public class GusanoN2Controller : MonoBehaviour
{
    [SerializeField] public GameObject panelDialogo;
    [SerializeField] private SpriteRenderer imagenDialogo;
    [SerializeField] private Sprite[] spritesDialogo;
    [SerializeField] private GameObject interactPrompt;
    [SerializeField] private GameObject tapaTunel;
    [SerializeField] private Animator animator;
    // --- Variables de Estado (Privadas) ---
    private bool jugadorEnRango = false;
    private bool haSaludado = false;
    private bool puedeInteractuar = false;
    private bool completado = false; // Para que solo se active una vez

    private Coroutine dialogoCoroutine; // Para detener el diálogo si el
jugador se va
    // --- Detección del Jugador (Triggers) ---
    private void OnTriggerEnter2D(Collider2D other)
    {
        // Si la misión ya se completó o no es el Player, ignorar.
        if (other.CompareTag("Player"))
        {
```



```
            if (completado) return;
            jugadorEnRango = true;
            // Si es la primera vez que entra, iniciar saludo.
            if (!haSaludado)
            {
                dialogoCoroutine = StartCoroutine(SaludoCoroutine());
            }
            else
            {
                // Si ya saludó, solo permitir interactuar
                puedeInteractuar = true;
            }
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            // El jugador se fue del rango
            jugadorEnRango = false;
            puedeInteractuar = false;

            // Ocultar todos los prompts
            interactPrompt.SetActive(false);
            panelDialogo.SetActive(false);

            // Si el jugador se va a mitad del saludo, detener la corrutina
            if (dialogoCoroutine != null)
            {
                StopCoroutine(dialogoCoroutine);
                dialogoCoroutine = null;
                haSaludado = false; // Permitir que el saludo se repita si
se interrumpió
            }
        }
    }
    // --- Lógica de Interacción (Update) ---
    private void Update()
    {
        if (GameManager.IsInputLocked()) return;
        // 1. Gestionar el aviso de Interacción ('E')
        bool mostrarAviso = jugadorEnRango && puedeInteractuar &&
!completado;
        if (interactPrompt != null)
        {
```






```
            if (interactPrompt.activeSelf != mostrarAviso)
                interactPrompt.SetActive(mostrarAviso);
        }
        else if (mostrarAviso)
        {
            Debug.LogWarning("GusanoN2Controller: 'interactPrompt' no
asignado pero se intenta mostrar aviso.");
        }
        // 2. Comprobar si el jugador presiona 'E'
        if (mostrarAviso && Input.GetKeyDown(KeyCode.E))
        {
            // Iniciar la excavación
            StartCoroutine(ExcavarTunel());
        }
    }
    // --- Corrutina de Saludo ---
    private IEnumerator SaludoCoroutine()
    {
        haSaludado = true;
        if (panelDialogo != null)
            panelDialogo.SetActive(true); // Mostrar la caja de diálogo
        else
            Debug.LogWarning("GusanoN2Controller: 'panelDialogo' no asignado
al intentar mostrar saludo.");

        yield return new WaitForSeconds(1f);
        // Iterar sobre todos los sprites en el array
        foreach (Sprite spriteMsg in spritesDialogo)
        {
            if (imagenDialogo != null)
                imagenDialogo.sprite = spriteMsg;
            else
                Debug.LogWarning("GusanoN2Controller: 'imagenDialogo' no
asignado; no se puede mostrar sprite de diálogo.");
            yield return new WaitForSeconds(4f);
        }
        // Saludo terminado
        puedeInteractuar = true; // Ahora sí puede presionar 'E'
        dialogoCoroutine = null;
    }
    // --- Corrutina de Excavación (Lógica Simplificada) ---
    private IEnumerator ExcavarTunel()
    {
        // 1. Setear estados (para que no se pueda interactuar más)
        animator.SetBool("cavando", true);
        completado = true;
```



```
        puedeInteractuar = false;
        interactPrompt.SetActive(false);
        panelDialogo.SetActive(false);
        // 2. Definir el destino
        // (Usamos un Vector3 porque transform.position es Vector3)
        Vector3 destinoFinal = new Vector3(
            50f, // Coordenada X fuera de pantalla
            transform.position.y,
            transform.position.z
        );
        // 3. Mover el Gusano (y su hija, la 'tapaTunel')
        while (Vector3.Distance(transform.position, destinoFinal) > 0.01f)
        {
            transform.position = Vector3.MoveTowards(
                transform.position,
                destinoFinal,
                5f * Time.deltaTime
            );
            yield return null; // Esperar al próximo frame
        }
        // 4. Limpieza: Destruir el Gusano y la Tapa)
        tapaTunel.SetActive(false);
        Destroy(gameObject);
    }
}
```

Gusano Nivel 3

Fase de análisis:

Historia de usuario HU17:

Quién: Como el Gusano Terrario (Nivel 3)
Qué: Necesito separar mi lógica en un "Cerebro" (FSM), un "Portavoz" (Diálogo) y un "Excavador" (Movimiento)
Para qué: para que pueda gestionar la misión de recolección del huevo de forma ordenada, modular y sin sobrecargar un solo script
Tabla EPS

Entrada	Proceso	Salida

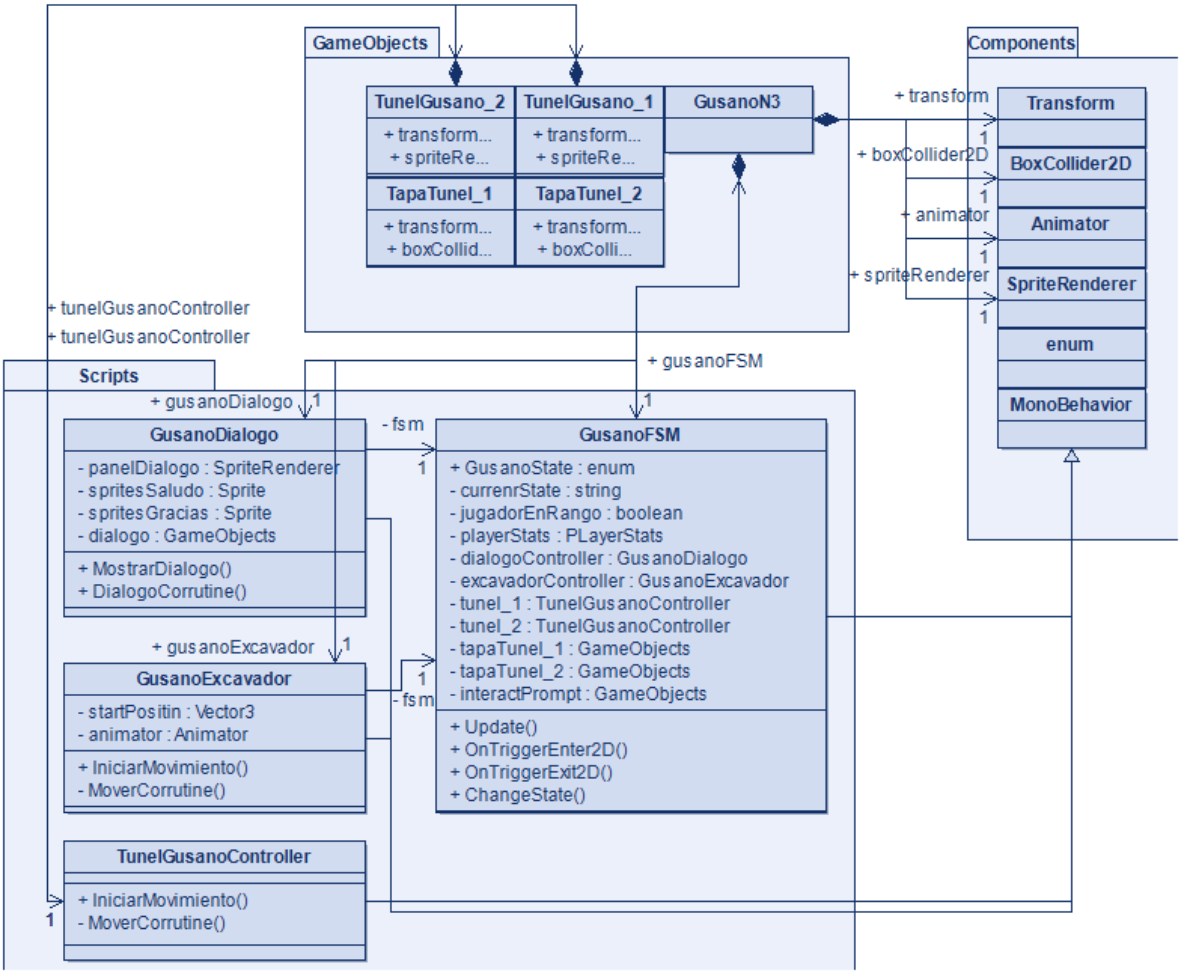




1. `OnTriggerEnter2D` (Player entra). 2. `GameInput.GetKeyDown(KeyCode.E)`. 3. `playerStats.HasMosquitoEgg()`.	1. **(Cerebro - `GusanoFSM`):** Detecta *inputs* (`Update`, `OnTriggerEnter`). 2. **(Cerebro - `GusanoFSM`):** Cambia su `currentState` (ej. a `DIALOGO_INICIAL`). 3. **(Cerebro - `GusanoFSM`):** Da órdenes a los otros scripts (ej. `gusanoDialogo.MostrarDialogo(currentState)`). 4. **(Portavoz/Músculo):** Los scripts (`GusanoDialogo`, `GusanoExcavador`) ejecutan sus corrutinas. 5. **(Portavoz/Músculo):** Al terminar sus corrutinas, notifican al Cerebro (ej. `fsm.ChangeState(GusanoState.ESPERANDO_INPUT)`). 6. **(Cerebro - `GusanoFSM`):** Repite el ciclo (ej. detecta 'E', cambia a `EXCAVANDO_TUNEL_1`, da la orden a `GusanoExcavador` y a `TapaTunelController`).	1. `GusanoDialogo` muestra/oculta la UI de diálogo. 2. `GusanoExcavador` mueve el `transform` del gusano. 3. `TapaTunelController` mueve el `transform` de la tapa. 4. `GusanoFSM` gestiona la destrucción de los `GameObjects` (tapas y gusano).

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU17

- **Script 1: GusanoFSM.cs (El Cerebro):**
  - Adjuntarlo al GameObject Gusano_Nivel_3.
  - Definir el enum GusanoState.
  - Implementar la función private void ChangeState(GusanoState newState) (que inicia las acciones).
  - Implementar Update() (solo para *escuchar* inputs 'E' o HasMosquitoEgg() según el estado).
  - Implementar OnTriggerEnter/Exit (para jugadorEnRango y para iniciar el DIALOGO_INICIAL).
  - Tener [SerializeField] a PlayerStats, GusanoDialogo, GusanoExcavador y las dos TapaTunelController.
- **Script 2: GusanoDialogo.cs (El Portavoz):**
  - Adjuntarlo al GameObject Gusano_Nivel_3.
  - Tener [SerializeField] para el panelDialogo, imagenDialogo, y **varios arrays** de Sprite[] (ej. spritesSaludo, spritesGracias).



    - Implementar public void MostrarDialogo(GusanoState estadoActual) (que inicia la corrutina correcta según el estado).
    - Las corrutinas, al terminar, deben llamar a fsm.ChangeState(...) para devolver el control.
- **Script 3: GusanoExcavador.cs (El Músculo):**
    - Adjuntarlo al GameObject Gusano_Nivel_3.
    - Implementar public void IniciarExcavacion(Vector3 destino, bool debeVolver) (el bool le dice si debe volver a startPosition o no).
    - La corrutina mueve al gusano y, al terminar, llama a fsm.ChangeState(...).
- **Script 4: TapaTunelController.cs (El Obstáculo):**
    - Adjuntarlo a los GameObject TapaVisual_1 y TapaVisual_2.
    - Implementar public void IniciarMovimiento(Vector3 destino, float velocidad).
    - La corrutina interna mueve el transform y al final llama a Destroy(gameObject).
- **Scripts Adicionales (Misión):**
    - Modificar PlayerStats.cs para que tenga el bool _hasMosquitoEgg (y sus métodos Grant/Has/Consume).
    - Crear el Prefab HuevoMosquito_Collectible que llame a playerStats.GrantMosquitoEgg().

Criterios de Aceptación HU17

- La lógica está separada: GusanoFSM solo gestiona estados, GusanoDialogo solo muestra diálogos, GusanoExcavador solo mueve al gusano.
- El GusanoFSM llama a GusanoDialogo y este muestra el *array* de *sprites* correcto según el estado (DIALOGO_INICIAL vs DIALOGO_HUEVO).
- La Corrutina en GusanoFSM (para EXCAVANDO_TUNEL_1) llama a GusanoExcavador e invoca a TapaTunelController.IniciarMovimiento() tras el retraso calculado.
- El GusanoExcavador mueve al gusano al Túnel 1 y luego **vuelve** a su startPosition.
- El GusanoFSM cambia a ESPERANDO_HUEVO y solo avanza si jugadorEnRango es true **y** playerStats.HasMosquitoEgg() es true.
- El GusanoExcavador mueve al gusano al Túnel 2 y **no** vuelve (se destruye).
- Las tapas visuales (TapaTunelController) se mueven y destruyen correctamente.
- Los TapaCollider (separados) se destruyen en el momento adecuado por el GusanoFSM.

Codificacion HU17

Script GusanoFSM

```
using UnityEngine;

public class GusanoFSM : MonoBehaviour
{
    // Estado del Gusano: público para que otros scripts puedan usar el tipo
```



```
si es necesario
    public enum GusanoState
    {
        ESPERANDO_JUGADOR, // (Estado inicial)
        DIALOGO_INICIAL,
        ESPERANDO_ACEPTAR_MISION,
        EXCAVANDO_TUNEL_1,
        ESPERANDO_HUEVO,
        DIALOGO_HUEVO,
        ESPERANDO_ENTREGAR_HUEVO,
        EXCAVANDO_TUNEL_2,
        COMPLETADO
    }
    private GusanoState currentState;
    public GusanoState CurrentState => currentState;
    private bool jugadorEnRango = false;
    [SerializeField] private PlayerStats playerStats;
    [SerializeField] private GusanoDialogo dialogoController;
    [SerializeField] private GusanoExcavador excavadarController;
    [SerializeField] private TunelGusanoController tunel_1;
    [SerializeField] private TunelGusanoController tunel_2;
    [SerializeField] private GameObject tapaTunel_1;
    [SerializeField] private GameObject tapaTunel_2;
    [SerializeField] private GameObject interactPrompt;

    void Start()
    {
        currentState = GusanoState.ESPERANDO_JUGADOR;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            jugadorEnRango = true;
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            jugadorEnRango = false;
        }
    }
    public void ChangeState(GusanoState newState)
```



```
    {
        currentState = newState;
        switch (currentState)
        {
            case GusanoState.DIALOGO_INICIAL:
                dialogoController.MostrarDialogo();
                break;
            case GusanoState.ESPERANDO_ACEPTAR_MISION:
                interactPrompt.SetActive(true);
                break;
            case GusanoState.EXCAVANDO_TUNEL_1:
                excavadarController.IniciarMovimiento(-1, 15f);
                Invoke("moverTunel1", 0.7f);
                break;
            case GusanoState.ESPERANDO_HUEVO:
                tapaTunel_1.SetActive(false);
                break;
            case GusanoState.ESPERANDO_ENTREGAR_HUEVO:
                tapaTunel_1.SetActive(false);
                break;
            case GusanoState.DIALOGO_HUEVO:
                dialogoController.MostrarDialogo();
                break;
            case GusanoState.EXCAVANDO_TUNEL_2:
                excavadarController.IniciarMovimiento(1, 20f);
                Invoke("moverTunel2", 1.55f);
                break;
            case GusanoState.COMPLETADO:
                tapaTunel_2.SetActive(false);
                Destroy(gameObject);
                break;
        }
    }
    void Update()
    {
        if (jugadorEnRango)
        {
            switch (currentState)
            {
                case GusanoState.ESPERANDO_JUGADOR:
                    ChangeState(GusanoState.DIALOGO_INICIAL);
                    break;
                case GusanoState.ESPERANDO_ACEPTAR_MISION:
                    interactPrompt.SetActive(true);
                    if (Input.GetKeyDown(KeyCode.E))
                    {
```






```
                        ChangeState(GusanoState.EXCAVANDO_TUNEL_1);
                        interactPrompt.SetActive(false);

dialogoController.GetDialogoObject().SetActive(false);
                    }
                    break;
                case GusanoState.ESPERANDO_HUEVO:
                    if (playerStats.HasMosquitoEgg())
                    {
                        ChangeState(GusanoState.DIALOGO_HUEVO);
                    }
                    break;
                case GusanoState.ESPERANDO_ENTREGAR_HUEVO:
                        interactPrompt.SetActive(true);
                        if (Input.GetKeyDown(KeyCode.E))
                        {
                            playerStats.SetHasMosquitoEgg(false);
                            ChangeState(GusanoState.EXCAVANDO_TUNEL_2);
                            interactPrompt.SetActive(false);

dialogoController.GetDialogoObject().SetActive(false);
                        }
                    break;
            }
        }
        else
        {
            interactPrompt.SetActive(false);
        }

    }

    void moverTunel1()
    {
        tunel_1.IniciarMovimiento(-1);
    }

    void moverTunel2()
    {
        tunel_2.IniciarMovimiento(1);
    }
}
```

Script GusanoDialogo





```
using UnityEngine;
using System.Collections;

public class GusanoDialogo : MonoBehaviour
{
    [SerializeField] private GusanoFSM fsm;
    [SerializeField] private GameObject dialogo;
    [SerializeField] private SpriteRenderer panelDialogo;
    [SerializeField] private Sprite[] spritesSaludo;
    [SerializeField] private Sprite[] spritesGracias;

    public void MostrarDialogo()
    {
        if (fsm.CurrentState == GusanoFSM.GusanoState.DIALOGO_INICIAL)
        {
            StartCoroutine(DialogoCoroutine(spritesSaludo));
        }
        else if (fsm.CurrentState == GusanoFSM.GusanoState.DIALOGO_HUEVO)
        {
            StartCoroutine(DialogoCoroutine(spritesGracias));
        }
    }

    private IEnumerator DialogoCoroutine(Sprite[] sprites)
    {
        dialogo.SetActive(true);
        foreach (var sprite in sprites)
        {
            panelDialogo.sprite = sprite;
            yield return new WaitForSeconds(4f); // Esperar 4 segundos entre
cada sprite
        }

        fsm.ChangeState(
            fsm.CurrentState == GusanoFSM.GusanoState.DIALOGO_INICIAL ?
            GusanoFSM.GusanoState.ESPERANDO_ACEPTAR_MISION :
            GusanoFSM.GusanoState.ESPERANDO_ENTREGAR_HUEVO
        );
    }

    public GameObject GetDialogoObject()
    {
        return dialogo;
    }
}
```






Script GusanoExcavador

```
using UnityEngine;
using System.Collections;

public class GusanoExcavador : MonoBehaviour
{
    [SerializeField] private GusanoFSM fsm;
    private Vector3 startPosition;
    [SerializeField] private Animator animator;

    public void IniciarMovimiento(int direccion, float distancia)
    {
        StartCoroutine(MoverCorrutine(direccion, distancia));
    }

    private IEnumerator MoverCorrutine(int direccion, float distancia)
    {
        animator.SetBool("cavando", true);
        if (fsm.CurrentState == GusanoFSM.GusanoState.EXCAVANDO_TUNEL_2)
transform.rotation = new Quaternion(0, 180, 0, 0);

        startPosition = transform.position;
        // 2. Calcular el destino final basado en la dirección
        Vector3 destinoFinal = new Vector3(
            startPosition.x + direccion * distancia, // Coordenada X fuera
de pantalla
            startPosition.y,
            startPosition.z
        );
        // 3. Mover el Gusano (y su hija, la 'tapaTunel')
        while (Vector3.Distance(transform.position, destinoFinal) > 0.01f)
        {
            transform.position = Vector3.MoveTowards(
                transform.position,
                destinoFinal,
                5f * Time.deltaTime
            );
            yield return null; // Esperar al próximo frame
        }
        if (fsm.CurrentState == GusanoFSM.GusanoState.EXCAVANDO_TUNEL_1)
        {
            transform.rotation = new Quaternion(0, 180, 0, 0);
            while (Vector3.Distance(transform.position, startPosition) >
0.01f)
            {
                transform.position = Vector3.MoveTowards(
```



```
                    transform.position,
                    startPosition,
                    5f * Time.deltaTime
                );
                yield return null; // Esperar al próximo frame
            }
            animator.SetBool("cavando", false);
            transform.rotation = new Quaternion(0, 0, 0, 0);
            fsm.ChangeState(GusanoFSM.GusanoState.ESPERANDO_HUEVO);
        }else
        {
            fsm.ChangeState(GusanoFSM.GusanoState.COMPLETADO);
        }
    }
}
```

Bomba explosiva

Fase de análisis:

Historia de usuario HU18:

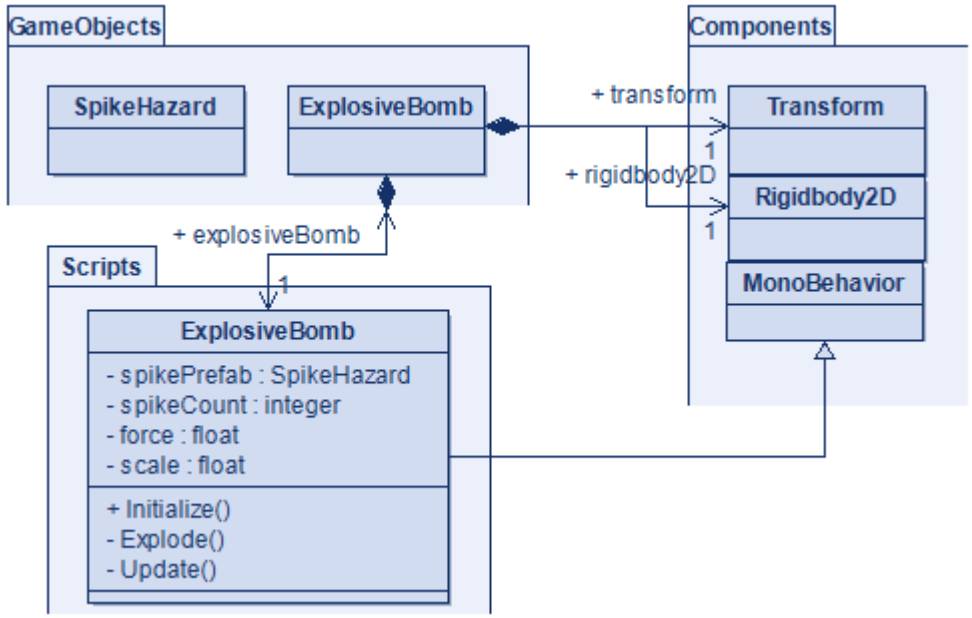
Quién: Como la Bomba Explosiva
Qué: Necesito caer con gravedad y rotar constantemente hasta alcanzar la altura central del escenario
Para qué: para que pueda explotar, instanciando los prefabs `SpikeHazard` en todas las direcciones (según los parámetros) antes de destruirme
Tabla EPS

Entrada	Proceso	Salida
El `BombSpawner` llama a `Initialize(count, force, scale)` en la bomba recién creada.	1. **(Update):** La bomba cae (por su `Rigidbody2D`) y rota (`transform.Rotate()`). 2. **(Update):** Comprueba `if (transform.position.y <= fixedExplosionY)`.	1. Múltiples `SpikeHazard` son creados. 2. El `GameObject ExplosiveBomb` se destruye.

	3. **(Explode):** Si llega al centro, llama a `Explode()`. 4. `Explode()` ejecuta un bucle `for` (0 a `spikeCount`). 5. **(Bucle):** Calcula el ángulo (`360 / spikeCount * i`) y el `Vector2` de dirección. 6. **(Bucle):** `Instantiate` el `spikePrefab`, aplica el `scale` y llama a `spikeScript.Initialize(direction, force)`.	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU18

- Crear el Prefab ExplosiveBomb.prefab.
- Añadir un Rigidbody2D (Dinámico, Gravity Scale = 1) y un Collider2D.
- Crear el script ExplosiveBomb.cs y adjuntarlo.



- En ExplosiveBomb.cs:
    - Definir [SerializeField] para el spikePrefab (el Prefab de la espina) y fixedExplosionY (la altura fija del centro).
    - Definir variables privadas para spikeCount, force y scale.
    - Implementar public void Initialize(int count, float force, float scale) para recibir los parámetros.
    - Implementar Update() para aplicar la rotación (transform.Rotate) y chequear la posición Y para llamar a Explode().
    - Implementar private void Explode() (el bucle for que calcula los ángulos e instancia las espinas).
    - En Explode(), asegurarse de pasar la direction, force y scale a cada nueva espina.
    - En Explode(), llamar a Destroy(gameObject) al final.

Criterios de Aceptación HU18

- La bomba rota sobre sí misma mientras cae por la gravedad.
- Cuando la bomba alcanza la altura fixedExplosionY, se llama a Explode().
- Explode() instancia el número correcto de espinas (spikeCount).
- Cada espina instanciada recibe la escala (scale) y los datos de inicialización correctos.
- La bomba se destruye después de instanciar las espinas.

Codificación HU18

Script ExplosiveBomb

```
using UnityEngine;

public class ExplosiveBomb : MonoBehaviour
{
    [SerializeField] private GameObject spikePrefab;
    [SerializeField] private int spikeCount;
    [SerializeField] private float force;
    [SerializeField] private float scale;

    public void Initialize(int count, float force, float scale)
    {
        spikeCount = count;
        this.force = force;
        this.scale = scale;
        transform.localScale = new Vector3(scale, scale, scale);

    }

    private void Explode()
    {
```



```
        float angleStep = 360f / spikeCount;

        // Bucle de Spawneo de Espinas
        for (int i = 0; i < spikeCount; i++)
        {
            // Calcular el ángulo actual
            float currentAngle = angleStep * i;

            // Convertir el ángulo (grados) a un vector de dirección 2D
(radianes)
            float radian = currentAngle * Mathf.Deg2Rad;
            Vector2 direction = new Vector2(Mathf.Cos(radian),
Mathf.Sin(radian));

            // Instanciar el prefab de la espina
            GameObject spikeGO = Instantiate(spikePrefab,
transform.position, Quaternion.identity);

            // Aplicar la escala a la nueva espina
            spikeGO.transform.localScale = new Vector3(scale, scale, scale);

            // Obtener el script de la espina y pasarle los datos
            pikeHazard spikeScript = spikeGO.GetComponent<SpikeHazard>();
            if (spikeScript != null)
            {
                spikeScript.Initialize(direction, force);
            }
            else
            {
                Debug.LogWarning("El Prefab 'SpikeHazard' no tiene el script
'SpikeHazard.cs'");
            }
        }
        Destroy(gameObject);
    }

    void Update()
    {
        transform.Rotate(0, 0, 300 * Time.deltaTime);

        // 3. Chequear si ha llegado a la altura de explosión
        if (transform.position.y <= -254)
        {
            Explode();
        }
    }
```



```
}
```

Espina

Fase de análisis:

Historia de usuario HU19:

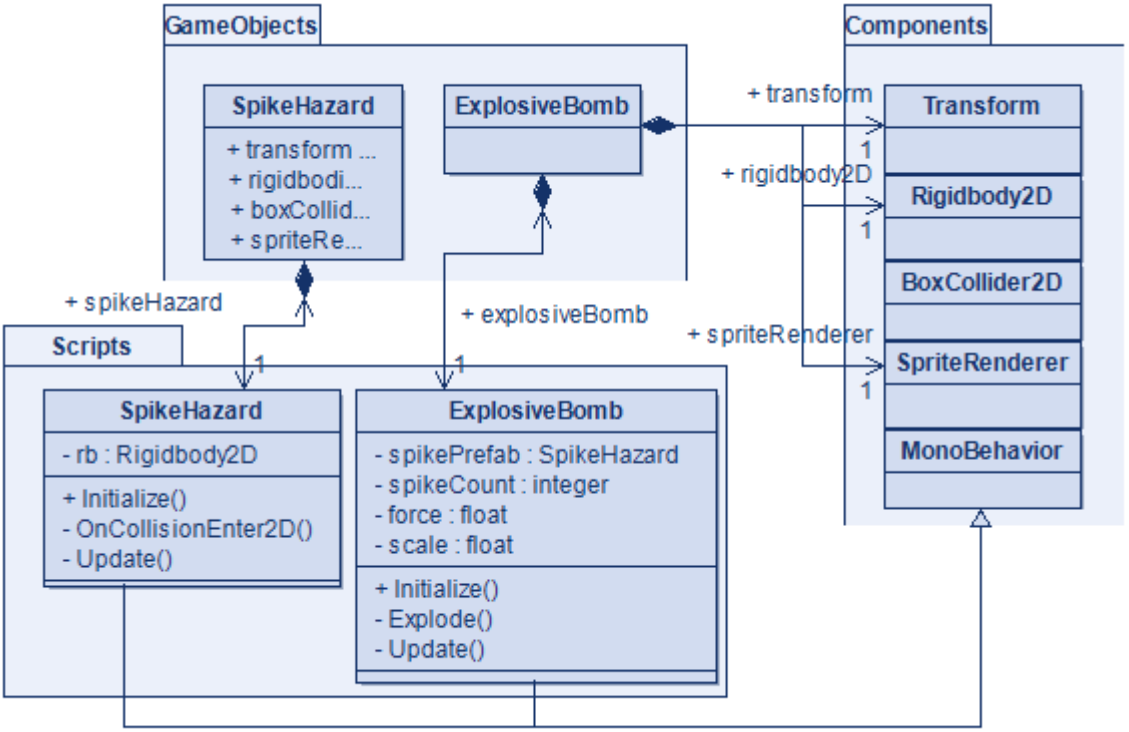
Quién: Comou una Espina (`SpikeHazard`)
Qué: Necesito ser instanciada con una dirección, fuerza y escala
Para qué: para que pueda recibir un impulso inicial (`AddForce`), volar en una trayectoria balística (con gravedad) y destruirme al chocar con cualquier superficie
Tabla EPS

Entrada	Proceso	Salida
La `ExplosiveBomb` llama a `Initialize(direction, force)` en la espina recién creada.	1. **(Start/Initialize):** El script obtiene su `Rigidbody2D` y le aplica la fuerza (`rb.AddForce(direction * force, ForceMode2D.Impulse)`). 2. **(Update):** El motor de física de Unity mueve la espina (con gravedad). 3. **(Collision):** `OnCollisionEnter2D` detecta un choque con *cualquier* collider.	El `GameObject SpikeHazard` se destruye (`Destroy(gameObject)`).

Fase de diseño:

Diagrama de clases:





Lista de Tareas HU19

- Crear el Prefab SpikeHazard.prefab.
- Añadir un Rigidbody2D (Dinámico, Gravity Scale = 1) y un Collider2D.
- Adjuntar el Prefab TargetDamage como hijo de la espina (para dañar al Player).
- Crear el script SpikeHazard.cs y adjuntarlo.
- En SpikeHazard.cs:
    - Implementar public void Initialize(Vector2 direction, float force) (que aplica la fuerza inicial al Rigidbody2D).
    - Implementar OnCollisionEnter2D(Collision2D collision).
- En OnCollisionEnter2D, llamar a Destroy(gameObject) (para que se destruya al tocar *cualquier* pared o suelo).

Criterios de Aceptación HU19

- La espina es afectada por la gravedad (sigue una curva parabólica si se dispara hacia arriba).
- La espina viaja en la dirección correcta según el ángulo de explosión.
- El TargetDamage hijo puede dañar al Player si lo toca.
- La espina se destruye automáticamente en cuanto choca con un Collider (pared, suelo, etc.).
- La escala del Prefab de la espina (y su hijo TargetDamage) se ajusta correctamente según el parámetro scale de la bomba.



Codificacion HU19

Script SpikeHazard

```
using UnityEngine;

public class SpikeHazard : MonoBehaviour
{
    [SerializeField] private Rigidbody2D rb;

    public void Initialize(Vector2 direction, float force)
    {
        // 1. APLICAR LA FUERZA DE LANZAMIENTO
        // Usamos ForceMode2D.Impulse para aplicar una fuerza instantánea
(la explosión)
        rb.AddForce(direction * force, ForceMode2D.Impulse);
    }

    // Update is called once per frame
    void Update()
    {
        // Obtener el ángulo de la velocidad actual (en radianes)
        float angleRad = Mathf.Atan2(rb.linearVelocity.y,
rb.linearVelocity.x);

        // Convertir a grados
        float angleDeg = angleRad * Mathf.Rad2Deg;

        // Aplicar la rotación al transform.
        // Esto asume que tu sprite de espina "apunta hacia la derecha" (eje
X positivo)
        transform.rotation = Quaternion.Euler(0, 0, angleDeg);
    }

    private void OnCollisionEnter2D(Collision2D collision)
    {
        // Destruir la espina al colisionar con cualquier cosa
        Destroy(gameObject);
    }
}
```



Boss Controller & Command Pattern

Fase de análisis:

Historia de usuario HU20:
Boss Controller & Command Pattern

Quién: Como el Sistema de Combate del Jefe
Qué: Necesito encapsular ataques de Raíces y Bombas en objetos "Comando" y organizarlos en "Bloques" reutilizables
Para qué: para que pueda ejecutar una **Fase 1 secuencial** y una **Fase 2 cíclica infinita**, permitiendo crear patrones complejos y escalables sin reescribir lógica
Tabla EPS

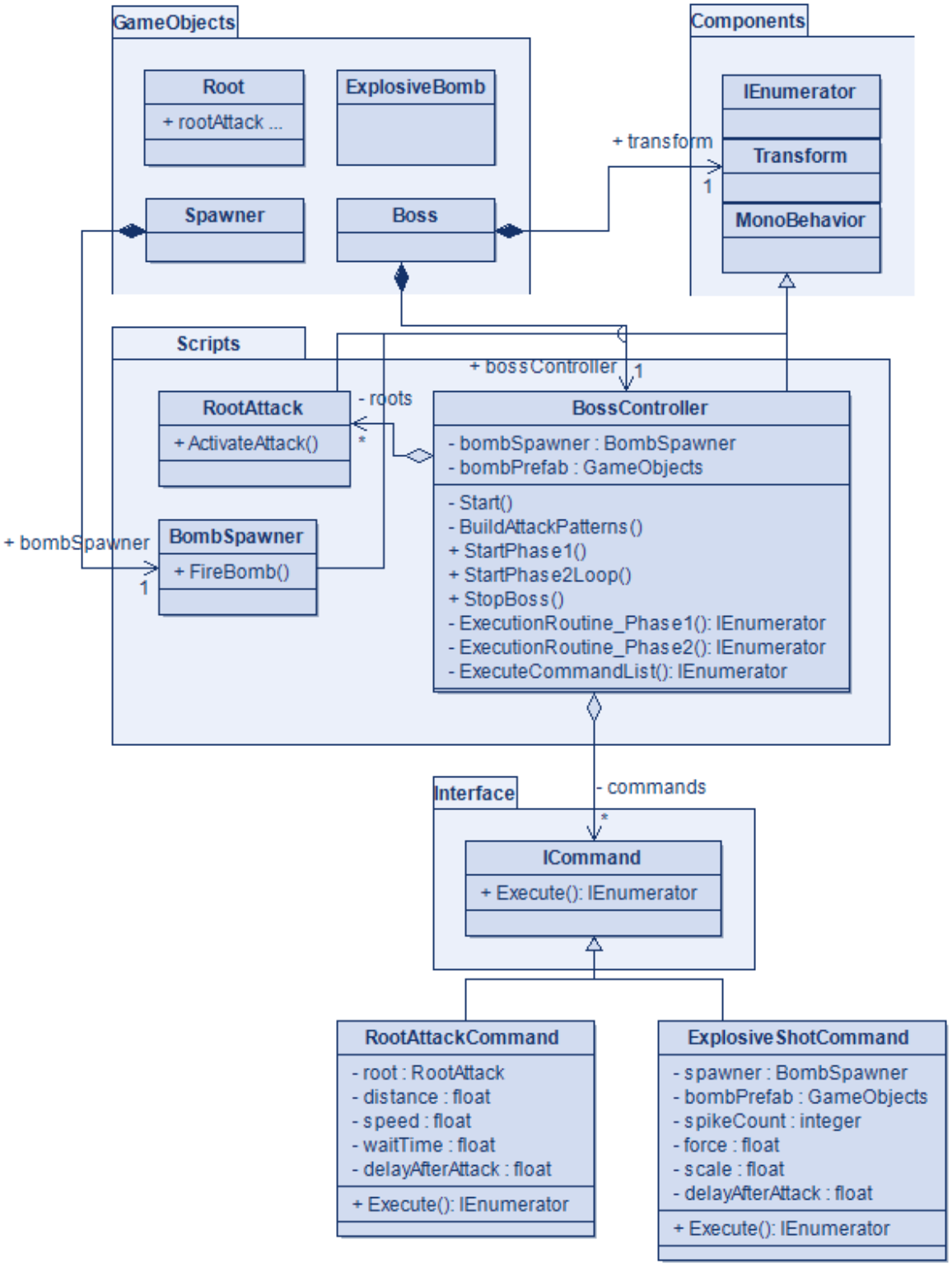
Entrada	Proceso	Salida
1. Referencias a `List<RootAttack>` (Raíces en escena). 2. Referencia a `BombSpawner` y `BombPrefab`. 3. Parámetros de diseño (velocidad, daño, escala, cantidad de espinas).	1. **(Construcción):** Al inicio (`Start`), el `BossController` crea instancias de `RootAttackCommand` y `ExplosiveShotCommand`, inyectando las dependencias (quién ataca) y los parámetros (cómo ataca). 2. **(Organización):** Agrupa estos comandos en listas pequeñas ("Bloques"). 3. **(Fase 1):** Al recibir `StartPhase1()`, una corrutina ejecuta los Bloques en un orden lineal predefinido. Al terminar, dispara el evento `OnPhase1Finished`. 4. **(Fase 2):** Al recibir `StartPhase2()`, una corrutina recorre una colección de Bloques (`Loop1`, `Loop2`...) infinitamente, añadiendo una espera entre cada bloque.	Llamadas a `.Execute()` desde el Controlador. 1. Las Raíces se mueven físicamente. 2. Las Bombas se instancian, caen y explotan. 3. El flujo de combate se mantiene activo o se detiene (`StopBoss`).



	5. **(Comandos):** Cada comando delega la acción a su receptor específico (`RootAttack.ActivateAttack` o `BombSpawner.FireBomb`).	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU20

1. Subsistema de Comandos (El "Papel")



- Crear la interfaz ICommand (con método IEnumerator Execute()).
- Crear la clase RootAttackCommand (implementa ICommand):
    - Constructor que recibe: RootAttack, distancia, velocidad, espera, demora posterior.
    - Execute llama a root.ActivateAttack() y espera la demora.
- Crear la clase ExplosiveShotCommand (implementa ICommand):
    - Constructor que recibe: BombSpawner, GameObject (prefab), cantidad espinas, fuerza, escala, demora posterior.
    - Execute llama a spawner.FireBomb() y espera la demora.

2. Subsistema de Receptores (Los "Soldados")

- Crear el script BombSpawner.cs y adjuntarlo a un GameObject vacío en el techo.
- Implementar el método FireBomb(...) que instancia el prefab y llama a Initialize() en el script de la bomba.
- *(Asumimos que RootAttack.cs y ExplosiveBomb.cs ya están listos y funcionales).*

3. Subsistema de Control (El "General")

- Crear el script BossController.cs y adjuntarlo al padre del Jefe.
- Declarar listas para los **Bloques** (block_Intro, block_Bombing, etc.) y la **Colección de Bucles** (phase2_Collection).
- Implementar BuildAllBlocks(): Llenar las listas instanciando new Command(...) con diferentes combinaciones.
- Implementar StartPhase1(): Corrutina que llama a ExecuteBlock(block) en secuencia (A -> B -> A).
- Implementar StartPhase2(): Corrutina con while(true) que itera por índice sobre phase2_Collection.
- Implementar método auxiliar ExecuteBlock(List<ICommand>): Un foreach que llama a .Execute() en cada comando.
- Implementar evento public event Action OnPhase1Finished.

4. Ensamblaje en Editor

- Arrastrar todas las Raíces de la escena a la lista del BossController.
- Arrastrar el BombSpawner y el BombPrefab al BossController.

Criterios de Aceptación HU20

1. **Fase 1 (Secuencia):** Al invocar StartPhase1(), el jefe ejecuta los ataques definidos en orden y se detiene automáticamente al final, lanzando el evento de finalización.
2. **Fase 2 (Bucle):** Al invocar StartPhase2(), el jefe ejecuta los bloques de ataque cíclicamente (1, 2, 3, 4, 1...) sin detenerse hasta que se llame a StopBoss().
3. **Bombas:** El comando de bomba instancia correctamente el prefab, y la bomba resultante tiene el tamaño (scale) y la cantidad de espinas definidos en el comando.
4. **Raíces:** El comando de raíz activa la animación/movimiento de la raíz correcta con la velocidad definida.



5. **Modularidad:** Se puede reutilizar un bloque (ej. block_Intro) múltiples veces dentro de StartPhase1() sin errores ni duplicación de código.
6. **Pausa:** Llamar a StopBoss() detiene inmediatamente cualquier ataque en curso y previene que el siguiente comience.

Codificacion HU20

Script BossController

```
using UnityEngine;
using System;
using System.Collections;
using System.Collections.Generic;

public class BossController : MonoBehaviour
{
    // --- 1. REFERENCIAS (Los Soldados) ---
    [Header("Asignaciones de Escena")]
    [Tooltip("Arrastra aquí todas las raíces en el orden que prefieras (0,
1, 2...)")]
    [SerializeField] private List<RootAttack> roots;

    [Tooltip("Arrastra el Spawner que está en el techo")]
    [SerializeField] private BombSpawner bombSpawner;

    [Header("Prefabs")]
    [Tooltip("El prefab de la bomba explosiva")]
    [SerializeField] private GameObject bombPrefab;

    // --- 2. ALMACÉN DE PATRONES (Las Partituras) ---

    // FASE 1: Una única lista larga (o bloques concatenados) que se ejecuta
UNA vez.
    private List<List<ICommand>> phase1_FullSequence;

    // FASE 2: Una colección de listas pequeñas que se ejecutan en BUCLE
infinito.
    // (Lista 1 -> Lista 2 -> Lista 3 -> Volver a Lista 1...)
    private List<List<ICommand>> phase2_LoopCollection;

    // --- 3. REFERENCIA A LA FSM PARA CAMBIAR EL ESTADO ---
    [SerializeField] private FinalLevelFSM fsm;

    [SerializeField] private GameObject muroIsquierda;

    private void Start()
```



```
    {
        BuildAttackPatterns();
    }

    private void BuildAttackPatterns()
    {
        // -------------------------------------------------------
        // CONFIGURACIÓN FASE 1 (Secuencia Lineal)
        // -------------------------------------------------------
        phase1_FullSequence = new List<List<ICommand>>();

        List<ICommand> phase1_PartA = new List<ICommand>();

        phase1_PartA.Add(new RootAttackCommand(roots[4], -25f, 15f, 6f,
3f));
        phase1_PartA.Add(new RootAttackCommand(roots[9], -17f, 15f, 7f,
2f));
        phase1_PartA.Add(new RootAttackCommand(roots[12], -10.15f, 15f, 7f,
0f));
        phase1_PartA.Add(new RootAttackCommand(roots[13], -7.8f, 15f, 7f,
0f));
        phase1_PartA.Add(new RootAttackCommand(roots[8], -17.5f, 15f, 7f,
2f));
        phase1_PartA.Add(new RootAttackCommand(roots[7], -18f, 15f, 8.5f,
2f));
        phase1_PartA.Add(new RootAttackCommand(roots[6], -22f, 15f, 5f,
2f));
        phase1_PartA.Add(new RootAttackCommand(roots[5], -24f, 15f, 2f,
5f));
        phase1_PartA.Add(new RootAttackCommand(roots[0], -16f, 15f, 2f,
0.5f));
        phase1_PartA.Add(new RootAttackCommand(roots[1], -16f, 15f, 2f,
0.5f));
        phase1_PartA.Add(new RootAttackCommand(roots[2], -16f, 15f, 2f,
0.5f));
        phase1_PartA.Add(new RootAttackCommand(roots[3], -16f, 15f, 2f,
5f));

        List<ICommand> phase1_PartB = new List<ICommand>();

        phase1_PartB.Add(new ExplosiveShotCommand(bombSpawner, 6, 10f, 0.3f,
2f, bombPrefab));
        phase1_PartB.Add(new ExplosiveShotCommand(bombSpawner, 9, 10f, 0.4f,
2f, bombPrefab));
        phase1_PartB.Add(new ExplosiveShotCommand(bombSpawner, 11, 8f, 0.7f,
4f, bombPrefab));
```



```
        List<ICommand> phase1_PartC = new List<ICommand>();

        phase1_PartC.Add(new RootAttackCommand(roots[10], -27f, 20f, 23f,
2f));
        phase1_PartC.Add(new RootAttackCommand(roots[9], -14f, 20f, 0f,
1f));
        phase1_PartC.Add(new RootAttackCommand(roots[6], -19f, 20f, 0f,
1f));
        phase1_PartC.Add(new RootAttackCommand(roots[8], -14.5f, 20f, 0f,
1f));
        phase1_PartC.Add(new RootAttackCommand(roots[4], -22f, 20f, 0f,
2f));
        phase1_PartC.Add(new RootAttackCommand(roots[13], -27.3f, 20f, 16f,
3f));
        phase1_PartC.Add(new RootAttackCommand(roots[9], -12.4f, 20f, 0f,
0f));
        phase1_PartC.Add(new RootAttackCommand(roots[4], -20.4f, 20f, 0f,
1f));
        phase1_PartC.Add(new RootAttackCommand(roots[6], -17.4f, 20f, 0f,
0f));
        phase1_PartC.Add(new RootAttackCommand(roots[7], -13.4f, 20f, 0f,
2f));
        phase1_PartC.Add(new ExplosiveShotCommand(bombSpawner, 12, 10f,
0.5f, 2f, bombPrefab));
        phase1_PartC.Add(new RootAttackCommand(roots[15], -27.8f, 25f, 0f,
2f));
        phase1_PartC.Add(new ExplosiveShotCommand(bombSpawner, 8, 15f, 0.3f,
1f, bombPrefab));
        phase1_PartC.Add(new ExplosiveShotCommand(bombSpawner, 12, 12f,
0.4f, 1f, bombPrefab));
        phase1_PartC.Add(new ExplosiveShotCommand(bombSpawner, 7, 17f, 0.3f,
2.5f, bombPrefab));
        phase1_PartC.Add(new ExplosiveShotCommand(bombSpawner, 12, 10f,
0.5f, 7f, bombPrefab));

        List<ICommand> phase1_PartD = new List<ICommand>();

        phase1_PartD.Add(new RootAttackCommand(roots[13], -27.3f, 25f, 2.5f,
1f));
        phase1_PartD.Add(new RootAttackCommand(roots[12], -26.7f, 15f, 0f,
4f));
        phase1_PartD.Add(new RootAttackCommand(roots[10], -27f, 20f, 30f,
1f));
        phase1_PartD.Add(new RootAttackCommand(roots[13], -27.3f, 20f, 15f,
1f));
```



```
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 10, 10f,
0.6f, 3f, bombPrefab));
        phase1_PartD.Add(new RootAttackCommand(roots[15], -27.8f, 20f, 10f,
0f));
        phase1_PartD.Add(new RootAttackCommand(roots[14], -26.7f, 20f, 10f,
3f));

        phase1_PartD.Add(new RootAttackCommand(roots[17], -11f, 15f, 0f,
0f));
        phase1_PartD.Add(new RootAttackCommand(roots[18], -11f, 15f, 0f,
3f));

        phase1_PartD.Add(new RootAttackCommand(roots[18], -6f, 15f, 1f,
0.5f));
        phase1_PartD.Add(new RootAttackCommand(roots[17], -16f, 20f, 0f,
3f));

        phase1_PartD.Add(new RootAttackCommand(roots[17], -6f, 15f, 1f,
0.5f));
        phase1_PartD.Add(new RootAttackCommand(roots[18], -16f, 20f, 0f,
3f));

        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 3, 12f, 0.4f,
1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 4, 12f, 0.4f,
1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 5, 12f, 0.4f,
1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 6, 12f, 0.4f,
1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 7, 12f, 0.4f,
1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 8, 12f, 0.4f,
1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 9, 12f, 0.4f,
1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 10, 12f,
0.4f, 1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 11, 12f,
0.4f, 1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 12, 12f,
0.4f, 1f, bombPrefab));
        phase1_PartD.Add(new ExplosiveShotCommand(bombSpawner, 13, 12f,
0.4f, 1f, bombPrefab));

        phase1_FullSequence.Add(phase1_PartA);
```



```
        phase1_FullSequence.Add(phase1_PartB);
        phase1_FullSequence.Add(phase1_PartC);
        phase1_FullSequence.Add(phase1_PartD);
        // ------------------------------------------------------------
        // CONFIGURACIÓN FASE 2 (Bucle Infinito de Listas)
        // ------------------------------------------------------------
        phase2_LoopCollection = new List<List<ICommand>>();

        List<ICommand> loopA = new List<ICommand>();

        loopA.Add(new RootAttackCommand(roots[10], -27f, 30f, 8f, 1f));
        loopA.Add(new RootAttackCommand(roots[13], -27.3f, 30f, 7f, 1f));
        loopA.Add(new RootAttackCommand(roots[17], -17f, 30f, 6.55f, 0f));
        loopA.Add(new RootAttackCommand(roots[18], -17f, 30f, 6.55f, 1f));
        loopA.Add(new RootAttackCommand(roots[15], -27.8f, 30f, 5f, 1f));
        loopA.Add(new RootAttackCommand(roots[14], -26.7f, 30f, 4f, 1f));

        loopA.Add(new RootAttackCommand(roots[9], -7, 30f, 0f, 0.2f));
        loopA.Add(new RootAttackCommand(roots[8], -7, 30f, 0f, 0.2f));
        loopA.Add(new RootAttackCommand(roots[7], -8, 30f, 0f, 0.2f));
        loopA.Add(new RootAttackCommand(roots[6], -12, 30f, 0f, 0.2f));
        loopA.Add(new RootAttackCommand(roots[5], -14, 30f, 0f, 0.2f));
        loopA.Add(new RootAttackCommand(roots[4], -15, 30f, 0f, 0f));
        loopA.Add(new RootAttackCommand(roots[11], -30, 40f, 0f, 1f));

        loopA.Add(new RootAttackCommand(roots[0], -9, 25f, 0f, 0.2f));
        loopA.Add(new RootAttackCommand(roots[1], -9, 25f, 0f, 0.2f));
        loopA.Add(new RootAttackCommand(roots[2], -9, 25f, 0f, 0.2f));
        loopA.Add(new RootAttackCommand(roots[3], -9, 25f, 0f, 2f));

        List<ICommand> loopB = new List<ICommand>();

        loopB.Add(new RootAttackCommand(roots[10], -27f, 30f, 22f, 1f));
        loopB.Add(new RootAttackCommand(roots[13], -30f, 30f, 21f, 0.5f));
        loopB.Add(new RootAttackCommand(roots[9], -12f, 30f, 1f, 0.2f));
        loopB.Add(new RootAttackCommand(roots[18], -8.32f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[8], -12f, 30f, 1f, 0.2f));
        loopB.Add(new RootAttackCommand(roots[7], -12.5f, 30f, 1f, 0.2f));
        loopB.Add(new RootAttackCommand(roots[6], -16.7f, 30f, 1f, 0.2f));
        loopB.Add(new RootAttackCommand(roots[5], -19.6f, 30f, 1f, 1.5f));
        loopB.Add(new RootAttackCommand(roots[16], -35f, 50f, 17.8f, 1.2f));
        loopB.Add(new RootAttackCommand(roots[18], -14f, 50f, 0f, 0f));

        loopB.Add(new RootAttackCommand(roots[0], -13f, 50f, 0f, 1f));
        loopB.Add(new RootAttackCommand(roots[3], -13f, 50f, 0f, 1f));
        loopB.Add(new RootAttackCommand(roots[2], -13f, 50f, 0f, 1f));
```



```
        loopB.Add(new RootAttackCommand(roots[0], -13f, 50f, 0f, 1f));

        loopB.Add(new RootAttackCommand(roots[1], -13f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[3], -13f, 50f, 0f, 1f));

        loopB.Add(new RootAttackCommand(roots[0], -13f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[2], -13f, 50f, 0f, 2f));

        loopB.Add(new RootAttackCommand(roots[0], -13f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[1], -13f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[2], -13f, 50f, 0f, 2f));

        loopB.Add(new RootAttackCommand(roots[1], -13f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[2], -13f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[3], -13f, 50f, 0f, 2f));

        loopB.Add(new RootAttackCommand(roots[0], -13f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[1], -13f, 50f, 0f, 0f));
        loopB.Add(new RootAttackCommand(roots[3], -13f, 50f, 0f, 2f));

        loopB.Add(new RootAttackCommand(roots[3], -13f, 50f, 0f, 0.6f));
        loopB.Add(new RootAttackCommand(roots[2], -13f, 50f, 0f, 0.6f));
        loopB.Add(new RootAttackCommand(roots[0], -13f, 50f, 0f, 0.6f));
        loopB.Add(new RootAttackCommand(roots[3], -13f, 50f, 0f, 0.6f));
        loopB.Add(new RootAttackCommand(roots[1], -13f, 50f, 0f, 0.6f));
        loopB.Add(new RootAttackCommand(roots[0], -13f, 50f, 0f, 0.6f));
        loopB.Add(new RootAttackCommand(roots[3], -13f, 50f, 0f, 2f));

        List<ICommand> loopC = new List<ICommand>();

        loopC.Add(new RootAttackCommand(roots[10], -29f, 30f, 17f, 2f));

        loopC.Add(new RootAttackCommand(roots[0], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[1], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[2], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[3], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[4], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[5], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[6], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[7], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[8], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[9], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[12], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[13], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[14], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[15], 6f, 10f, 16f, 0f));
```






```
        loopC.Add(new RootAttackCommand(roots[17], 6f, 10f, 16f, 0f));
        loopC.Add(new RootAttackCommand(roots[18], 6f, 10f, 16f, 1f));

        loopC.Add(new ExplosiveShotCommand(bombSpawner, 15, 10f, 0.6f, 3f,
bombPrefab));
        loopC.Add(new ExplosiveShotCommand(bombSpawner, 4, 12f, 0.4f, 0.3f,
bombPrefab));
        loopC.Add(new ExplosiveShotCommand(bombSpawner, 7, 9f, 0.35f, 0.3f,
bombPrefab));
        loopC.Add(new ExplosiveShotCommand(bombSpawner, 3, 10f, 0.4f, 0.3f,
bombPrefab));
        loopC.Add(new ExplosiveShotCommand(bombSpawner, 9, 11f, 0.45f, 0.3f,
bombPrefab));
        loopC.Add(new ExplosiveShotCommand(bombSpawner, 5, 8f, 0.4f, 0.3f,
bombPrefab));
        loopC.Add(new ExplosiveShotCommand(bombSpawner, 4, 13f, 0.35f, 3f,
bombPrefab));
        loopC.Add(new ExplosiveShotCommand(bombSpawner, 15, 10f, 0.8f, 4f,
bombPrefab));
        loopC.Add(new ExplosiveShotCommand(bombSpawner, 17, 10f, 0.2f, 4f,
bombPrefab));

        List<ICommand> loopD = new List<ICommand>();

        loopD.Add(new RootAttackCommand(roots[10], -28f, 30f, 13f, 2f));

        loopD.Add(new RootAttackCommand(roots[13], -11.5f, 30f, 0.1f, 0f));
        loopD.Add(new RootAttackCommand(roots[18], -12.5f, 30f, 0.1f, 0f));
        loopD.Add(new RootAttackCommand(roots[17], -12f, 30f, 0.1f, 0f));
        loopD.Add(new RootAttackCommand(roots[15], -11.5f, 30f, 0.1f, 0f));
        loopD.Add(new RootAttackCommand(roots[14], -10f, 30f, 0.1f, 1f));

        loopD.Add(new ExplosiveShotCommand(bombSpawner, 15, 12f, 0.6f, 1.5f,
bombPrefab));

        loopD.Add(new RootAttackCommand(roots[9], -14.5f, 40f, 0f, 0.2f));
        loopD.Add(new RootAttackCommand(roots[8], -14.5f, 40f, 0f, 0.2f));
        loopD.Add(new RootAttackCommand(roots[7], -15f, 40f, 0f, 0.2f));
        loopD.Add(new RootAttackCommand(roots[6], -19.2f, 40f, 0f, 0.2f));
        loopD.Add(new RootAttackCommand(roots[5], -22.2f, 40f, 0f, 0.2f));
        loopD.Add(new RootAttackCommand(roots[4], -23f, 40f, 0f, 0.8f));

        loopD.Add(new RootAttackCommand(roots[0], -14.5f, 40f, 0f, 0.2f));
        loopD.Add(new RootAttackCommand(roots[1], -14.5f, 40f, 0f, 0.2f));
        loopD.Add(new RootAttackCommand(roots[2], -14.5f, 40f, 0f, 0.2f));
        loopD.Add(new RootAttackCommand(roots[3], -14.5f, 40f, 0f, 2f));
```



```
        loopD.Add(new ExplosiveShotCommand(bombSpawner, 10, 15f, 0.4f, 1f,
bombPrefab));
        loopD.Add(new ExplosiveShotCommand(bombSpawner, 8, 10f, 0.8f, 1f,
bombPrefab));
        loopD.Add(new ExplosiveShotCommand(bombSpawner, 15, 12f, 0.5f, 4f,
bombPrefab));

        phase2_LoopCollection.Add(loopA);
        phase2_LoopCollection.Add(loopC);
        phase2_LoopCollection.Add(loopB);
        phase2_LoopCollection.Add(loopD);


        // (Puedes añadir tantos Loops como quieras a la colección)
    }
    //
============================================================================
    //              MÉTODOS DE CONTROL (Llamados por la FSM)
    //
============================================================================

    public void StartPhase1()
    {
        StopBoss(); // Seguridad: detener cualquier ataque previo
        StartCoroutine(ExecutionRoutine_Phase1());
    }

    public void StartPhase2Loop()
    {
        StopBoss();
        StartCoroutine(ExecutionRoutine_Phase2());
        muroIsquierda.SetActive(true);
    }

    [ContextMenu("Stop Boss")]
    public void StopBoss()
    {
        StopAllCoroutines();
        // Resetear todas las raíces a su posición inicial
        foreach (RootAttack root in roots)
        {
            root.ResetPosition();
        }
    }
```






```
    //
================================================================================
    //              MOTORES DE EJECUCIÓN (Lógica Interna)
    //
================================================================================

    // --- Motor FASE 1 (Finita) ---
    private IEnumerator ExecutionRoutine_Phase1()
    {
        int currentIndex = 0;

        // Bucle infinito (hasta que se llame a StopBoss)
        while (phase1_FullSequence.Count > currentIndex)
        {
            // 1. Obtener la lista de ataques actual (Ciclo: A -> B -> C ->
A...)
            // Usamos el operador módulo (%) para volver a 0 cuando llegamos
al final.
            if (phase1_FullSequence.Count > 0)
            {
                List<ICommand> currentList =
phase1_FullSequence[currentIndex];

                // 2. Ejecutar esa lista completa
                yield return
StartCoroutine(ExecuteCommandList(currentList));

                // 3. Pasar al siguiente índice
                currentIndex++;
            }
            else
            {
                yield return null; // Evitar cuelgue si está vacía
            }

        }
        yield return new WaitForSeconds(2.0f);

        // Notificar a la FSM que ya terminamos
        fsm.ChangeState(FinalLevelFSM.BossState.APERTURA_CORAZON);
    }

    // --- Motor FASE 2 (Infinita) ---
    private IEnumerator ExecutionRoutine_Phase2()
    {
```



```
        int currentIndex = 0;

        // Bucle infinito (hasta que se llame a StopBoss)
        while (true)
        {
            // 1. Obtener la lista de ataques actual
            // Usamos el operador módulo (%) para volver a 0 cuando llegamos
al final.
            if (phase2_LoopCollection.Count > 0)
            {
                List<ICommand> currentList =
phase2_LoopCollection[currentIndex];

                // 2. Ejecutar esa lista completa
                yield return
StartCoroutine(ExecuteCommandList(currentList));

                // 3. Elegir aleatoriamente la siguiente lista a ejecutar
                // Asegurarnos de que no sea el mismo índice que el actual
                if (phase2_LoopCollection.Count > 1)
                {
                    int newIndex;
                    do
                    {
                        newIndex = UnityEngine.Random.Range(0,
phase2_LoopCollection.Count);
                    }
                    while (newIndex == currentIndex);

                    currentIndex = newIndex;
                }
                // Si solo hay 1 elemento, se mantiene el índice actual (no
se puede cambiar)
            }
            // 4. Pausa entre bucles
            yield return new WaitForSeconds(2.0f);
        }
    }

    // --- Ayudante Genérico ---
    // Recorre y ejecuta cualquier lista de comandos ICommand
    private IEnumerator ExecuteCommandList(List<ICommand> pattern)
    {
        foreach (ICommand command in pattern)
        {
            // Ejecuta el comando y ESPERA a que termine (gracias a
```






```
IEnumerator)
            yield return StartCoroutine(command.Execute());
        }
    }

}
```

Nivel Final FSM

Fase de análisis:

Historia de usuario HU21:

Quién: Como el Director del Nivel Final (`FinalLevelFSM`)
Qué: Necesito controlar el flujo de cámaras, diálogos y fases de combate mediante una secuencia de estados
Para qué: para que la batalla narre una historia coherente que reaccione a la cantidad de Savia del jugador, derivando en un "Final de Purificación" o un "Final de Sacrificio"

Tabla EPS

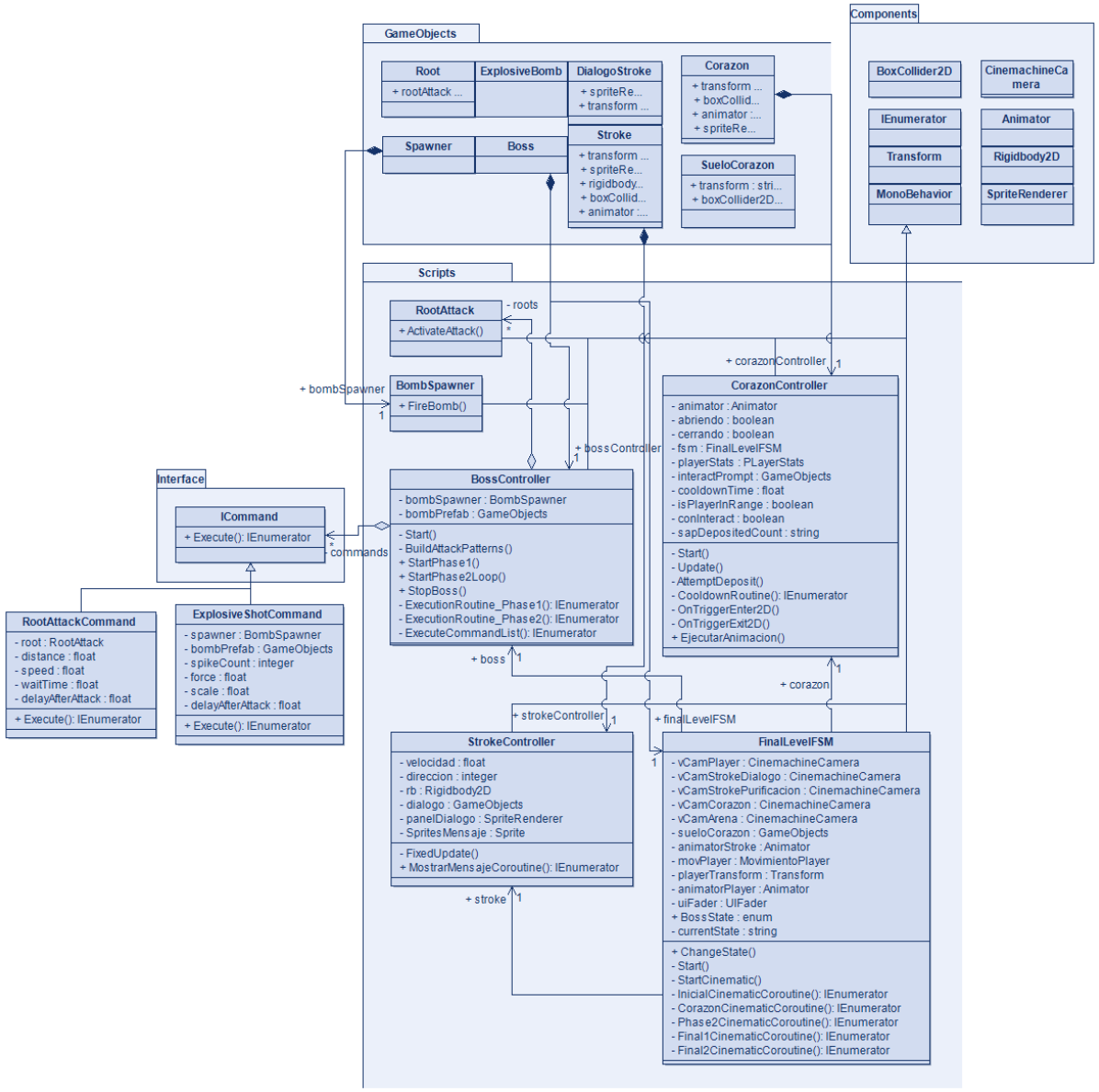
Entrada	Proceso	Salida
1. `Start()` (Carga de escena). 2. `Invoke` (Tiempo de espera inicial). 3. `BossController.OnPhase1Finished`. 4. `HeartTrigger` (Interacción 'E'). 5. `PlayerStats.CurrentSap` (Cantidad restante).	1. **(Inicio):** `FinalLevelFSM` espera 1s e inicia `INTRO`. Mueve a Stroke, muestra diálogo y cambia cámara. 2. **(Combate 1):** Inicia `FASE_1`. El `BossController` ejecuta su secuencia finita. Al terminar, la FSM pasa a `APERTURA_CORAZON`. 3. **(Interacción):** El `CorazonController` se abre. El jugador presiona 'E'. - **Si tiene Savia:** Resta 1, suma al contador local. Si es la 1ra	1. Cambios de Cámara (`SetCamera`). 2. UI de Diálogo (Mostrar/Ocultar). 3. Control del Boss (Start/Stop/Animación Muerte). 4. Control del Player (Bloqueo Input / Mover al corazón). 5. Carga de Escena (`SceneManager`).



	vez, activa `FASE_2` (Bucle). Si llega a 10, activa `FINAL_1`. - **Si no tiene Savia (y depositó >= 6):** Activa `FINAL_2`. 4. **(Finales):** La FSM ejecuta las corrutinas de final (cámaras, animaciones de purificación/muerte, fade out).	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU21

1. Script: `FinalLevelFSM.cs` (El Cerebro)

- **Estructura de Datos:**
  - Declarar el `enum BossState` con los estados: `ESPERA_INICIAL`, `INTRO`, `FASE_1`, `APERTURA_CORAZON`, `ESPERANDO_INTERACCION`, `FASE_2`, `FINAL_1`, `FINAL_2`.
  - Declarar referencias `[SerializeField]` para: Cámaras (Player, Stroke, Arena, Corazón), Controladores (`Boss`, `Corazon`, `Stroke`), UI (`Fader`) y Player (`Movimiento`, `Anim`).
- **Inicialización:**



    - En `Start()`, invocar `StartCinematic` con 1s de retraso.
    - En `StartCinematic()`, activar el objeto raíz (`rootSuperior`) y llamar a `ChangeState(APERTURA_CORAZON)` (o `INTRO`, según tu flujo preferido).
- **Método `ChangeState(newState)`:**
    - Implementar un `switch(currentState)` que dispare las corrutinas correspondientes:
        - `INTRO` -> `StartCoroutine(InicialCinematicCoroutine())`
        - `FASE_1` -> `boss.StartPhase1()`
        - `APERTURA_CORAZON` -> `StartCoroutine(CorazonCinematicCoroutine())`
        - `FASE_2` -> `StartCoroutine(Phase2CinematicCoroutine())`
        - `FINAL_1` -> `StartCoroutine(Final1CinematicCoroutine())`
        - `FINAL_2` -> `StartCoroutine(Final2CinematicCoroutine())`
- **Corrutinas de Cinemáticas (Lógica Visual):**
    - **`InicialCinematicCoroutine`:** Bloquear input (`GameManager`), subir prioridad de `vCamStrokeDialogo`, mover a Stroke (`SetDireccion`), mostrar diálogo (`stroke.MostrarMensaje`), cambiar a `vCamArena`, desbloquear input, pasar a `FASE_1`.
    - **`CorazonCinematicCoroutine`:** Bloquear input, subir prioridad `vCamCorazon`, activar animación corazón (`Abriendo`), esperar, subir prioridad `vCamPlayer`, desbloquear input, pasar a `ESPERANDO_INTERACCION`.
    - **`Final1CinematicCoroutine` (Purificación):** Detener Boss, bloquear input, enfocar `vCamCorazon`, cerrar corazón, enfocar `vCamStrokePurificacion`, animar Stroke (`Purificar`), Fade Out, cargar escena `Final_1`.
    - **`Final2CinematicCoroutine` (Sacrificio):** Similar al Final 1, pero antes de cerrar el corazón: mover el `playerTransform` manualmente hacia el centro del corazón, animar Player (`Sacrificio`), y luego cargar escena `Final_2`.

2. Script: `CorazonController.cs` (El Trigger)

- **Control de Estado:**
    - En `Update()`, verificar tres condiciones antes de mostrar la 'E': `isPlayerInRange` (Trigger), `canInteract` (Cooldown) y `fsm.GetCurrentState()` (debe ser `ESPERANDO` o `FASE_2`).
- **Lógica de Depósito (`AttemptDeposit`):**
    - Al presionar 'E', iniciar `CooldownRoutine` (desactiva interacción por 5s).
    - Leer `playerStats.GetCurrentSap()`.



    - **Condición Victoria:** Si `sapDepositedCount >= 10` -> Llamar `fsm.ChangeState(FINAL_1)`.
    - **Condición Sacrificio:** Si `currentSap <= 0` Y `sapDepositedCount >= 6` -> Llamar `fsm.ChangeState(FINAL_2)`.
    - **Condición Normal:** Si tiene savia -> `playerStats.ReleaseSap(1)`, aumentar contador.
    - **Transición a Fase 2:** Si el estado era `ESPERANDO_INTERACCION`, llamar `fsm.ChangeState(FASE_2)` inmediatamente.
- **Animaciones:**
    - Implementar `EjecutarAnimacion()` (invocada repetidamente) para triggerear "Latir" si no se está abriendo/cerrando.
    - Métodos `Abriendo(bool)` y `Cerrando(bool)` para controlar los parámetros del Animator.

3. Script: `StrokeController.cs` (El Actor Enemigo)

- **Movimiento Físico:**
    - En `FixedUpdate()`, aplicar velocidad al `Rigidbody2D` en el eje X basándose en la variable `direccion`.
    - Implementar `SetDireccion(float)` para que la FSM pueda moverlo (-1 izquierda, 1 derecha, 0 quieto).
- **Sistema de Diálogo:**
    - Implementar `MostrarMensajeCoroutine()`: Activar el objeto `dialogo`, iterar por el array `spriteMensaje` cambiando el sprite del `SpriteRenderer` cada 4 segundos, y desactivar al final.

4. Configuración en Editor (Scene Assembly)

- **Cámaras:** Crear y posicionar las 5 cámaras virtuales (`Player`, `StrokeDialogo`, `StrokePurif`, `Corazon`, `Arena`).
- **Asignación FSM:** Arrastrar todas las cámaras, controladores (Stroke, Corazon, Boss, Player) y UI al script `FinalLevelFSM`.
- **Asignación Corazón:** Arrastrar la FSM y `PlayerStats` al `CorazonController`. Configurar `sapTargetToWin = 10`.
- **Asignación Stroke:** Llenar el array de Sprites de diálogo en `StrokeController`.

Criterios de Aceptación HU21

1. El juego espera 1 segundo antes de iniciar la intro automáticamente.
2. Stroke se mueve, habla y la cámara vuelve a la arena antes de que el Boss ataque.
3. El Corazón **solo** permite interactuar después de la cinemática de apertura.
4. Al depositar la **primera** savia, la FSM cambia a `FASE_2` y el Boss comienza el bucle de ataques.



5. El jugador no puede spammear la 'E' (debe esperar 5 segundos entre depósitos).
6. Si el jugador llega a 10 savias depositadas, se activa la cinemática de purificación y carga la escena `Final_1`.
7. Si el jugador tiene 0 savia (y depositó al menos 6), su personaje camina solo hacia el corazón, muere, y carga la escena `Final_2`.

Codificacion HU21

Script FinalLevelFSM

```
using UnityEngine;
using System.Collections;
using Unity.Cinemachine;
using UnityEngine.SceneManagement;
public class FinalLevelFSM : MonoBehaviour
{
    [SerializeField] private GameObject rootSuperior;
    [Header("Camaras")]
    [SerializeField] private CinemachineCamera vCamPlayer;
    [SerializeField] private CinemachineCamera vCamStrokeDialogo;
    [SerializeField] private CinemachineCamera vCamStrokePurificacion;

    [SerializeField] private CinemachineCamera vCamCorazon;
    [SerializeField] private CinemachineCamera vCamArena;
    [Header("Corazon")]
    [SerializeField] private CorazonController corazon;
    [SerializeField] private GameObject sueloCorazon;
    [Header("Strong")]
    [SerializeField] private StrokeController stroke;
    [SerializeField] private Animator animatorStrong;
    [Header("Boss Controller")]
    [SerializeField] private BossController boss;
    [Header("Player")]
    [SerializeField] private MovimientoPlayer movPlayer;
    [SerializeField] private Transform playerTransform;
    [SerializeField] private Animator animatorPlayer;
    [Header("Fade Final")]
    [SerializeField] private UIFader uiFader;
    public enum BossState
    {
        ESPERA_INICIAL,
        INTRO,
        FASE_1,
        APERTURA_CORAZON,
        ESPERANDO_INTERACCION,
        FASE_2,
```



```
        FINAL_1,
        FINAL_2
    }
    private BossState currentState = BossState.ESPERA_INICIAL;


    public void ChangeState(BossState newState)
    {
        currentState = newState;
        // Lógica para manejar la transición entre estados
        switch (currentState)
        {
            case BossState.INTRO:
                StartCoroutine(InicialCinematicCoroutine());
                break;
            case BossState.FASE_1:
                boss.StartPhase1();
                break;
            case BossState.APERTURA_CORAZON:
                StartCoroutine(CorazonCinematicCoroutine());
                break;
            case BossState.ESPERANDO_INTERACCION:
                // Esperar interacción del jugador
                break;
            case BossState.FASE_2:
                StartCoroutine(Phase2CinematicCoroutine());
                break;
            case BossState.FINAL_1:
                StartCoroutine(Final1CinematicCoroutine());
                // Final bueno
                break;
            case BossState.FINAL_2:
                StartCoroutine(Final2CinematicCoroutine());
                // Final malo
                break;
        }
        Debug.Log("Current State: " + currentState);
    }

    void Start()
    {
        // 1. Espera inicial
        Invoke("StartCinematic", 1f);
    }
    void StartCinematic()
    {
```






```
        ChangeState(BossState.APERTURA_CORAZON);
        rootSuperior.SetActive(true);
    }

    private IEnumerator InicialCinematicCoroutine()
    {
        // Lógica para la cinemática inicial
        // Bloquear input del jugador
        GameManager.RequestInputLock(true);
        yield return new WaitForSeconds(1f);
        // Cambiar a cámara de Stroke
        vCamStrokeDialogo.Priority = 20;
        yield return new WaitForSeconds(2f);
        // Mueve a Stroke hacia adelante
        stroke.SetDireccion(-1);
        yield return new WaitForSeconds(2.5f);
        stroke.SetDireccion(0);
        // Mostrar mensaje de Stroke
        yield return StartCoroutine(stroke.MostrarMensajeCoroutine());
        // Mueve a Stroke hacia atras
        stroke.SetDireccion(1);
        yield return new WaitForSeconds(2.5f);
        // Cambia a la camara de la arena complenta
        vCamStrokeDialogo.Priority = 1;
        vCamArena.Priority = 20;
        yield return new WaitForSeconds(2f);
        // Volver el control al jugador
        GameManager.RequestInputLock(false);

        ChangeState(BossState.FASE_1);
    }

    private IEnumerator CorazonCinematicCoroutine()
    {
        // Lógica para la cinemática del corazón
        // Bloquear input del jugador
        GameManager.RequestInputLock(true);
        yield return new WaitForSeconds(1f);
        // Cambiar a cámara del corazón
        vCamArena.Priority = 1;
        vCamCorazon.Priority = 20;
        yield return new WaitForSeconds(2f);
        // Animación de apertura del corazón
        corazon.Abriendo(true);
        sueloCorazon.SetActive(false);
        yield return new WaitForSeconds(3f);
```






```
        vCamCorazon.Priority = 1;
        vCamPlayer.Priority = 20;
        yield return new WaitForSeconds(1f);
        // Volver el control al jugador
        GameManager.RequestInputLock(false);
        ChangeState(BossState.ESPERANDO_INTERACCION);
    }

    private IEnumerator Phase2CinematicCoroutine()
    {
        // Lógica para la cinemática de transición a Fase 2
        yield return new WaitForSeconds(1f);
        vCamPlayer.Priority=1;
        vCamArena.Priority=20;
        yield return new WaitForSeconds(2f);
        boss.StartPhase2Loop();
    }

    private IEnumerator Final1CinematicCoroutine()
    {
        // Lógica para la cinemática del final bueno
        // Bloquear input del jugador
        boss.StopBoss();
        GameManager.RequestInputLock(true);
        yield return new WaitForSeconds(1f);
        // Cambiar a cámara del corazón
        vCamArena.Priority=1;
        vCamCorazon.Priority=20;
        yield return new WaitForSeconds(2f);
        // Se cierra el corazón
        corazon.Cerrando(true);
        yield return new WaitForSeconds(2.5f);
        //Cambia a cámara de purificación
        vCamCorazon.Priority=1;
        vCamStrokePurificacion.Priority=20;
        yield return new WaitForSeconds(3f);
        // Animación de purificación
        animatorStrong.SetTrigger("Purificar");
        yield return new WaitForSeconds(5f);
        // Fade out y carga de escena
        uiFader.FadeToBlack();
        yield return new WaitForSeconds(1f);
        SceneManager.LoadScene("Final_1_Purificado");
    }

    private IEnumerator Final2CinematicCoroutine()
```






```
    {
        // Lógica para la cinemática del final malo
        // Bloquear input del jugador
        boss.StopBoss();
        GameManager.RequestInputLock(true);
        yield return new WaitForSeconds(1f);
        // Cambiar a cámara del corazón
        vCamArena.Priority=1;
        vCamCorazon.Priority=20;
        yield return new WaitForSeconds(2f);
        // Mover al jugador hacia el corazon
        while (playerTransform.position.x < corazon.transform.position.x -
0.1f)
        {
            movPlayer.Velocidad = 2f;
            movPlayer.Direccion = 1;
            yield return null;
        }
        movPlayer.Direccion = 0;
        movPlayer.Velocidad = 10f;
        yield return new WaitForSeconds(1f);
        // Animación de muerte del jugador
        animatorPlayer.SetTrigger("sacrificio");
        yield return new WaitForSeconds(4f);
        // Se cierra el corazón
        corazon.Cerrando(true);
        yield return new WaitForSeconds(2.5f);
        //Cambia a cámara de purificación
        vCamCorazon.Priority=1;
        vCamStrokePurificacion.Priority=20;
        yield return new WaitForSeconds(3f);
        // Animación de purificación
        animatorStrong.SetTrigger("Purificar");
        yield return new WaitForSeconds(5f);
        // Fade out y carga de escena
        uiFader.FadeToBlack();
        yield return new WaitForSeconds(1f);
        SceneManager.LoadScene("Final_2_Sacrificio");
    }
    public BossState GetCurrentState()
    {
        return currentState;
    }

}
```






Script CorazonConroller

```
using UnityEngine;
using System.Collections;

public class CorazonController : MonoBehaviour
{
    [SerializeField] private Animator animator;
    private bool abriendo = false;
    private bool cerrando = false;

    [Header("Referencias")]
    [SerializeField] private FinalLevelFSM fsm;
    [SerializeField] private PlayerStats playerStats;
    [SerializeField] private GameObject interactPrompt; // El aviso visual
"E"

    [Header("Configuración")]
    [SerializeField] private float cooldownTime = 5.0f;
    // --- Estado Interno ---
    private bool isPlayerInRange = false;
    private bool canInteract = true; // Controla el cooldown
    private int sapDepositedCount = 0;

    void Start()
    {
        InvokeRepeating("EjecutarAnimacion", 1f, 2f);
    }

    private void Update()
    {
        // 1. Validar condiciones básicas
        if (!isPlayerInRange || !canInteract)
        {
            if (interactPrompt != null && interactPrompt.activeSelf)
interactPrompt.SetActive(false);
            return;
        }
        // 2. Validar ESTADO de la FSM
        // Solo permitimos interactuar en "ESPERANDO_INTERACCION" o "FASE_2"
        FinalLevelFSM.BossState currentState = fsm.GetCurrentState(); //
(Necesitamos este getter en la FSM)

        bool isStateValid = (currentState ==
FinalLevelFSM.BossState.ESPERANDO_INTERACCION ||
                             currentState ==
```






```
FinalLevelFSM.BossState.FASE_2);

        if (!isStateValid)
        {
            if (interactPrompt != null && interactPrompt.activeSelf)
interactPrompt.SetActive(false);
            return;
        }

        // 3. Mostrar el aviso si todo es válido
        if (!interactPrompt.activeSelf) interactPrompt.SetActive(true);

        // 4. Detectar Input
        if (Input.GetKeyDown(KeyCode.E))
        {
            AttemptDeposit();
        }
    }

    // --- Lógica de Depósito ---
    private void AttemptDeposit()
    {
        // Iniciar cooldown inmediatamente para evitar spam
        StartCoroutine(CooldownRoutine());

        // Leer cuánta savia tiene el jugador en este momento
        int currentSap = playerStats.GetCurrentSap();

        // Chequear condición de VICTORIA INMEDIATA (el jugador depositó la
cantidad necesaria)
        if (sapDepositedCount >= 10)
        {
            fsm.ChangeState(FinalLevelFSM.BossState.FINAL_1); // Final 1
            return;
        }

        // El jugador NO TIENE Savia (Sacrificio)
        if (currentSap <= 0 && sapDepositedCount >= 6)
        {
            // Si intenta interactuar sin savia y deeposito al menos 6, se
sacrifica
            fsm.ChangeState(FinalLevelFSM.BossState.FINAL_2); // Final 2
            return;
        }

        // El jugador TIENE Savia -> Consumir y sumar
```



```
        playerStats.ReleaseSap(1);
        sapDepositedCount++;



        // Si no se alcanzó la victoria, si el estado actual era
ESPERANDO_INTERACCION,
        // entonces la primera depositación activa la Fase 2.
        if (fsm.GetCurrentState() ==
FinalLevelFSM.BossState.ESPERANDO_INTERACCION)
        {
            fsm.ChangeState(FinalLevelFSM.BossState.FASE_2);
        }
    }

    // --- Cooldown ---
    private IEnumerator CooldownRoutine()
    {
        canInteract = false;
        interactPrompt.SetActive(false); // Ocultar aviso durante cooldown

        // (Opcional: Aquí podrías poner una animación de UI tipo "reloj
recargando")

        yield return new WaitForSeconds(cooldownTime);

        canInteract = true;
    }

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            isPlayerInRange = true;
        }
    }
    private void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            isPlayerInRange = false;
        }
    }

    private void EjecutarAnimacion()
    {
```






```
        if (abriendo)
        {
            abriendo = false;
            animator.SetBool("Abierto", true);
            return;
        }
        else if (cerrando)
        {
            cerrando = false;
            animator.SetBool("Abierto", false);
            return;
        }
            animator.SetTrigger("Latir");
    }

    public void Abriendo(bool value)
    {
        abriendo = value;
    }
    public void Cerrando(bool value)
    {
        cerrando = value;
    }
}
```

Script StrokeController

```
using UnityEngine;
using System.Collections;
public class StrokeController : MonoBehaviour
{
    private float velocidad = 2f;
    [Range(-1f, 1f)]
    [SerializeField] private float direccion;
    [SerializeField] private Rigidbody2D rb;
    [Header("Mensaje")]
    [SerializeField] private GameObject dialogo;
    [SerializeField] private SpriteRenderer panelDialogo;
    [SerializeField] private Sprite[] spritesMensaje;

    void FixedUpdate()
    {
        rb.linearVelocity = new Vector2(
                direccion * velocidad,
                rb.linearVelocity.y);
```






```
    }

    public void SetDireccion(float dir)
    {
        direccion = dir;
    }

    public IEnumerator MostrarMensajeCoroutine()
    {
        dialogo.SetActive(true);
        foreach (var sprite in spritesMensaje)
        {
            panelDialogo.sprite = sprite;
            yield return new WaitForSeconds(4f); // Esperar 4 segundos entre
cada sprite
        }
        dialogo.SetActive(false);
    }
}
```

Menú de Pausa

Fase de análisis:

Historia de usuario HU22:

Quién: Como Jugador
Qué: Necesito que al pausar el juego se abra un menú navegable con las mismas mecánicas que el menú principal
Para qué: para que pueda acceder a "Reanudar", "Ajustes" o "Salir" usando una interfaz familiar y consistente, sin que el juego se ejecute de fondo

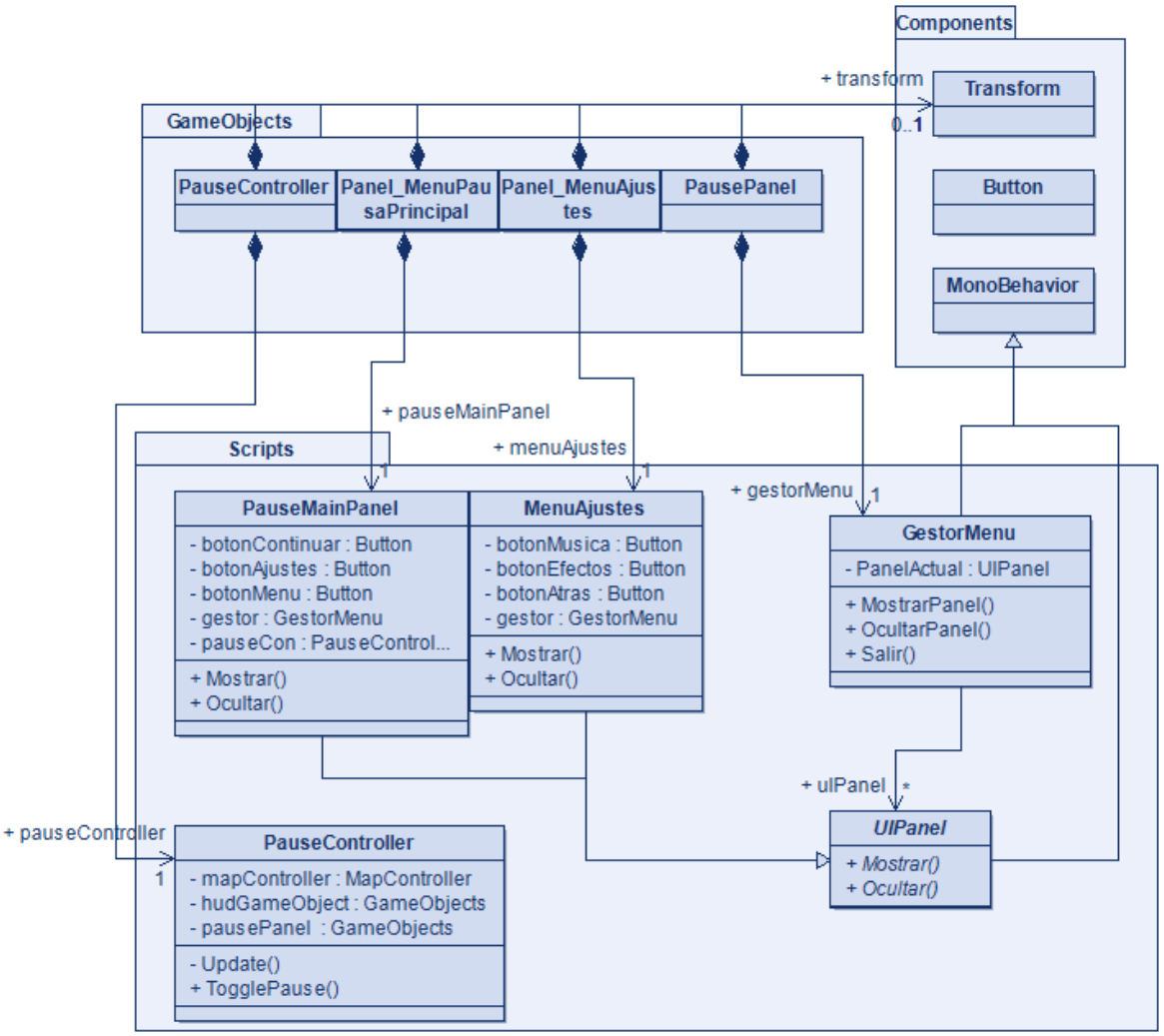
Tabla EPS

Entrada	Proceso	Salida
Tecla `Escape` (Down). Botón "Reanudar" (OnClick).	**(`PauseController`):** Detecta `Escape` en `Update()`. **(Prioridad Mapa):** Verifica `if (mapController != null`	Juego congelado/descongelado . `PausePanel` visible/invisible. HUD visible/invisible.

	`&& mapController.IsMapOpen)`. **(Caso A):** Si es `true` -> Llama a `mapController.ToggleMap( )` y hace `return`. **(Caso B):** Si es `false` -> Llama a su propio método `TogglePause()`. **(`PauseMainPanel`):** Recibe el clic. **(Delegación):** Llama a `pauseController.TogglePa use()`. Lógica interna de `PauseController:` Invierte `isPaused`. **(Visual):** `pausePanel.SetActive(isP aused)` y `hudGameObject.SetActive( !isPaused)`. **(Global):** `Time.timeScale` (0 o 1) y `GameManager.RequestInput Lock(isPaused)`.	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU22

A. Configuración de Escena (Jerarquía)

- Crear el `Canvas_Pause`.
- Crear el hijo `PauseController` (Vacío).
- Crear el hijo `PausePanel` (Panel visual) dentro del Controller.
- Dentro de `PausePanel`, configurar el sistema de menús polimórfico (`GestorMenu` y los sub-paneles).

B. Script `PauseController.cs`

- **Referencias:** `[SerializeField]` para `mapController`, `hudGameObject`, y `pausePanel` (el hijo visual).
- **Variables:** `private bool isPaused`.
- **Update:**



    - Detectar `KeyCode.Escape`.
    - Lógica de prioridad: Si hay mapa abierto -> Cerrarlo y Return. Si no -> `TogglePause()`.
- **Método `TogglePause()`:**
    - `isPaused = !isPaused`.
    - `pausePanel.SetActive(isPaused)`.
    - `hudGameObject.SetActive(!isPaused)`.
    - `Time.timeScale = isPaused ? 0f : 1f`.
    - `GameManager.RequestInputLock(isPaused)`.

C. Script `PauseMainPanel.cs` (Nuevo Panel)

- Heredar de `UIPanel` (tu interfaz/clase abstracta).
- Añadir referencia: `[SerializeField] private PauseController pauseController`.
- Implementar método `OnClickResume()`: Llama a `pauseController.TogglePause()`.
- Implementar método `OnClickSettings()`: Llama a `gestor.MostrarPanel(...)`.
- Implementar método `OnClickExit()`: Carga el Menú Principal.

D. Conexiones (Inspector)

- **En `PauseController`:** Arrastrar el `MapController` (si es Nivel 2), el objeto del HUD y el objeto hijo `PausePanel`.
- **En `PauseMainPanel`:** Arrastrar el padre `PauseController` a la referencia.

Criterios de Aceptación HU22

1. **Jerarquía:** El script `PauseController` nunca se desactiva, solo se desactiva su hijo `PausePanel`.
2. **Prioridad Escape:** Si el mapa está abierto, Escape lo cierra sin pausar el juego.
3. **Pausa:** Si el mapa está cerrado, Escape oculta el HUD, muestra el `PausePanel` y congela el tiempo.
4. **Botón Reanudar:** Al hacer clic, el menú se cierra, el HUD vuelve y el juego continúa.
5. **Niveles sin Mapa:** El sistema funciona en el Nivel 1 (sin `MapController` asignado) sin lanzar errores.

Codificacion HU22

Script PauseController



```
using UnityEngine;

public class PauseController : MonoBehaviour
{
    [SerializeField] private MapController mapController;
    [SerializeField] private GameObject HUD;
    [SerializeField] private GameObject pauseMenu;
    private bool isPaused = false;
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            if (mapController != null && mapController.IsMapOpen())
            {
                // Si el mapa está abierto, cerrarlo en lugar de pausar el
juego
                mapController.ToggleMap();
            }
            else
            {
                // Alternar el estado de pausa normal
                TogglePause();
            }
        }
    }

    public void TogglePause()
    {
        isPaused = !isPaused;
        GameManager.RequestPauseState(isPaused);
        GameManager.RequestInputLock(isPaused);
        if (isPaused)
        {
            // Mostrar menú de pausa y ocultar HUD
            if (pauseMenu != null)
            {
                pauseMenu.SetActive(true);
            }
            if (HUD != null)
            {
                HUD.SetActive(false);
            }
        }
        else
        {
            // Ocultar menú de pausa y mostrar HUD
```






```
            if (pauseMenu != null)
            {
                pauseMenu.SetActive(false);
            }
            if (HUD != null)
            {
                HUD.SetActive(true);
            }
        }
    }
    public bool IsPaused()
    {
        return isPaused;
    }
}
```

Script PauseMainPanel

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;

public class PauseMainPanel : UIPanel
{
    [SerializeField] private PauseController pauseCon;
    [SerializeField] private Button botonContinuar;
    [SerializeField] private Button botonAjustes;
    [SerializeField] private Button botonMenu;
    [SerializeField] private GestorMenu gestor;

    public override void Mostrar()
    {
        gameObject.SetActive(true);
        if (gestor == null)
        {
            Debug.LogError("GestorMenu no asignado en PauseMainPanel");
            return;
        }
        botonContinuar.onClick.RemoveAllListeners();
        botonContinuar.onClick.AddListener(() => pauseCon.TogglePause());

        botonAjustes.onClick.RemoveAllListeners();
        botonAjustes.onClick.AddListener(() => gestor.MostrarPanel(1));

        botonMenu.onClick.RemoveAllListeners();
```






```
        botonMenu.onClick.AddListener(() =>
SceneManager.LoadScene("MainMenu"));
    }

    public override void Ocultar()
    {
        gameObject.SetActive(false);
    }
}
```

PostGameScene

Fase de análisis:

Historia de usuario HU23:

Quién: Como Jugador que acaba de terminar el juego
Qué: Necesito ver la escena final (Win/Loss) y poder presionar *cualquier tecla* para que el juego ejecute una transición suave de vuelta al menú principal
Para qué: para que el proceso de cierre de la sesión sea rápido y fluido

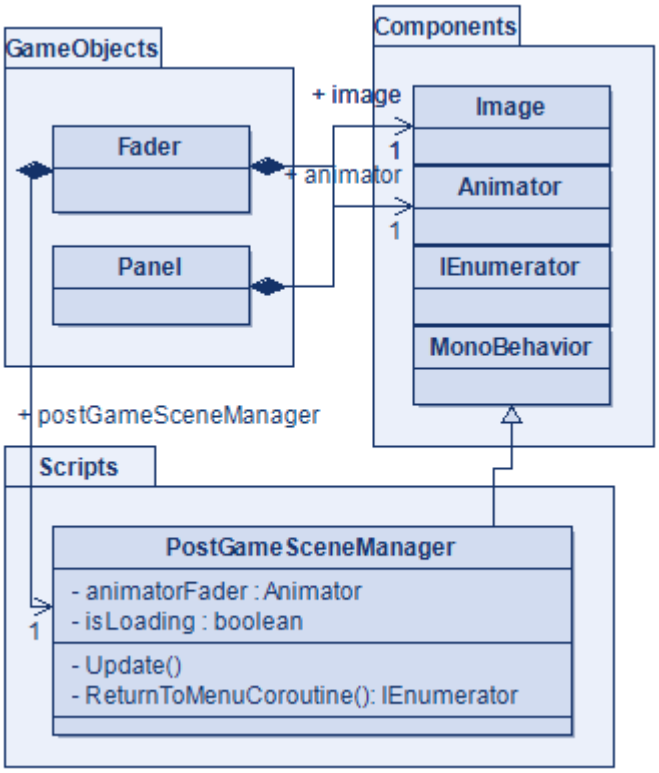
Tabla EPS

Entrada	Proceso	Salida
Carga de escena (`Scene_Final_1`, `Scene_Final_2`, `Scene_Game_Over`). `Input.anyKeyDown` (Cualquier tecla o clic del ratón).	1. **(Start):** El script `PostGameSceneManager.cs` se inicia. 2. Llama a `UIFader.FadeFromBlack(0.5s)` para revelar la imagen estática. 3. **(Update):** El script escucha constantemente `Input.anyKeyDown`. 4. Si se detecta input, inicia la corrutina `ReturnToMenuCoroutine()`.	Se vuelve a la escena "MenuPrincipal".

	5. La corrutina espera un breve momento (visual). 6. Llama a `UIFader.FadeToBlack(1.0s)`. 7. El script espera la notificación de que el *fade out* ha terminado. 8. Llama a `SceneManager.LoadScene("MenuPrincipal")`	

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU23

- **Crear Escenas:** Crear tres escenas separadas: `GameOverScene`, `Final_1_Purificacion`, `Final_2_Sacrificio`.



- **Script:** Crear el script `PostGameSceneManager.cs`.
- **Referencias:** Definir `[SerializeField]` para el `UIFader.cs` y el `GameObject` que contiene la imagen/panel de resultado.
- **`Update()`:**
    - Implementar la detección de input: `if (Input.anyKeyDown && !isLoading)`.
    - La detección debe ser envuelta en un *flag* (`isLoading`) para asegurar que solo se active una vez.
- **Corrutina:** Implementar `IEnumerator ReturnToMenuCoroutine()`:
    - Incluir una pequeña espera visual (ej. `yield return new WaitForSeconds(1.5f);`).
    - Llamar a `UIFader.FadeToBlack(1.0f)`.
    - **Crucial:** Implementar una forma de esperar la finalización del Fade Out (mediante una Corrutina en el Fader, un evento, o un simple `yield return new WaitForSeconds(1.0f);` si es que el fader tarda 1 segundo).
    - Llamar a `SceneManager.LoadScene("MenuPrincipal")`.

Criterios de Aceptación HU23

- El juego comienza con la imagen de victoria/derrota visible (la pantalla no está en negro al cargar).
- Al presionar **cualquier tecla** o clic del ratón, el sistema inicia la secuencia de retorno.
- La secuencia de retorno incluye una pausa visual (1.5s) y un *fade to black* suave (1.0s) antes de cargar la escena.
- El juego no se congela ni se traba si el jugador presiona la tecla de forma repetida.
- El script funciona sin cambios en las tres escenas finales

Codificación HU23

```
using UnityEngine;
using UnityEngine.SceneManagement;
using System.Collections;

public class PostGameSceneManager : MonoBehaviour
{
    [SerializeField] private Animator animatorFader;
    private bool isLoading = false;

    // Update is called once per frame
    void Update()
    {
        if (isLoading) return;
        if (Input.anyKeyDown)
```



```
        {
            StartCoroutine(ReturnToMenuCoroutine());
        }
    }
    private IEnumerator ReturnToMenuCoroutine()
    {
        if (isLoading) yield break;
        isLoading = true;

        if (animatorFader != null)
        {
            animatorFader.SetTrigger("FadeIn");
            yield return new WaitForSeconds(1f); // Esperar a que termine la
animación de fade out
        }

        SceneManager.LoadScene("MainMenu");
    }
}
```

Audio Manager

Fase de análisis:

Historia de usuario HU24:

Quién: Como Jugador
Qué: Necesito escuchar efectos de sonido y música de fondo continua que cambie según el nivel, además de poder silenciarlos independientemente en el menú de ajustes
Para qué: para que a experiencia sea inmersiva y personalizable, cumpliendo con los requisitos de feedback auditivo del juego

Tabla EPS

Entrada	Proceso	Salida
1. **AudioMixer** con grupos "Music" y "SFX".	1. **(Inicio):** `AudioManager` se instancia como Singleton. Si ya existe uno, se autodestruye. Si	Sonido saliendo por los parlantes. Persistencia del objeto

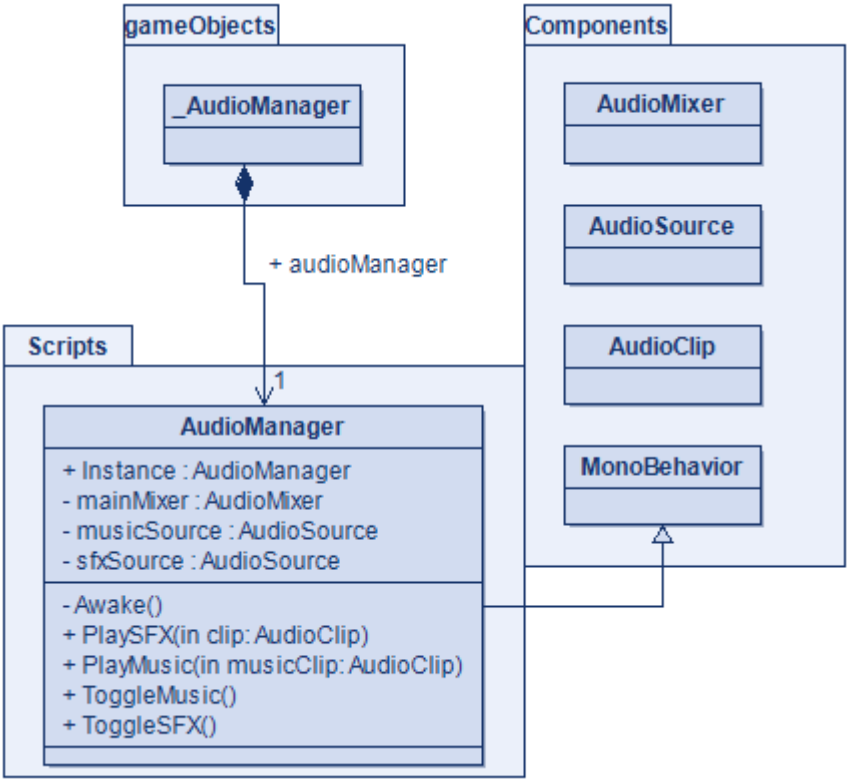




2. `AudioClips` asignados en scripts clientes (Player, Items). Interacción con **Toggles** (On/Off).	es el primero, ejecuta `DontDestroyOnLoad`. 2. **(Ejecución):** Un script (ej. `SaltoPlayer`) llama a `AudioManager.Instance.PlaySFX(clip)`. 3. **(Cambio de Nivel):** `LevelStartLogic` llama a `PlayMusic(newClip)`. El Manager verifica si es la misma canción; si no, la cambia. 4. **(Ajustes):** Al cambiar el Toggle, se llama a `ToggleMusic(bool)`. El Manager convierte el bool en decibelios (0dB o -80dB) y lo aplica al Mixer.	`_AudioManager` entre escenas.

Fase de diseño:

Diagrama de clases:



Lista de Tareas HU24

1. Configuración de Assets (Editor)

- **Audio Mixer:**
  - Crear `MainMixer`.
  - Crear grupos hijos `Music` y `SFX`.
  - Exponer los parámetros de volumen y renombrarlos a `MusicVol` y `SFXVol`.
- **Prefab AudioManager:**
  - Crear un GameObject `_AudioManager` en la escena **MenuPrincipal**.
  - Añadir dos componentes `AudioSource` hijos (o en el mismo objeto).
  - Configurar `AudioSource 1`: Output -> `Music` (Mixer Group), Loop -> `True`.
  - Configurar `AudioSource 2`: Output -> `SFX` (Mixer Group), Loop -> `False`.

2. Script `AudioManager.cs` (El Singleton)

- Implementar el patrón Singleton con `DontDestroyOnLoad`.
- Referencias: `AudioMixer`, `musicSource`, `sfxSource`.
- Método `PlaySFX(AudioClip clip)`: Ejecuta `sfxSource.PlayOneShot(clip)`.
- Método `PlayMusic(AudioClip clip)`: Verifica si el clip es diferente al actual. Si lo es, lo asigna y da `Play()`.



- Método `ToggleMusic(bool isOn)`: Setea `MusicVol` en Mixer a 0 o -80.
- Método `ToggleSFX(bool isOn)`: Setea `SFXVol` en Mixer a 0 o -80.

3. Integración en Scripts (Clientes)

- **En el Player (`SaltoPlayer`, `MovimientoPlayer`):** Añadir `[SerializeField] AudioClip jumpSound;` y llamar a `AudioManager.Instance.PlaySFX` al saltar.
- **En Coleccionables (`SaviaCollectible`):** Añadir clip y reproducir al recoger.
- **En Enemigos (`BombSpawner`):** Añadir sonido de explosión.
- **En Niveles (`LevelStartLogic`):** En `Start()`, llamar a `AudioManager.Instance.PlayMusic(bgmClip)` con la música de ese nivel.

4. Integración en UI (`MenuAjustes`)

- Crear Toggles para Música y Efectos.
- En el evento `OnValueChanged` de cada Toggle, arrastrar el `AudioManager` y seleccionar el método `ToggleMusic` o `ToggleSFX`.

Criterios de Aceptación HU24

1. **Persistencia:** La música no se corta ni se reinicia al cambiar de escena (si es la misma canción).
2. **Unicidad:** Nunca hay dos `AudioManager` en la escena al mismo tiempo.
3. **Control:** Los Toggles del menú de ajustes silencian efectivamente la música o los efectos.
4. **Feedback:** El salto, la recolección de savia y las explosiones tienen sonido.
5. **Música por Nivel:** Cada nivel (y el menú) reproduce su música de fondo correspondiente automáticamente al cargar.

Codificación HU24

Script AudioManager

```
using UnityEngine;
using UnityEngine.Audio;

public class AudioManager : MonoBehaviour
{
    // --- Singleton ---
    public static AudioManager Instance;

    [Header("Configuración")]
```



```
    [SerializeField] private AudioMixer mainMixer;

    [Header("Fuentes")]
    [SerializeField] private AudioSource musicSource; // Debe tener 'Loop' activado
    [SerializeField] private AudioSource sfxSource;   // Debe tener 'Loop' desactivado

    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }

    // ============================================================
    //              MÉTODOS DE REPRODUCCIÓN
    // ============================================================

    // Llamar esto cuando el jugador salta, golpea, etc.
    public void PlaySFX(AudioClip clip)
    {
        if (clip != null)
        {
            sfxSource.PlayOneShot(clip);
        }
    }

    // Llamar esto al iniciar cada nivel (en el Start de un script de nivel)
    public void PlayMusic(AudioClip musicClip)
    {
        // Lógica inteligente: Si la canción que pides YA está sonando, no la reinicies.
        // Esto permite que la música continúe suavemente si dos niveles comparten tema.
        if (musicSource.clip == musicClip)
            return;

        musicSource.Stop(); // Detener la anterior
        musicSource.clip = musicClip;
```






```
        musicSource.Play();
    }

    // ============================================================
    //              MÉTODOS DE AJUSTES (ON/OFF)
    // ============================================================

    // Conecta esto al evento 'OnValueChanged' de tu Toggle de Música
    public void ToggleMusic(bool isOn)
    {
        // Si está encendido -> 0 dB (Volumen normal)
        // Si está apagado   -> -80 dB (Silencio absoluto en el Mixer)
        float volumeDb = isOn ? 0f : -80f;

        mainMixer.SetFloat("MusicVol", volumeDb);
    }

    // Conecta esto al evento 'OnValueChanged' de tu Toggle de SFX
    public void ToggleSFX(bool isOn)
    {
        float volumeDb = isOn ? 0f : -80f;

        mainMixer.SetFloat("SFXVol", volumeDb);
    }
}
```

Escena de Introducción

Fase de análisis:

Historia de usuario HU25:

Quién: Como Jugador
Qué: Necesito ver una secuencia de 5 imágenes narrativas, cada una con su propia duración específica, acompañadas de un audio continuo
Para qué: para que pueda entender la historia de Finewood antes de llegar al menú principal.

Tabla EPS

Entrada	Proceso	Salida



1. Lista de objetos `IntroFrame` (Cada uno tiene: `Sprite` imagen, `float` duración). 2. `AudioClip` de la narración completa.	1. **(Start):** El `AudioManager` inicia la música/narración. Se inicia la Corrutina. 2. **(Bucle):** La corrutina recorre la lista. 3. **(Paso):** Pone la imagen en el Canvas -> Espera `frame.duracion` segundos -> Pasa al siguiente.	Cambio visual de imágenes sincronizado y carga final de la escena `MenuPrincipal`.

Fase de diseño:

Diagrama de clases:

Lista de Tareas HU25

- **Escena:**
  - Crear la escena `Intro_Scene`.
  - Crear un Canvas con un objeto `Image` (fondo negro) y otro objeto `Image` (el visor de la cinemática).
- **Script `IntroManager.cs`:**
  - Variables: `AudioSource`, `Image visor`, `Sprite[] imagenes` (Array de 5 espacios)
  - **`Start()`:** `StartCoroutine(RutinaIntro());`.
  - **`IEnumerator RutinaIntro()`:**
    - `visor.sprite = imagenes[0];`
    - `yield return new WaitForSeconds(12f);` *(Aquí escribes a mano el tiempo del audio)*.
    - `visor.sprite = imagenes[1];`
    - `yield return new WaitForSeconds(8f);`
    - ... *(Repetir para las 5 imágenes)*.
    - `SceneManager.LoadScene(escenaSiguiente);`.
- **Configuración:**
  - Arrastrar los 5 sprites al array en el Inspector.
  - Ajustar los números en el código (`WaitForSeconds`) escuchando el audio hasta que calce bien.



Criterios de Aceptación HU25

1. Al dar Play, suena la narración.
2. Las imágenes cambian exactamente en los segundos que escribiste en el código.
3. Al terminar la última espera, se carga el Menú Principal.
1.

Codificación HU25

Script IntroManager

```
using UnityEngine;
using UnityEngine.UI;
using System.Collections;
using UnityEngine.SceneManagement;

public class IntroManager : MonoBehaviour
{
    [SerializeField] private AudioClip narracion;
    [SerializeField] private AudioClip music;
    [SerializeField] private Image visor;
    [SerializeField] private Sprite[] imagenes;
    [SerializeField] private Animator animatorFader;


    void Start()
    {
        StartCoroutine(CinematicaInicio());
    }

    private IEnumerator CinematicaInicio()
    {
        AudioManager.Instance.PlayMusic(music);
        visor.sprite = imagenes[0];
        yield return new WaitForSeconds(1f);
        AudioManager.Instance.PlaySFX(narracion);
        yield return new WaitForSeconds(20f);
        animatorFader.SetTrigger("FadeIn");
        yield return new WaitForSeconds(2f);
        visor.sprite = imagenes[1];
        animatorFader.SetTrigger("FadeOut");
        yield return new WaitForSeconds(18f);
        animatorFader.SetTrigger("FadeIn");
        yield return new WaitForSeconds(2f);
        visor.sprite = imagenes[2];
        animatorFader.SetTrigger("FadeOut");
```






```
        yield return new WaitForSeconds(19f);
        animatorFader.SetTrigger("FadeIn");
        yield return new WaitForSeconds(2f);
        visor.sprite = imagenes[3];
        animatorFader.SetTrigger("FadeOut");
        yield return new WaitForSeconds(20f);
        animatorFader.SetTrigger("FadeIn");
        yield return new WaitForSeconds(2f);
        visor.sprite = imagenes[4];
        animatorFader.SetTrigger("FadeOut");
        yield return new WaitForSeconds(27f);
        animatorFader.SetTrigger("FadeIn");
        yield return new WaitForSeconds(2f);
        SceneManager.LoadScene("Nivel_1_Arbol");
    }
}
```